PROCESAREA DIGITALĂ A SEMNALELOR

Teorie și aplicații

Gabriel V. Iana

EDITURA UNIVERSITĂȚII DIN PITEȘTI
2015

Cuprins

1 Operarea cu secvențe numerice 7
1.1 Scopul lucrării 7
1.2 Noțiuni teoretice 7
1.2.1 Semnale digitale 7
1.2.2 Mediul de programare MATLAB 12
1.3 Aplicații ale secvențelor numerice 20
1.3.1 Eșantionarea semnalelor 20
1.3.2 Teorema eșantionării 20
1.3.3 Generarea secvențelor elementare 21
1.3.4 Generarea secvențelor complexe 23
1.3.5 Operații simple cu secvențe 23
1.4 Desfășurarea lucrării 27
2 Operarea și realizarea de semnale cu microcontrolerul de semnal dsPIC33FJ256GP506 29
2.1 Scopul lucrării 29
2.2 Prezentarea mediului de programare MPLAB IDE 29
2.3 Instrucțiuni de implementare a unei aplicații cu sistemul de dezvoltare MPLAB IDE 29
2.4 Prezentarea microcontrolerului și a platformei de dezvoltare 34
2.5 Desfășurarea lucrării 40
3 Implementarea de algoritmi rapizi pentru transformata Fourier 49
3.1 Scopul lucrării 49
3.2 Noțiuni teoretice 49
3.2.1 TFR cu decimare în timp 51

3.2.2 TFR cu decimare în frecvență 55
3.3 Aplicații 57
3.3.1 Implementarea de algoritmi pentru TF 57
3.3.2 Influența tipului ferestrei asupra spectrului estimat cu TFD 62
3.3.3 Analiza în domeniul frecvență a unui semnal folosind microcontrolerul dsPIC33FJ256GP506 65
3.4 Desfășurarea lucrării 75
4 Aplicații ale sistemelor în timp discret 76
4.1 Scopul lucrării 76
4.2 Noțiuni teoretice 76
4.2.1 Prelucrarea semnalelor în timp discret 76
4.2.2 Modelarea matematică a sistemelor numerice 77
4.2.3 Proprietăți generale ale sistemelor numerice 77
4.2.4 Analiza sistemelor numerice liniare și invariante în timp 80
4.2.5 Exemple de sisteme numerice simple 83
4.3 Aplicații 87
4.3.1 Sisteme numerice liniare 1D 87
4.3.2 Funcția de transfer, polii și zerourile unui sistem numeric 88
4.3.3 Funcția pondere a unui sistem numeric 88
4.3.4 Stabilitatea sistemelor numerice 89
4.3.5 Efectul zerourilor și polilor asupra funcției de transfer a unui sistem numeric 91
4.3.6 Implementarea unui sistem numeric pe microcontrolerul dsPIC33FJ256GP506 91
4.4 Desfășurarea lucrării 98
5 Proiectarea și implementarea filtrelor FIR 100
5.1 Scopul lucrării 100

5.2 Noțiuni teoretice 100
5.2.1 Funcția de transfer a unui filtru numeric 100
5.2.2 Avantajele filtrelor numerice 101
5.2.3 Etapele proiectării unui filtru numeric 101
5.3 FN cu răspuns finit la impuls 102
5.3.1 Proiectarea FN-FIR prin metoda ferestrelor (sau metoda seriei Fourier) 102
5.3.2 Proiectarea FN-FIR prin metoda eșantionării în frecvență 103
5.3.3 Proiectarea FN-FIR prin optimizare 103
5.4 Aplicații FN-FIR în MATLAB 104
5.4.1 Implementarea FN-FIR prin metoda ferestrelor 104
5.4.2 Implementarea FN-FIR prin metoda eșantionării în frecvență 106
5.4.3 Implementarea FN-FIR prin optimizare folosind algoritmul Remez elaborat de Parks-McClellan 108
5.5 Desfășurarea lucrării 118
6 Proiectarea și implementarea filtrelor IIR 119
6.1 Scopul lucrării 119
6.2 Noțiuni teoretice 119
6.2.1 Funcția de transfer a unui filtru numeric 119
6.3 FN cu răspuns infinit la impuls 120
6.3.1 Proiectarea FN-IIR prin metoda aproximării numerice a ecuației diferențiale ce caracterizează un FA 122
6.3.2 Proiectarea FN-IIR prin metoda invarianței la impuls unitate 122
6.3.3 Proiectarea FN-IIR prin metoda transformării biliniare 122
6.3.4 Proiectarea FN-IIR prin metode de optimizare 122
6.4 Aplicații FN-IIR în MATLAB 123
6.4.1 Implementarea FN prin aproximare în planul z 124

6.4.2 Implementarea FN-IIR prin metoda invarianței la impuls 126
6.4.3 Implementarea FN-IIR prin transformarea biliniară 127
6.4.4 Implementarea FN-IIR prin optimizarea în planul *z* 128
6.5 Implementarea unui filtru IIR pe dsPIC33FJ256GP506 129
6.6 Desfășurarea lucrării 136
Bibliografie 138

1 Operarea cu secvențe numerice

1.1 Scopul lucrării

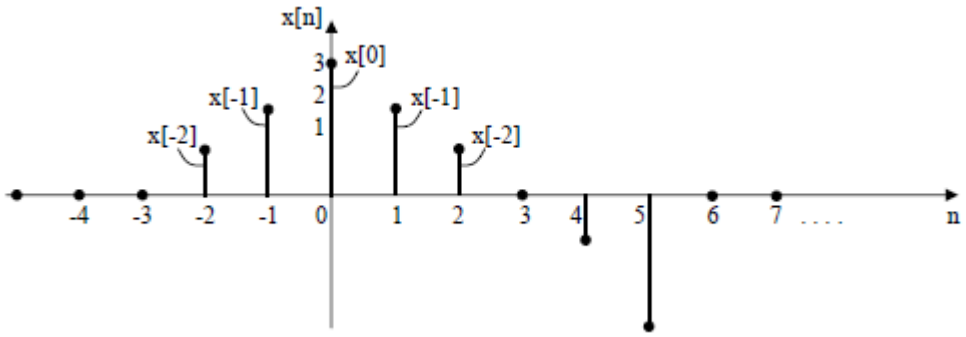
Lucrarea are ca scop însușirea, prin aplicații, amodului de reprezentare al semnalelor din timp continuu în timp discret. În cadrul lucrării se vor realiza programe prin care se aplică operații elementare asupra semnalelor în timp discret. De asemenea, se realizează o scurtă recapitulare despre mediul de programare MATLAB, dar cu accent pe funcțiile de reprezentare și prelucrare a semnalelor digitale.

1.2 Noțiuni teoretice

1.2.1 Semnale digitale

Un semnal în timp discret este reprezentat ca o secvență de numere, numite **eșantioane**. Valoarea unui eșantion al unui semnal în timp discret, $\{x[n]\}$ este notată ca $x[n]$, argumentul *n* fiind un număr întreg în intervalul $(-\infty, \infty)$.

De exemplu, o secvență de date are valorile $\{\ldots, 0, 1, 2, 3, 2, 1, 0, -1, -2, \ldots\}$ corespunzătoare momentelor discrete de timp (momentelor de eșantionare) $n = \ldots, -3, -2, -1, 0, 1, 2, 3, 4, 5, \ldots$, așa cum se prezintă în figura de mai jos:



Semnalul în timp discret poate fi o secvență cu lungime finită sau infinită. O *secvență cu lungime finită* este definită doar pentru un interval finit de timp:

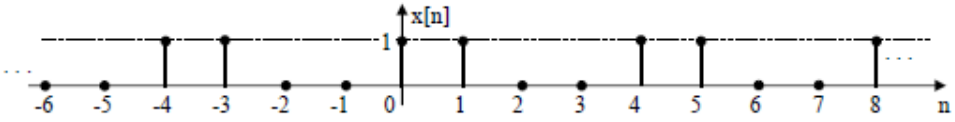
$N_1 \le n \le N_2$, unde N_1, N_2 sunt numere întregi finite

N_1și N_2 sunt numere întregi finite. În acest caz, secvența are lungimea $N = N_2 - N_1 + 1$.

O secvență $x[n]$ este *periodică* cu perioada *N* dacă îndeplinește relația:

$x[n] = x[n + kN]$, unde*N* întreg pozitiv, iar *k* întreg oarecare

Exemplu de secvență periodică:

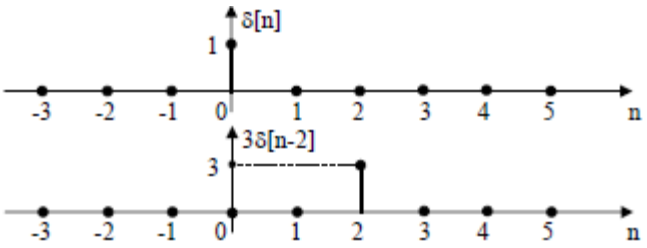


Secvențe elementare:

- *Secvența impuls unitate* – se notează cu $\delta[n]$ și este definită ca:

$$\delta[n] = \begin{cases} 1, \text{ pentru } n = 0 \\ 0, \text{ pentru } n \neq 0 \end{cases}$$

Secvența $3\delta[n-2]$ se reprezintă astfel:



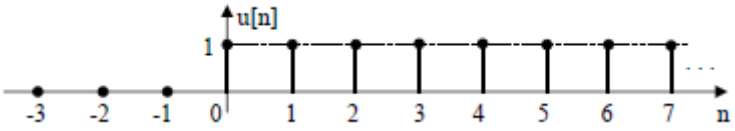
Un semnal oarecare $x[n]$ poate fi reprezentat cu ajutorul impulsurilor Dirac:

$$x[n] = \sum_i a_i \cdot \delta[n-i] = ... + a_0 \cdot \delta[n] + a_1 \cdot \delta[n-1] + a_2 \cdot \delta[n-2] + ...$$

- *Secvența treaptă unitate* – se notează cu $u[n]$ și este definită ca:

$$u[n] = \begin{cases} 1, \text{ pentru } n \geq 0 \\ 0, \text{ pentru } n < 0 \end{cases}$$

Reprezentarea secvenței treaptă unitate:



Se poate scrie că $u[n]=\sum_{i=0}^{\infty}1\cdot\delta[n-i]=\delta[n]+\delta[n-1]+\delta[n-2]+....$

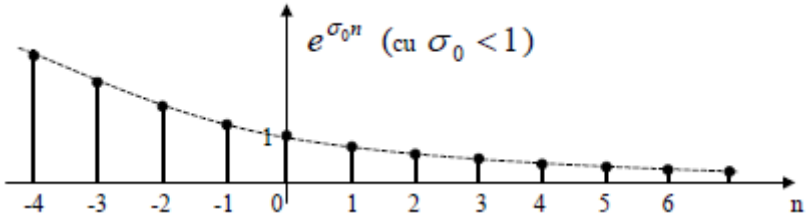
- *Secvența exponențială* – este definită ca:

$$x[n]=A\cdot\alpha^n,$$

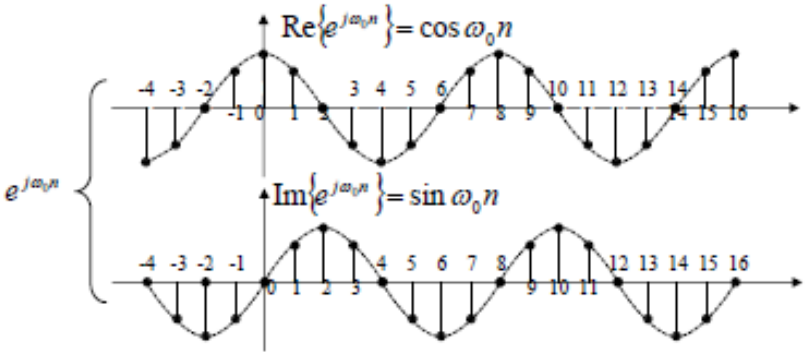
unde *A* și α sunt numere reale și complexe care pot fi exprimate prin relațiile:

$$\alpha=e^{\sigma_0+j\omega_0} \text{ și } A=|A|e^{\phi}, \text{ cu } \sigma_0,\omega_0 \text{ reale}$$

Secvența reală $e^{\sigma_0 n}$ se reprezintă astfel:



Secvența pur imaginară $e^{j\omega_0 n}=\cos(\omega_0 n)+j\sin(\omega_0 n)$ poate fi reprezentată prin:



- *Secvența sinusoidală reală cu amplitudine constantă* – este de forma:

$$x[n]=A\cos(\omega_0 n+\phi), \text{ unde } A,\omega_0,\phi \text{ reale}$$

A se numește *amplitudine*, ω_0 este *frecvența unghiulară*, iar ϕ reprezintă *faza inițială*.

Frecvența unui semnal periodic este:

$$f_0 = \frac{\omega_0}{2\pi}$$

Secvența exponențială complexă din $\sigma_0 = 0$ și secvența sinusoidală sunt secvențe periodice dacă $\omega_0 N$ este un multiplu întreg al lui 2π , adică:

$\omega_0 N = 2k\pi$, unde *N* este întreg pozitiv, iar *k* este întreg oarecare

Cel mai mic număr *N* posibil care satisface această condiție reprezintă *perioada secvenței*.

Suma a două secvențe $x[n]$ și $h[n]$ de lungime *N* este secvența $y[n]$, de lungime *N*, dată de relația:

$$y[n] = x[n] + h[n]$$

Produsul a două secvențe $x[n]$ și $h[n]$ de lungime *N* este secvența $y[n]$, de lungime *N*, dată de relația:

$$y[n] = x[n] \cdot h[n]$$

Înmulțirea cu un scalar, A, a unei secvențe $x[n]$ de lungime *N,* are ca rezultat secvența $y[n]$, de lungime *N*, dată de relația:

$$y[n] = A \cdot x[n]$$

Inversarea în timp a unei secvențe $x[n]$ de lungime infinită are ca rezultat o secvență $y[n]$ de lungime infinită, definită prin:

$$y[n] = x[-n]$$

Întârzierea unei secvențe $x[n]$ de lungime infinită cu un număr întreg pozitiv *M,* are ca rezultat secvența $y[n]$ de lungime infinită, dată de:

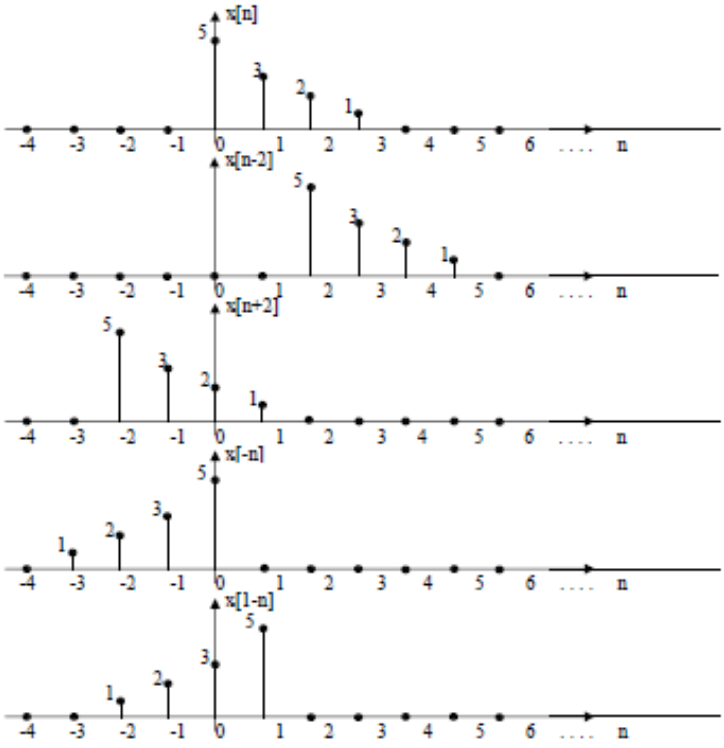
$$y[n] = x[n - M]$$

Dacă *M* este un număr întreg negativ, atunci se realizează o *avansare în timp* a secvenței $x[n]$.

O secvență $x[n]$ de lungime *N* poate fi *adăugată* unei alte secvențe $g[n]$ de lungime *M,* având ca rezultat o secvență mai lungă $y[n]$, de lungime *N+M,* dată de relația:

$$\{y[n]\} = \{\{x[n]\}, \{g[n]\}\}$$

Exemple de operații elementare aplicate secvenței $x[n] = [5, 3, 2, 1, 0, 0, ...]$:



Energia unei secvențe $x[n]$ este definită prin:

$$\varepsilon = \sum_{n=-\infty}^{\infty} |x[n]|^2$$

Energia unei secvențe pe un interval finit $[-K, K]$, este definită de:

$$\varepsilon = \sum_{n=-K}^{K} |x[n]|^2$$

Puterea medie a unei secvențe neperiodice $x[n]$ este definită de:

$$P_{av} = \lim_{K\to\infty} \frac{1}{2K+1} \varepsilon_K = \lim_{K\to\infty} \frac{1}{2K+1} \sum_{n=-K}^{K} |x[n]|^2$$

Puterea medie a unei secvențe periodice $x[n]$ cu perioada *N* este dată de:

$$P_{av} = \frac{1}{N}\sum_{n=0}^{N-1}\left|x[n]\right|^2$$

1.2.2 Mediul de programare MATLAB

Mediul software de simulare MATLAB[1] (*Matrix Laboratory*) este un pachet de programe dedicat calculului numeric și reprezentărilor grafice în domeniul științei inginerești. El prezintă avantajul integrării analizei numerice, calculului matriceal, procesării semnalului și reprezentărilor grafice într-un mediu ușor de învățat și utilizat, în care enunțurile problemelor și rezolvările acestora sunt exprimate în modul cel mai natural posibil, așa cum sunt scrise matematic, fără a fi necesară programarea tradițională. MATLAB permite manipularea matricelor, vizualizarea funcțiilor, implementarea algoritmilor, crearea de interfețe și poate interacționa cu alte aplicații. Chiar dacă e specializat în calcul numeric, există pachete software care îi permit să interacționeze cu motoarele de calcul simbolic gen Maple. Un pachet adițional, Simulink, oferă posibilitatea de a realiza simulări ale sistemelor dinamice, descrise utilizând modele matematice.

Informații generale. Funcții de control general

După lansarea în execuție, programul MATLAB intră în modul de comandă, afișând prompterul >>, și așteaptă introducerea unei comenzi de către utilizator.

De exemplu, comanda:

```
>> v=1:10
```

va crea variabila *v* afișând cele 11 elemente ale vectorului linie *v*, de la *v[1]=0* la *v[11]=10*.

Atunci când se operează în MATLAB este indicat să se țină seamă de următoarele recomandări:

- mediul de programare MATLAB este sensibil la tipul de litere (mari sau mici), dar există comenzi care fac trecerea între modurile sensibil și nesensibil;
- numele de funcție este obligatoriu să fie scris cu litere mici;
- liniile de comentariu dintr-un fișier script/funcție sunt precedate de caracterul %;

[1]Abrevierea a fost propusă de către CliveMoler, președintele departamentului de informatică al Universității din New Mexico, SUA, la sfârșitul anilor ’70. El este cel care a dezvoltat prima versiune de MATLAB.

- pentru ajutor se tastează *help* pentru meniul întreg sau *help* urmat de denumirea funcției sau fișierului *.m*. (exemplu: `>> help fft`).
- numărul de cifre zecimale în care sunt afișate diverse variabile nu reprezintă precizia în care sunt efectuate calculele. Pentru a schimba formatul afișat se tastează *format short e* pentru 5 cifre zecimale, *format long e* pentru 15 cifre zecimale și *format bank* pentru plasarea a două cifre zecimale la dreapta punctului zecimal. Același lucru se poate face din meniu, selectând *File → Preferences → General*, iar aici formatul dorit;
- *cd* apelează directorul curent;
- *dir* listează fișierele din directorul curent;
- *dirnume_director*afișează fișierele din directorul *nume_director*.
- comenzile *who*și *whos* furnizează numele variabilelor ce au fost definite în spațiul de lucru MATLAB:
    - *who* afișează variabilele curente din memorie;
    - *whos* afișează variabilele, dimensiunile lor, precum și tipul acestora (reale sau complexe).
- *what*afișează fișierele *.m, .mat* si *.mex* din directorul curent;

Definirea variabilelor

Variabilelor li se atribuie valori numerice, tipărindu-se direct expresia numerică. De exemplu:

```
>> a=1+2

a =    3
```

Rezultatul nu se afișează dacă se pune punct și virgulă la sfârșitul expresiei, de exemplu:

```
>> a=1+2;
```

În MATLAB se utilizează următorii operatori aritmetici:

Operator aritmetic	Operație
+	Adunare
-	Scădere
*	Multiplicare
/	Împărțire
^	Ridicare la putere
‘	Transpunere

Unei variabile i se poate atribui o formulă ce utilizează operatorii menționați anterior și una sau mai multe mărimi definite anterior chiar în cadrul comenzii curente. De exemplu, presupunând că *a* este definită anterior:

```
>> b=3^a
```

va returna valoarea:

```
b=     27
```

Există variabile predefinite:*i=sqrt(-1), pi=3,1416...*, etc.

Pentru definirea variabilelor există un număr de funcţii predefinite, dintre care, cele mai uzuale pentru prelucrarea semnalelor sunt prezentate în tabelul următor:

Funcţie	Rezultat
abs	Valoarea absolută
angle	Faza ataşată unui număr complex, în radiani
cos	Funcţia cosinus, cu argumentul în radiani
sin	Funcţia sinus, cu argumentul în radiani
exp	Funcţia exponenţială

De exemplu:

```
>> y=2*(1+4*j)
>>c=abs(y)
```

conduce la:

```
c=
      8.2462
```

Expresia:

```
>>u=angle(y)
```

conduce la:

```
u=
      1.3258
```

Definirea scalarilor, vectorilor şi matricelor

MATLAB este un pachet de programe care lucrează numai cu un singur tip de obiecte, matrice numerice rectangulare, cu elemente reale sau complexe. În acest sens, scalarii sunt asimilaţi matricelor cu o linie şi o coloană (1×1), iar vectorii sunt asimilaţi matricelor cu o linie (1×n) sau o coloană (n×1).

Introducerea matricelor în mediul de lucru se face prin una din metodele:

- introducerea explicită;
- generarea prin instrucţiuni şi funcţii;
- crearea de fişiere *.m*;
- încărcarea din fişiere de date externe.

Cea mai simplă metodă constă în utilizarea unei liste explicite. Trebuie respectate următoarele reguli:

- elementele unei linii trebuie separate prin spaţii libere sau virgulă;
- liniile se separă prin punct şi virgulă „;”;

- elementele matricei sunt cuprinse între paranteze drepte „[]”.

Elementele matricelor pot fi numere reale sau complexe sau orice altă variabilă MATLAB.

Elementele unei matrice *A* pot fi identificate, în MATLAB, prin notația *A(i,j)*și semnifică elementul de la intersecția liniei *i* cu coloana *j*. Pentru a face referire la un element al matricei sunt necesari doi indici, iar referirea la un element al unui vector se face cu un singur indice.

De exemplu, matricele sunt definite introducând elementele linie cu linie:

```
>>A=[1 2 3; 3 6 8]
```

Această comandă creează matricea:

```
A =
     1     2     3
     3     6     8
```

Comanda `>>A(2,1)` afișează rezultatul:

```
ans =
     3
```

Se remarcă faptul că dacă se atribuie o valoare unui element care ocupă o poziție în afara dimensiunii maxime a unei matrice sau unui vector, dimensiunea acesteia/acestuia este mărită automat până la valoarea indicelui noului element, iar elementele nedefinite sunt setate la valoarea zero.

Comanda `>>A(2,5)=4`creează matricea:

```
A =
     1     2     3     0     0
     3     6     8     0     4
```

Comanda `>>size(A)` returnează dimensiunea matricei *A*.

Se definesc mai multe matrice particulare:

- matricea vidă:`>>A=[ ];`
- matricea nulă de ordin n×m: `>> A=zeros(n,m);`
- matricea unitară de ordin n×m: `>>A=ones(n,m);`
- matricea cu diagonală principală 1, de ordin n×n: `>>A= eye(n).`

Vectorii pot fi definiți în două moduri:

- introducerea explicită a componentelor vectorului;

Comanda:

```
>> v=[1 2 3 4 5]
```

creează un vector cu o linie și cinci coloane.

Comanda:

```
>>v(6)=6
```

conduce la vectorul v=[1 2 3 4 5 6].

- cu elemente egal spațiate prin comanda:

```
>>k=amin:pas:amax
```

unde *amin* este valoarea primului element din vector, *pas* este valoarea incrementului, iar*amax* limita până la care pot lua valori componentele vectorului. Dacă incrementul lipsește, atunci implicit valoarea sa este 1.

De exemplu:

```
>>k=0:0.5:5.1
```

creează un vector cu elementele 0, 0.5, 1, 1.5,....5.

Comanda `>>length(v)` returnează lungimea vectorului *v*.

Operații cu matrice

Fie *X*și *Y* două matrice. Se definesc următoarele operații:

Operație	Descriere
Z=X+Y	Adunarea definită pentru cazul în care dimensiunile matricei *X* sunt egale cu dimensiunile matricei *Y*
Z=X-Y	Scăderea definită pentru cazul în care dimensiunile matricei *X* sunt egale cu dimensiunile matricei *Y*
Z=X*Y	Înmulțirea, definită atunci când *X* are dimensiunile (m×n), iar *Y* (n×p)
Z=X/Y	Împărțirea la dreapta, definită atunci când *X* are dimensiunile (n×n), iar *Y* (n×n), și este identică cu $X*Y^{-1}$ (Y^{-1} este inversa matricei *Y*).
Z=X.*Y	Înmulțirea element cu element (între componentele a două matrice de aceleași dimensiuni)
Z=X\Y	Împărțirea la stânga, și este identică cu X*inv(Y) (inv(Y) este inversa matricei *Y*)
Z=X^p	Ridicarea la putere, și reprezintă ridicarea la puterea *p* a matricei *X*. Expresia X^p are sens numai pentru matrice pătratice și *p* scalar.
Z=p.^X	Ridicarea la putere a elementelor unui vector/matrice
Z=X’	Calculul matricei transpuse

Operații cu vectori

Operațiile cu vectori se efectuează prin particularizarea regulilor de la operații cu matrice impunând ca una din dimensiuni sa fie egală 1, acolo unde dimensiunea permite acest lucru.

Produsul scalar a doi vectori se poate calcula în următoarele moduri:

```
>>z=sum(a.*b)
```

dacă vectorii *a*și *b* sunt amândoi fie vectori coloană, fie vectori linie, sau:

```
>>z=(a*b')
```

dacă *a*și *b* sunt vectori linie.

Observație. Vectorii *a*și *b* trebuie să fie de aceeași dimensiune.

Construirea unei funcții

Mediul de programare MATLAB oferă posibilitatea creării unei funcții ce folosește funcții MATLAB, funcții definite anterior, operatori predefiniți etc. Numele fișierului în care este scrisă funcția trebuie să fie identic cu numele funcției. De asemenea, prima linie a fișierului trebuie să conțină sintaxa caracteristică definiției unei funcții:

```
functionvar_returnate = nume_functie(var_intrare)
```

De exemplu, o funcție ce realizează suma a două variabile *x*și *y*, rezultatul fiind returnat în variabila *z*, se realizează astfel:

```
function z=suma(x,y);
z=x+y;
```

Fișierul astfel scris se salvează sub aceeași de numire a funcției, în cazul nostru *suma.m*. Funcția definită anterior poate fi apelată din linia de comenzi MATLAB, dintr-un script sau dintr-o altă funcție. De exemplu:

```
>>a=1;b=2; c=suma(a,b);
```

Reprezentări grafice elementare

Funcțiile de bază MATLAB pentru reprezentările grafice sunt:

- **plot**;
- **loglog**;
- **semilogx**;
- **semilogy**.

Pentru reprezentarea graficelor în coordonate liniare se utilizează funcția plot.

Funcția ***plot*** se apelează se poate apela în mai multe moduri:

- **plot(y)**;

Dacă argumentul *y* este complex, plot(y) este echivalent cu: **plot(real(y), imag(y))**.

Dacă *y* este un vector real (linie sau coloană), atunci funcția plot trasează graficul y=y[i], unde i=1,2,…L, este numărul de ordine al elementului *y*, *L* fiind lungimea vectorului *y*.

- **plot(x,y)** reprezintă vectorul *y* funcție de vectorul *x*.

Dacă *x* este vector, iar *y* este matrice, atunci coloanele lui *y* sunt trasate în funcție de vectorul *x*, rezultatul fiind reprezentarea mai multor grafice în aceeași fereastră grafică.

Dacă *x*și *y* sunt matrice de aceeași dimensiune, atunci se reprezintă coloanele lui *y* în funcție de coloanele lui *x*.

- **plot(x,y,'*linie tip*')**,*linie tip*fiind o succesiune de caractere care specifică tipul liniei cu care este trasat graficul;
- **plot(x1,y1,x2,y2,…)** reprezintă simultan mai multe grafice, în același sistem de coordonate și în aceeași fereastră grafică.

Graficele se pot trasa utilizând linii și indicatoare de diferite culori. Acest lucru este prezentat în tabelul următor.

Linii tip		Indicatori
Continuă	-	+
Întreruptă	--	*
Două puncte	:	o
Linie-punct	-.	x

Culori	Cod MATLAB
Galben	y
Mov	m
Turcoaz	c
Roșu	r
Verde	g
Albastru	b
Alb	w
Negru	k

Reprezentarea discretă a datelor

Reprezentarea discretă a datelor se face cu funcția *stem* sub forma unor linii terminate cu cerculeț. Se apelează cu una din sintaxele:

- **stem(y)** – trasează un grafic y=y[i], i=1,2,3,....L, din linii terminate cu cerculeț;
- **stem(x,y)** – trasează un grafic cu linii terminate cu cerculeț, cu locațiile specificate de vectorul *x*. Valorile lui *x* trebuie să fie egal spațiate;
- **stem(x,y,'*linietip*')** este similară funcției plot(x,y,'*linie tip*'), cu deosebirea că se trasează graficul cu linii terminate cu cerculeț.

Observație. Atât la funcția *plot*, cât și la funcția *stem*, lungimile vectorilor de pe abscisă și ordonată trebuie sa fie egale.

Personalizarea graficelor

Pentru plasarea în câmpul graficelor a unor texte, etichete ale axelor, precum și a titlului se utilizează următoarele funcții:

- **title('*text*')** – comandă prin care titlul graficului se plasează deasupra acestuia; 'text' este un șir de caractere care reprezintă titlul graficului;
- **xlabel('*text*')** – precizează eticheta axei *x*; '*text*' este un șir de caractere care reprezintă numele axei, unitatea de măsură etc.;
- **ylabel('*text*')** – precizează eticheta axei *y*; '*text*' fiind un șir de caractere care reprezintă numele axei, unitatea de măsură etc.;
- **grid on** – trasează o rețea de linii orizontale și verticale pe grafic;
- **grid off** – elimină rețeaua de linii orizontale și verticale trasate pe grafic de grid on.

Exemplu

Să se traseze grafic funcția $f(t)=\cos(2\pi t\cdot 100)$, folosind culoarea albastră și linie întreruptă, și $g(t)=-f(t)$ cu indicatori de tip *și culoare roșie. Să se scrie titlul „*Graficele functiilor f(t) si g(t)*". Pe axa *x* să se scrie „*t*", iar pe axa *y* să se scrie „*f(t) si g(t)*".

```
t =0 :.001:0.04;
f = cos(2*pi*100*t);
g = -f;
plot(t,f,'--b',t,g,'*r'); grid on;
title('Graficele functiilor f(t) si g(t)');
xlabel('t'); ylabel('f(t) si g(t)');
```

1.3 Aplicaţii ale secvenţelor numerice

1.3.1 Eşantionarea semnalelor

Să se realizeze codul MATLAB care afişează formele de undă pentru un semnal sinusoidal continuu şi discret in timp cu următorii parametri: semnalul în timp continuu are frecvenţa $F_0 = 1200Hz$, faza $\varphi_0 = \frac{\pi}{4} rad$ şi amplitudinea $A = 10$. Acest semnal se va eşantiona cu o frecvenţă $F_e = 16kHz$. Perioada de afişare a semnalelor este cuprinsă între $t_0 = 0$ şi $t_f = 5ms$.

```
Fe=16e3; t=0:1/Fe:5e-3; n=0:length(t)-1;
xa=10*sin(2*pi*1200*t+pi/4);
subplot(211); plot(t,xa);grid;
xlabel('timp continuu'); ylabel('amplitudine');
title('Semnal sinusoidal continuu');
xd= 10*sin(2*pi*(1200/16000)*n+pi/4);
subplot(212); stem(xd);grid;
xlabel('timp discret'); ylabel('amplitudine');
title(' Semnal sinusoidal discret');
```

1.3.2 Teorema eşantionării

1. Să se genereze un semnal de *0,5s* compus din suma a două sinusoide de frecvenţe 100*Hz*şi 156*Hz*, cu amplitudinea de 1*V*. Se consideră o frecvenţă de eşantionare de 256*Hz*. Reprezentaţi semnalul sumă. Comentaţi rezultatul.

```
n=(1:128); f1=100; f2=156;fe=256;
y=sin(2*pi*f1/fe*n)+sin(2*pi*f2/fe*n);
stem(n/fe,y); xlabel('timp (s)');grid;
ylabel('amplitudine (V)');
```

2. Să se genereze, să se reprezinte şi să se compare două sinusoide de frecvenţe 100*Hz*şi 356*Hz*şi amplitudinea de 1*V*. Se consideră o frecvenţă de eşantionare de 256*Hz*. Comentaţi rezultatul.

```
n=(1:100); f1=100; f2=356; fe=256;
y1=sin(2*pi*f1/fe*n);
y2=sin(2*pi*f2/fe*n);
subplot(211); stem(n,y1);grid;
ylabel('amplitudine (V)');
title('sinusoida de frecventa 100Hz')
subplot(212); stem(n,y2);grid;
xlabel('esantioane (n)'); ylabel('amplitudine');
title('sinusoida de frecventa 356Hz');
```

1.3.3 Generarea secvențelor elementare

Să se genereze și să se reprezinte grafic secvențele numerice unidimensionale elementare următoare:

a) Impuls Dirac $\delta[n-5]$;

```
figure();
a1 = axes('Position',[.1 .1 .8 .8]);
stem([zeros(1,14) 1 zeros(1,5)]);
set(a1,'YLim',[0 1.5],'XTick',[0:5:20],...
'XTickLabel',[-10:5:10],'FontSize',8);
legend('Impuls Dirac',2)
```

b) Semnal treaptă unitate $u[n]$;

```
figure();
a2 = axes('Position',[.1 .1 .8 .8]);
stem([zeros(1,10) ones(1,10)]);
set(a2,'YLim',[0 1.5],'XTick',[0:5:20],...
'XTickLabel',[-10:5:10],'FontSize',8);
legend('Semnal treapta unitate',2)
```

c) Semnal poartă $r[n-5]$;

```
figure();
a3 = axes('Position',[.1 .1 .8 .8]);
stem([zeros(1,10) ones(1,5) zeros(1,5)]);
set(a3,'YLim',[0 1.5],'XTick',[0:5:20],...
'XTickLabel',[-10:5:10],'FontSize',8);
legend('Semnal poarta',2)
```

d) Semnal sinusoidal în timp discret cu frecvența 1*kHz*, eșantionat la 10*kHz*, și cu faza inițială $\frac{\pi}{4} rad$ (semnalul are următoarea formulă: $\sin\left(2\pi\frac{1000}{10000}n+\frac{\pi}{4}\right)$);

```
figure();
a4 = axes('Position',[.1 .1 .8 .8]);
stem([zeros(1,10) ones(1,5) zeros(1,5)]);
set(a4,'YLim',[0 1.5],'XTick',[0:5:20],...
'XTickLabel',[-10:5:10],'FontSize',8);
legend('Semnal poarta',2)
```

e) Semnal sinus cardinal $\mathrm{sinc}[n]$;

```
figure();
a5 = axes('Position',[.1 .1 .8 .8]);
stem(sinc(.25*[-9:10]));
set(a5,'XLim',[0 20],'YLim',[-.25 1.5],'XTick',...
    [0:5:20],'XTickLabel',[-10:5:10],'FontSize',8);
legend('Semnal sinc',2)
```

f) Semnal exponențial e^{-n};

```
figure();
a6 = axes('Position',[.1 .1 .8 .8]);
stem(exp(-(0:20)));
set(a6,'YLim',[0 1.2],'XLim',[1
21],'XTick',[1:5:21],...
'XTickLabel',[0:5:20],'FontSize',8);
legend('Semnal exponential e^-^n',2)
```

g) Semnal putere $2^{\frac{-n}{2}}$;

```
figure();
a7 = axes('Position',[.1 .1 .8 .8]);
stem(pow2(-0.5*(0:20)))
set(a7,'YLim',[0 1.2],'XLim',[1
21],'XTick',[1:5:21],...
'XTickLabel',[0:5:20],'FontSize',8);
legend('Semnal putere 2^-^0^.^5^n',2)
```

h) Semnal logaritm natural $\ln n$;

```
figure();
a8 = axes('Position',[.1 .1 .8 .8]);
stem(log([.1:.1:2]))
```

```
set(a8,'YLim',[-3 3],'XLim',[0 20],'XTick',[0:5:20],...
'XTickLabel',[0:.5:2],'FontSize',8);
legend('Semnal logaritm natural',2)
```

i) Semnal aleatoriu cu repartiție normală cu media 1,5 și dispersia 0,25.

```
figure();
a9 = axes('Position',[.1 .1 .8 .8]);
stem(1.5+sqrt(0.25)*randn(1,20))
set(a9,'YLim',[0 4],'XLim',[0 20],'XTick',[0:5:20],...
'XTickLabel',[-10:5:10],'FontSize',8);
legend('Semnal aleator normal',2)
```

1.3.4 Generarea secvențelor complexe

Să se genereze semnalul $x(n) = K \cdot e^{c \cdot n}$, unde $K = 2, c = -\frac{1}{12} + j\frac{\pi}{6}, 0 \le n \le 40, n$ natural.

```
c = -(1/12)+(pi/6)*i;
K = 2; n = 0:40;
x = K*exp(c*n);
subplot(2,1,1);
stem(n,real(x));
xlabel(' Index temporal n');
ylabel('Amplitudine');
title('Parte reala');
subplot(2,1,2);
stem(n,imag(x));
xlabel(' Index temporal n');
ylabel('Amplitudine');
title('Parte imaginara');
```

1.3.5 Operații simple cu secvențe

a. Deplasarea și inversarea în timp a secvențelor

Se dă următoarea secvență de semnal $x[n] = \{0,5; 1; 1,5; 2; 2,5\}$. Să se realizeze următoarele operațiile: $x[n+2], x[n+1], x[-n], x[1-n], x[-2-n]$

```
figure();
n=-10:1:10;
x=[0.5 1 1.5 2 2.5];
subplot(2,3,1);
```

```
stem(n,[zeros(1,10) x zeros(1,6)]);
title('x[n]')
subplot(2,3,2);
stem(n,[zeros(1,8) x zeros(1,8)]);
title('x[n+2]')
subplot(2,3,3);
stem(n,[zeros(1,11) x zeros(1,5)]);
title('x[n-1]')
x=[2.5 2 1.5 1 0.5]
subplot(2,3,4);
stem(n,[zeros(1,6) x zeros(1,10)]);
title('x[-n]')
subplot(2,3,5);
stem(n,[zeros(1,7) x zeros(1,9)]);
title('x[1-n]')
subplot(2,3,6);
stem(n,[zeros(1,4) x zeros(1,12)]);
title('x[-2-n]')
```

b. Adunarea secvențelor

Se consideră două semnale sinusoidale cu frecvențele de 100*Hz*, respectiv 300*Hz*și amplitudinea 1*V*. Frecvența de eșantionare a acestora este de 3*kHz*. Să se realizeze afișarea acestor semnale în domeniul discret, cât și suma lor.

```
figure();
f1=100; f2=300; fe=3000;
n=1:30;
x=sin(2*3.14*f1/fe*n);
subplot(3,1,1);
stem(x);
title('x[n]')
y=sin(2*3.14*f2/fe*n);
subplot(3,1,2);
stem(y);
title('y[n]')
z= x + y;
subplot(3,1,3);
stem(z);
title('x[n]+y[n]')
```

c. Modularea (înmulțirea) secvențelor

Să se genereze semnalul modulat în amplitudine:

$$y(n)=\left[1+m\cdot\sin\left(2\pi f_b\cdot n\right)\right]\cdot\sin\left(2\pi f_h\cdot n\right),\ \text{unde}$$

$$m=0,4,\ f_b=0,01,\ f_h=0,1,0\le n\le 100,n\in\square$$

```
n = 0:100 ; m = 0.4 ; fh = 0.1 ; fb = 0.01 ;
xh = sin(2*pi*fh*n) ; xb = sin(2*pi*fb*n) ;
y = (1+m*xb).*xh ; stem(n,y) ; grid ;
xlabel(' Index temporal n') ; ylabel('Amplitudine');
```

d. Corelarea secvențelor

Să se realizeze corelarea dintre un semnal aleatoriu și copia sa întârziată cu trei eșantioane.

```
x = randn(30,1);
subplot(3,1,1); stem(x);
x1 = [0 00 x(1:27)'];
subplot(3,1,2); stem(x1);
r = xcorr(x, x1);
subplot(3,1,3); stem(r);
```

Întârziați semnalul cu 10 eșantioane și comentați rezultatul. Realizați aceeași operație asupra unui semnal de intrare sinusoidal.

Lista funcțiilor MATLAB des întâlnite

Funcție	Rezultat
abs	Modul
acos	Arccosinus
all	Verifică dacă toate valorile unei matrice sunt nule
angle	Faza unei valori complexe
any	Verifică dacă cel puțin o valoare dintr-o matrice este nenulă
asin	Arcsinus
atan	Arctangentă
axis	Modificarea axelor unui grafic
butter	Crearea unui filtru Butterworth
cd	Schimbarea directorului de lucru

ceil	Rotunjire în sus
cheby1	Crearea unui filtru Chebyshev de ordinul 1
cheby2	Crearea unui filtru Chebyshev de ordinul 2
colormap	Crearea unei hărți de culoare pentru afișarea suprafețelor
conv	Realizarea convoluției între două semnale
cos	Crearea unei forme de undă cosinusoidale
czt	Transformata Z
det	Calculul determinantului unei matrice
diag	Crearea unei matrice cu valori nenule pe diagonala principală
ellip	Crearea unui filtru eliptic
exist	Existența unei variabile
exit	Ieșire din MATLAB
exp	Crearea unei forme de undă exponențiale
fft	Transformata Fourier rapidă
filter	Filtrarea unui semnal
fiind	Găseşte valorile nenule ale unei matrice
fir1	Crearea unui filtru FIR
floor	Rotunjire în jos
freqz	Răspunsul în frecvență al unui filtru
grid	Afișarea liniilor ajutătoare pe grafic
help	Afișarea documentației asociate unei funcții
inv	Inversa unei matrice
isempty	Verifică dacă o matrice este vidă
mesh	Afișarea unei suprafețe prin model *wireframe*
meshgrid	Crearea matricelor din combinația unor vectori pentru afișarea suprafețelor
length	Lungimea unui vector
log	Logaritm natural
log10	Logaritm în baza 10
log2	Logaritm în baza 2
ones	Crearea unei matrice cu valori de 1
plot	Afișarea unui grafic
pwd	Afișarea directorului de lucru
sawtooth	Crearea unei forme de undă triunghiulare
sin	Crearea unei forme de undă sinusoidale

size	Dimensiunea unei matrice
sqrt	Radical
square	Crearea unei forme de undă dreptunghiulare
subplot	Crearea mai multor grafice pe o figură
surf	Afișarea unei suprafețe solide
rand	Crearea unei matrice cu numere aleatoare
rem	Restul împărțirii
repmat	Crearea unei matrice mai mari prin replicarea unei matrice
round	Rotunjire
tan	Tangentă
title	Modificarea titlului graficului
unwrap	Conectarea unui vector de faze
xlabel	Modificarea etichetei de pe axa *x*
ylabel	Modificarea etichetei de pe axa *y*
zeros	Crearea unei matrice cu valori de 0

Pentru o descriere mai detaliată a acestor funcții consultați documentația lor disponibilă folosind comanda *help*.

1.4 Desfășurarea lucrării

1. Se vor rula în mediul MATLAB toate aplicațiile descrise în breviarul teoretic și se va urmări execuția lor.

2. Să se genereze și să se reprezinte grafic secvențele numerice unidimensionale următoare:

- semnal sinusoidal cu frecvența de 3,2*kHz*, eșantionat la 10*kHz*, cu o fază inițială de ;
- semnal exponențial $e^{-0,75 \cdot n}$;
- semnalul $\lg[2n]$;
- semnal aleator cu repartiția normală, cu media 1,64 și dispersia 0,1.

3. Să se genereze și să se reprezinte în timp discret:

- două perioade dintr-un semnal sinusoidal cu frecvență de 8*kHz*, eșantionat cu frecvența de 30*kHz*;
- semnalul format din suma a două semnale sinusoidale de 2*kHz*, respectiv 3*kHz*, eșantionate la 13*kHz* într-o fereastră de 100 de eșantioane;

- semnal dreptunghiular cu perioada de $125\mu s$, eșantionat la $15kHz$.

4. Să se realizeze modulația în amplitudine având semnalul modulator $x[n]=\sin(2\pi\cdot 0,01\cdot n)+\cos(2\pi\cdot 0,03\cdot n)$ și semnalul modulat $s[n]=\sin(2\pi\cdot 0,2\cdot n)$, unde $0\le n\le 300, n\in$. Indicele de modulație *m* este 0,6.

5. Folosind programul MATLAB să se genereze următoarele semnale în timp:

- $x[n]=\sin(0,3\cdot n)$ într-o fereastră de 64 de eșantioane;
- $x[n]=\log(0,2\cdot n)$ într-o fereastră de 256 de eșantioane.

6. Să se realizeze în MATLAB convoluția dintre cele două semnale:

$$x[n]=\{1,2,3,4\} \text{ și } h[n]=\{0,5;1\}$$

7. Se consideră o secvență sinusoidală cu frecventa de $50kHz$și amplitudinea $1V$ ce este eșantionată cu o frecvență de $1MHz$. Acestei secvențe i se suprapune un zgomot cu amplitudinea de 0,5 după care este întârziată cu 10 eșantioane (adică $10\mu s$). Reprezentați grafic în MATLAB corelația celor două secvențe și comentați rezultatul obținut.

2 Operarea și realizarea de semnale cu microcontrolerul de semnal dsPIC33FJ256GP506

2.1 Scopul lucrării

Lucrarea are ca scop însușirea structurii și modului de lucru cu microcontrolerul dsPIC33F. În cadrul lucrării se va prezenta mediul de programare MPLAB IDE v8.21, și se va descrie modul în care se pot realiza aplicații cu dsPIC33F, cu ajutorul acestui program.

2.2 Prezentarea mediului de programare MPLAB IDE

Mediul de programare MPLAB IDE (*MPLAB Integrated Development Environment*) reprezintă un program software destinat dezvoltării aplicațiilor cu microcontrolere și microprocesoare Microchip, PIC (*ProgrammableInterfaceController*[2]) și dsPIC (*Digital Signal PIC*). Programul este denumit mediul de dezvoltare integrat, deoarece acesta oferă mai multe posibilități de a dezvolta coduri pentru microcontrolere.

MPLAB IDE reprezintă o varietate de instrumente hardware și software atât pentru programarea componentelor, cât și pentru configurarea unui întreg sistem.

Un circuit integrat este de obicei un sistem care se folosește de un microcontroler, cum ar fi PIC sau DSC (*Digital Signal Controller*). Aceste microcontrolere conțin atât microprocesor, cât și circuite periferice, constituindu-se în module de control de dimensiuni foarte mici. Aceste circuite pot fi integrate în alte aparate electronice și mecanice pentru dispozitive de control de cost redus.

2.3 Instrucțiuni de implementare a unei aplicații cu sistemul de dezvoltare MPLAB IDE

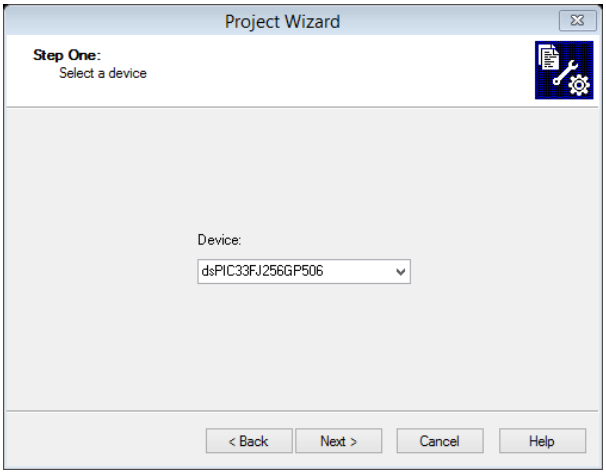
Pentru realizarea unei aplicații în MPLAB IDE trebuie câțiva pași de bază. O modalitate ușoară pentru a realiza acest lucru este folosirea aplicației *Project*

[2] Acronimul*PIC* semnifica inițial *PeripheralInterface Controller*. Ulterior, semnificația s-a schimbat în *Programmable Interface Controller*, găsindu-se chiar și cu sensul *ProgrammableIntelligent Computer*.

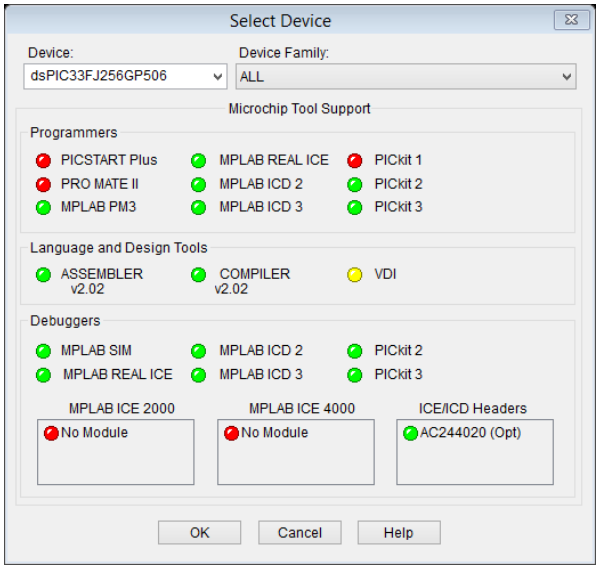
Wizard. După deschiderea programului MPLAB IDE se va accesa această aplicație din meniul *Project*. Paşii pentru realizarea unei aplicații sunt descrişi în continuare.

1. Selectarea dispozitivului

Se va selecta dispozitivul necesar aplicației dorite. Selecția dispozitivului trebuie însă făcută în etapa de proiectare a aplicației, în funcție de rolul acestuia.



O altă modalitate de a selecta dispozitivul dorit este prin accesarea panoului *Select Device* din meniul *Configure*.



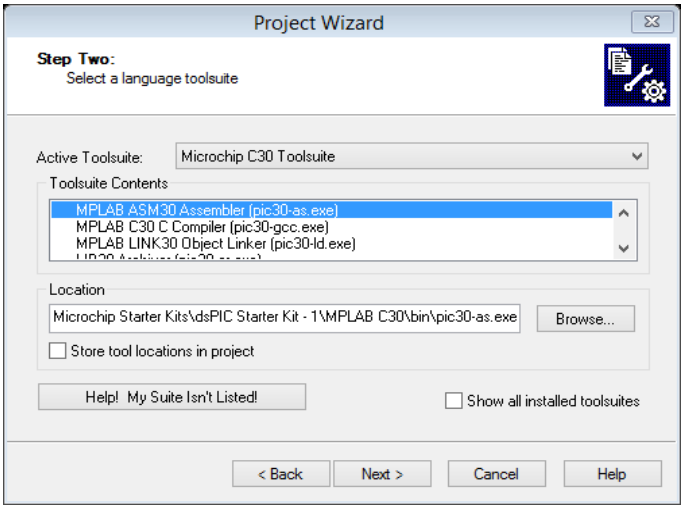
După cum se observă, se poate selecta și familia din care face parte dispozitivul. Culorile LED-urilor indică ce componente MPLAB IDE susțin acest dispozitiv. Semnificația culorilor este următoarea:

- verde – suport deplin;
- galben – suport preliminar pentru o parte viitoare a instrumentului particular al mediului MPLAB;
- roșu – niciun suport pentru dispozitiv. Sprijinul poate urmează a fi implementat sau poate fi nepotrivit pentru instrument.

În ambele cazuri se va selecta *dsPIC33FJ256GP506*.

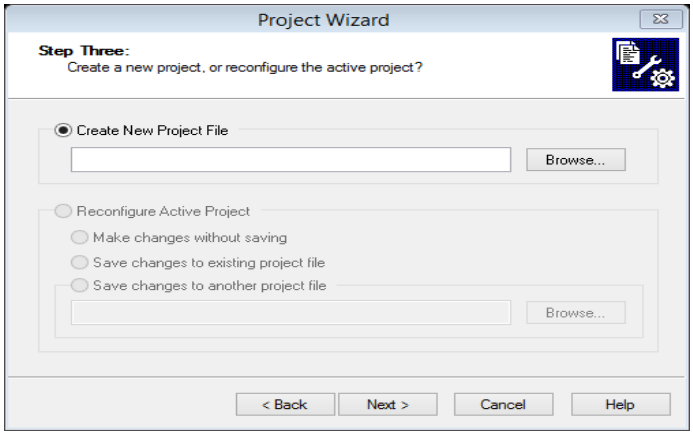
2. Selectarea compilatorului

Compilatorul (*LanguageToolsuite*) se alege din meniul aplicației *Project Wizard*. Pentru această aplicație vom folosi bibliotecile și compilatorul C30.



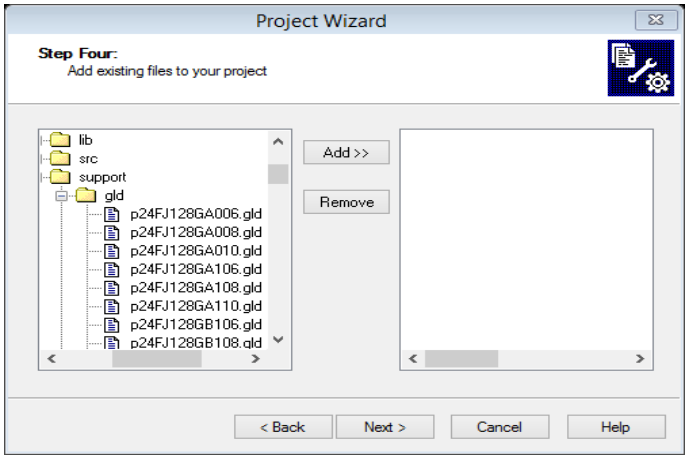
3. Crearea unui proiect

Se va alege locația în care se va salva proiectul: *D:\Projects\Laborator1*. În cazul în care directorul nu există, veți fi întrebat dacă doriți să îl creați. Selectați *OK*.



4. Adăugarea fișierelor în proiect

Se vor adăuga fișierele necesare acestei aplicații. De asemenea, pot fi folosite și fișiere care au fost utilizate în alte aplicații.



Un document sursă nu a fost selectat încă, așa că vom folosi fișiere standard MPLAB IDE. Acestea sunt fișiere care pot fi folosite pentru a începe un proiect. Din locația *D:\Laboratoare_Licenta\Lab_dsp_dsPIC\Include* se vor adăuga fișierele:

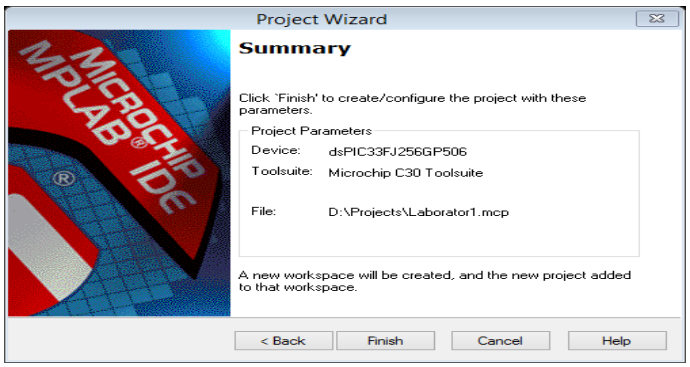
- *p33FJ256GP510.h*, din directorul *h*;

Fișierul antet (*header*) conține definiții pentru toțiregiștrii microcontrolerului. Acestea pot fi folosite la scrierea codului.

- *p33FJ256GP506.gld*, din directorul *gld*.

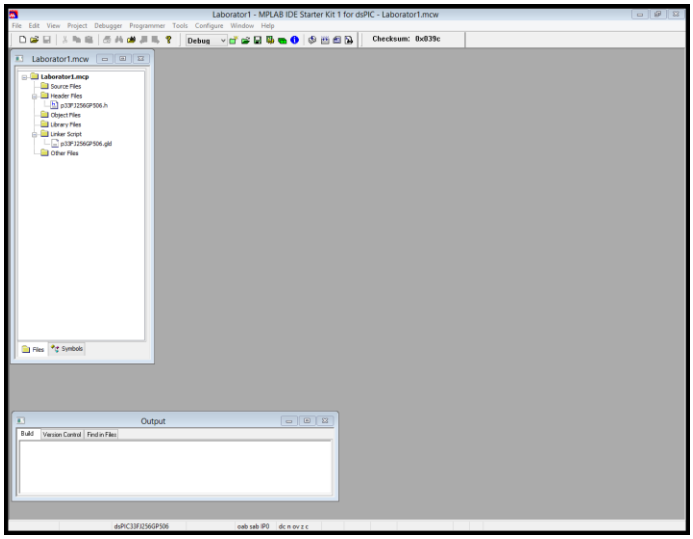
Fișierul *gld* (folosit de aplicația*ObjectLinker*) conține adresele la care pot fi accesați regiștrii.

Ecranul final este un rezumat ce ilustrează dispozitivul folosit, numele proiectului şi bibliotecile utilizate.



5. Crearea codului

După apăsarea butonului *Finish*, se vizualizează proiectul în ecranul MPLAB IDE. Dacă proiectul nu este deschis se va selecta *View* → *Project*.



Se va scrie codul programului. Se deschide un fişier nou selectând *File* → *New*. Se va salva fişierul în directorul *D:\Projects*, ca fişier sursă *.c*, alegând opţiunea *Save As Type – C Source File*. Se va adăuga proiectului fişierul sursă: click dreapta pe *SourceFiles* → *AddFiles…*şi se alege fişierul salvat anterior. În acest fişier se va scrie codul programului.

6. Compilare proiect

Proiectul va fi compilat, asamblat și trimis într-un cod mașină care poate rula în dispozitivul selectat. Se selectează *Programmer → Select Programmer → 3 Starter Kits*. Se compilează proiectul: click dreapta pe *Laborator1.mcp → BuildAll.* Dacă nu apar erori, se poate trece la programarea microcontrolerului: *Programmer → Program*.

7. Testarea prin simulare

În final, proiectul va fi testat cu ajutorul simulatorului (*Debugger*). Se va selecta *Debugger → Select tool → 11 Starter Kits*. Se deschide panoul de observare al variabilelor și al regiștrilor: *View → Watch*, după care se alege registrul sau variabila de interes și să selectează *AddSymbol*. Se vor stabili puncte de întrerupere a execuției (*breakpoints*), prin dublu click pe linia de program dorită, și se rula programul. În momentul în care acesta atinge punctul de întrerupere, execuția va fi oprită și se pot vizualiza valorile regiștrilor sau ale variabilelor observate. Se poate continua execuția din acest punct în mod continuu sau pas cu pas.

2.4 Prezentarea microcontrolerului și a platformei de dezvoltare

Microcontrolerul dsPIC33FJ256GP506 face parte din familia de microcontrolere dsPIC33. Această familie include circuite cu un număr mare de pini (64, 80 și 100), dimensiune mare a memoriei de program (64*kocteți*, 128*kocteți*și 256*kocteți*), și cu memorie RAM de capacitate mărită (8*kocteți*, 16*kocteți*și 30*kocteți*), iar viteza de procesare este până la 40*MIPS*. Această caracteristică face ca familia de circuite să fie utilizată pentru o mare varietate de aplicații în procesarea digitală a semnalelor.

Această familie de microcontrolere, în general, este compatibilă cu pini familiei de circuite PIC24H, și de asemenea are un mare grad de compatibilitate cu familia de circuite dsPIC30F. Aceasta permite migrarea ușoară între familiile de circuite, ceea ce poate fi util pentru o anumită funcționalitate.

Familia de circuite dsPIC33 utilizează o arhitectură pe 16 biți care integrează caracteristicile de control al unui microcontroler (MCU – *Microcontroller Unit*) cu capacitatea de calcul a unui procesor de semnal digital (DSP – *Digital Signal Processor*). Funcționalitatea rezultată este ideală pentru aplicații care se bazează pe viteză mare de calcul, precum și de control.

Nucleul DSP prezintă acumulatori dubli pe 40 de biți, resurse hardware pentru operațiile de divizare, o gamă largă de regiștri de lucru pe 16 biți și o mare varietate de moduri de adresare a datelor care, împreună cu unitatea centrală de procesare (CPU – *Central Processing Unit*) are și o capacitate extinsă de prelucrare matematică. Ca și microcontroler gestionează un număr considerabil de întreruperi. În plus, prin accesarea directă a memoriei (DMA – *Direct Memory Access*) permite transferul de date între mai multe periferice fără a mai implica nucleul controlerului.

Spațiul de date poate fi adresat ca 32*k* cuvinte sau 64*kocteți* și este împărțit în două blocuri, menționate ca memoriile de date *X*și *Y*. Fiecare bloc de memorie are propria unitate de generare a adreselor (AGU – *AddressGeneration Unit*) independentă. Clasa de instrucțiuni a microcontrolerului operează exclusiv prin AGU a memoriei *X*, care accesează harta întreagă a memoriei ca un spațiu liniar de date. Anumite instrucțiuni DSP operează prin unitățile de generare a adreselor *X*și *Y* pentru a suporta operații duble de citire, care împarte spațiul de adresare date în două părți. Spațiul de date *X*și *Y* este limitat de specificațiile dispozitivului.

Memoriile tampon (*buffers*) de adresare circulară sunt suportați în ambele spații de adresare *X*și *Y*. Adresarea circulară a AGU *X* poate fi folosită cu orice clasă de instrucțiuni a microcontrolerului. De asemenea, AGU *X* sprijină adresarea de biți reversivi pentru a simplifica foarte mult intrarea sau ieșirea de date pentru reordonarea secvențelor de date precum la transformata Fourier rapidă (FFT – *Fast Fourier Transform*).

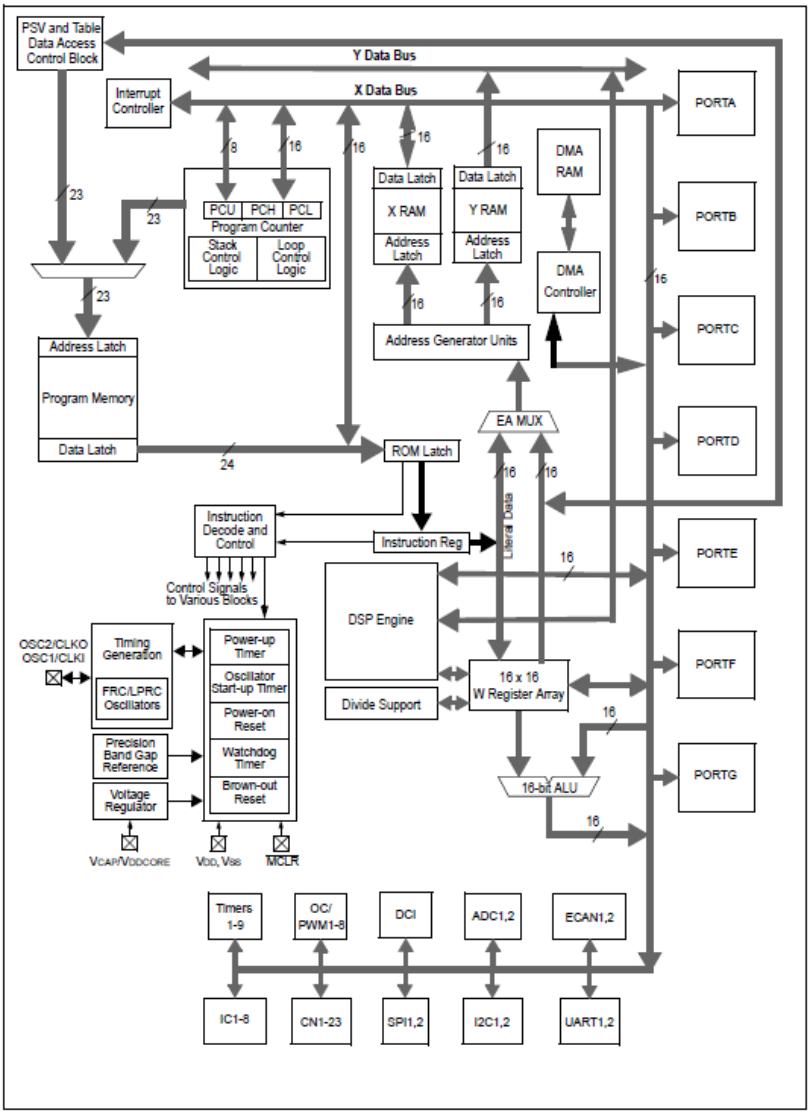
Nucleul DSP este format dintr-un multiplicator de mare viteză, un sumator pe 40 de biți și o unitate de deplasare.

Circuitele dsPIC33FJ256GP506 au arhitectura fluxului de instrucțiuni pe un singur ciclu mașină, prin urmare funcționarea concomitentă a nucleului DSP cu fluxul de instrucțiuni de la MCU nu este posibilă. Totuși, anumite resurse ale ALU (*Arithmetic Logic Unit*), MCU și ale nucleului DSP pot fi folosite concomitent de aceleași instrucțiuni. De asemenea, nucleul DSP are capacitatea de a efectua operații intrinseci între acumulatori fără a necesita date suplimentare.

Câteva din caracteristicile microcontrolerului dsPIC33FJ256GP506 sale sunt următoarele:

- arhitectură Harvard modificată;
- 40 *MIPS* (*MillionInstructions Per Second*);
- suportă *DMA* (*Direct Memory Access*);
- memorie *flash* cu capacitate de 256 kB;
- memorie *SRAM* cu capacitate de 30 kB, din care 2kB reprezintă *DMA RAM*;
- module de comunicații *SPI, I^2C, UART*;
- convertoare analog-numerice și numeric-analogice.

Schema bloc a microcontrolerului:



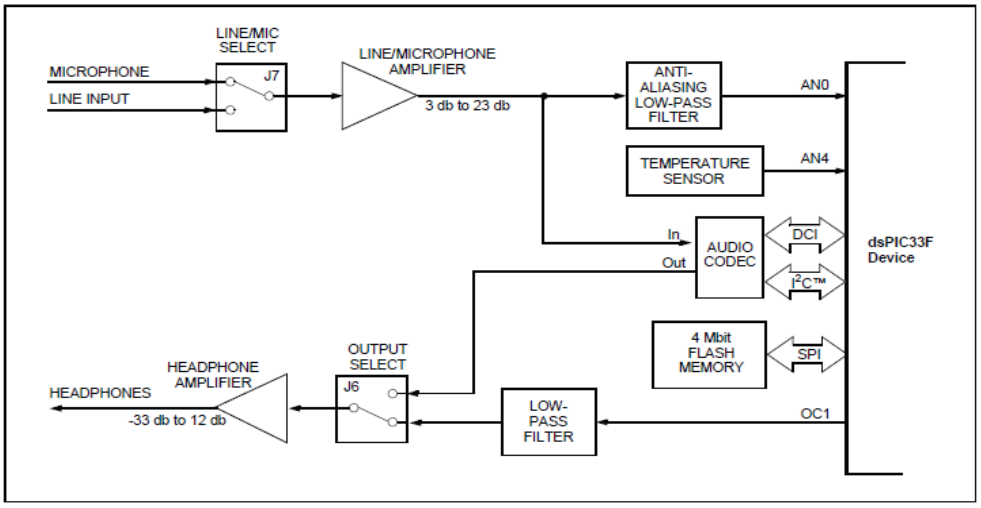
Acest microcontroler face parte din sistemul *Starter Kit for dsPIC Digital Signal Controllers*, creat de către firma Microchip în special pentru dezvoltarea aplicațiilor de procesare a semnalului vocal. Sistemul se conectează la portul USB al calculatorului, permițând fi programarea și depanarea folosind programul MPLAB. Semnalele audio sunt preluate cu ajutorul unui microfon extern sau alte

echipamente audio, sunt transmise către modulul de conversie analog-numerică, sau către codecul audio pe 24 de biți, WM8510. De asemenea, semnalele de ieşire pot fi generate ca impulsuri modulate în durată (*PWM*), care apoi sunt filtrare trece jos, sau pot fi furnizate cu ajutorul codecului audio. Sistemul conține şi o memorie flash de capacitate de 4*MB*.

Semnalul audio de la intrare poate veni de la o linie de intrare sau de la un microfon. Selecția se face prin jumperul *J7*. Semnalul selectat este amplificat de un amplificator neinversor (*Line/Microphone Amplifier*) şi transmis către intrările de conversie analog digitală ale controlerului dsPIC33F printr-un filtru de tip antialiere, care elimină componentele ce depăşesc frecvența care nu mai îndeplineşte criteriul lui Nyquist (este un filtru trece jos activ, de tip SallenKey, de ordinul 6, având frecvența de tăiere 3,3*kHz*). Ieşirea filtrului de antialiere este conectată la intrarea analogică *AN0* controlerului. Dacă la intrarea analogică este plasat un microfon şi se face selecția pentru acesta, semnalului provenind de la acesta i se adaugă o tensiune continuă de 1,65*V*. În modul de intrare *Line*, nu este necesară această tensiune.

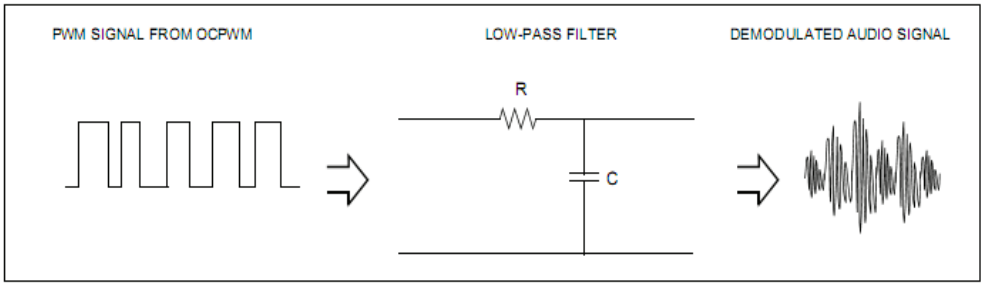
Amplificatorul are o creştere care variază de la 3*dB* la 23*dB*, aceasta putând fi ajustată pentru a controla sensibilitatea microfonului sau creşterea semnalului slab de la linia de intrare.

Schema bloc a sistemului utilizat:



Dacă nu este folosită intrarea analogică din microcontroler, poate fi utilizat codecul audio care asigură o calitate mai bună a semnalului în domeniul audio. Intrarea codecului audio este conectată direct la ieşirea preamplificatorului. Codecul comunică cu microcontrolerul prin intermediul a două canale de

comunicație. Prin protocolul I^2C codecul primește, respectiv transmite, comenzile sau stările în care se află (de exemplu: protocolul de comunicație, rata de eșantionare, controlul volumului, controlul nivelului, setările filtrului, etc.), iar prin interfața *DCI* (*Data ConverterInterface*) transmite și primește secvențele de date care trebuie să le transpună în mediul analogic. Pentru operarea cu interfața DCI este implementat un protocol de comunicație specific, care impune transmisii de cadre, având dimensiunea maximă de 256 biți și suportând până la 16 intervale de transmisie separate (*timeslots*). În vederea comunicării cu mediul analogic, sistemul mai dispune de o linie digitală pe care se poate transmite semnalului sub formă de impulsuri modulate în durată (PWM – *PulseWidthModulation*). Semnalul este apoi filtrat cu un filtru trece jos de ordinul întâi, după cum se vede în figura de mai jos:



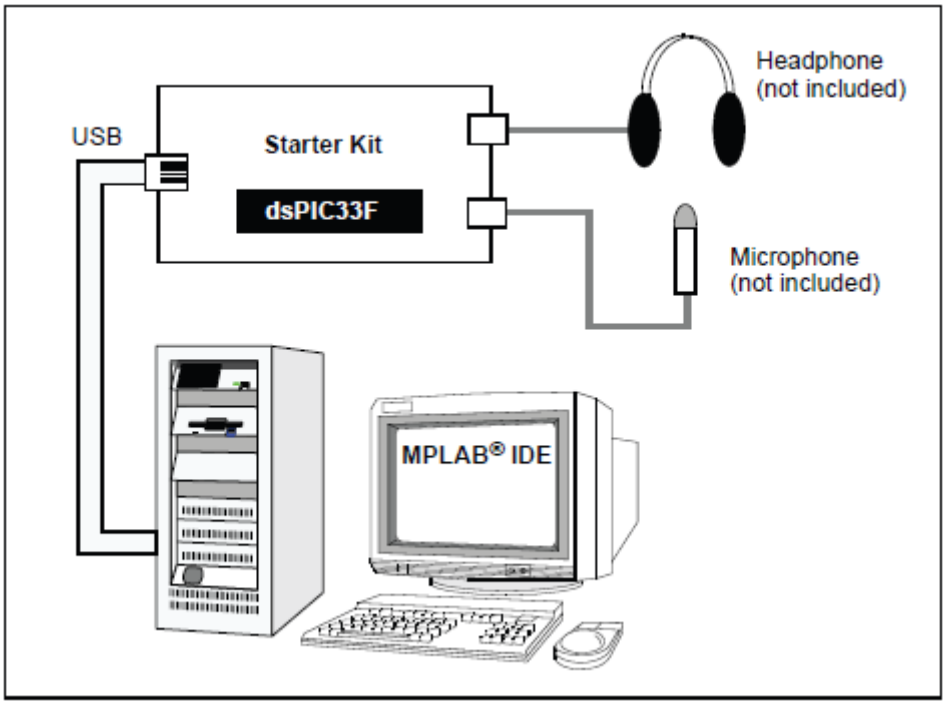
Filtrul se comportă ca un integrator, iar frecvența semnalului PWM trebuie să fie un multiplu întreg al ratei de eșantionare.

Dispozitivul *Starter Kit* include și o memorie serială de tip flash, de 4*MB*, ce poate fi utilizată pentru stocare de informații. Memoria este conectată la magistrala *SPI* de pe dispozitivul dsPIC33F și poate fi utilizată, de obicei, de aplicațiile care necesită depozitarea eșantioanelor semnalului audio înregistrat pentru a putea fi redat.

Sistemul mai deține un modul integrat de programare a microcontrolerului, dar permite și lucrul în modul *Debug*. Programele pot fi scrise în limbajul C, dar și în limbaj de asamblare, în mediul de programarea MPLAB. Compilarea programului C se face cu pachetul software C30. În cadrul mediului de dezvoltare MPLAB se găsesc o serie de biblioteci implementate direct in limbaj de asamblare ce pot fi apelate în limbajul C, pentru a fi utilizate în algoritmi de procesarea digitală a semnalelor.

Microsistemul este conectat la mediul exterior prin intermediul a două mufe jack 3,5mm, pentru semnalul analogic de intrare, respectiv de ieșire, un port USB, prin care face transferul datelor digitale cu mediul MPLAB, două butoane și două LED-uri de uz general.

Schemă pentru realizarea aplicației:



Această figură conține:

- placa de dezvoltare *DSC Starter Kit* cu microcontroler dsPIC33F;
- microfonul și căștile care trebuie conectate la placă;
- conexiunea USB ce asigură comunicarea și alimentarea plăcii de dezvoltare.

Aplicația software pentru dispozitivul dsPIC33F redă mesajul salvat în memoria flash de pe placă și permite înregistrarea și redarea mesajului înregistrat.

Conectarea microsistemului

Pentru conectarea plăcii trebuie urmăriți pașii:

- conectarea microfonului la ieșirea J9;
- conectarea căștilor la ieșirea J8;
- alimentarea plăcii prin conectarea acesteia la calculator prin portul USB;
- când placa va fi alimentată aplicația va reda un mesaj introductiv.

2.5 Desfășurarea lucrării

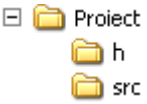
În continuare sunt prezentate câteva exemple de programe care pun în evidență:

- eșantionarea și redarea imediată a unei secvențe (realizarea ecoului);
- un generator de semnal sinusoidal;
- un generator haotic pe baza ecuației logistice.

1. Eșantionarea și redarea imediată a unei secvențe

Acest proiect ilustrează preluarea și redarea imediată a unei secvențe analogice de semnal.

Pentru aceasta aplicatie se creeaza un director *proiect* pe unitatea D:\ .In acest director sunt create doua directoare: *h* si *src*.



Din directorul *D:\Laboratoare_Licenta\Lab_dsp_dsPIC\Include* vor fi transfera urmatoarele fisiere dupa cum urmeaza:

In directorul *h* sunt transferate fișierele *WM8510CodecDrv.h* si *p33FJ256GP506.h*, in directorul *src* se transfera *WM8510CodecDrv.c* si in rădăcina proiectului se va transfera fișierul *p33FJ256GP506.gld*.

Aceste operații se vor repeta si pentru proiectele viitoare.

După ce au fost efectuate operațiile de mai sus, se va crea un proiect nou după descrierea de la paragraful 3. Toate fișierele transferate in directorul *proiect* vor fi adăugate la proiect.

Pentru adăugarea programului principal, se va crea un fișier nou, si se va salva cu denumirea *main.c*, după care se va adăuga proiectului.

Programul principal este următorul:

```
#include "..\h\p33FJ256GP506.h"
#include "..\h\WM8510CodecDrv.h"
//#include "..\h\ADCChannelDrv.h" //se adauga proiectului si
ADCChannelDrv.h si ADCChannelDrv.c

_FGS(GWRP_OFF & GCP_OFF);//Faraprotectie la scriere si
faraprotectia codului
_FOSCSEL(FNOSC_FRC);//Se foloseste oscilatorul RC rapid (Fast
RC)
_FOSC(FCKSM_CSECMD & OSCIOFNC_ON & POSCMD_NONE);//Se
activeaza comutarea ceasurilor, OSC2 ca pin digital, se
dezactiveaza oscilatorul primar
_FWDT(FWDTEN_OFF);//Se opreste cronometrul watchdog
```

```
 #define FRAME_SIZE    128
intcodecBuffer[WM8510DRV_DRV_BUFFER_SIZE];

//Instantierea driverelor
//ADCChannelHandleadcChannelHandle;
//ADCChannelHandle *pADCChannelHandle = &adcChannelHandle;

WM8510Handle codec;
WM8510Handle *codecHandle=&codec;

//Programul principal
intmain(){

intsamples[FRAME_SIZE];

//Configurarea oscilatorului astfel incat dispozitivul sa
opereze la 40MHz
//Fosc= Fin*M/(N1*N2), Fcy=Fosc/2
//Fosc= 7.37M*40/(2*2)=80Mhz pentru tact intrare de 7.37M
    PLLFBD=41;//M=39
CLKDIVbits.PLLPOST=0;//N1=2
CLKDIVbits.PLLPRE=0;//N2=2
    OSCTUN=0;

    __builtin_write_OSCCONH(0x01);//Initializare oscilator la
FRC (Fast RC) cu PLL
    __builtin_write_OSCCONL(0x01);
while(OSCCONbits.COSC!=0b01);//Astepta realizarea comutarii
while(!OSCCONbits.LOCK);

// Initializari
    WM8510Init(codecHandle,codecBuffer);//Initializeaza
codecul
    WM8510Start(codecHandle);//Porneste codecul
    WM8510SampleRate8KConfig(codecHandle);//Configureaza
codecul sa functionezecu frecventa de esantionare de 8kHz

//Bucla de program
while(1){
//Esantionarea semnalului de la microfon
//Asteapta ca WM8510 sa devina disponibil
while(WM8510IsReadBusy(codecHandle));
//Esantioneaza semnalul audio
        WM8510Read(codecHandle,samples,FRAME_SIZE);

//Asteapta ca WM8510 sa devina disponibil
while(WM8510IsWriteBusy(codecHandle));
//Redare semnal inregistrat
```

```
            WM8510Write(codecHandle,samples,FRAME_SIZE);
}
}
```

Nota: Eșantioanele preluate de la codec sunt salvate în vectorul tampon *samples*, după care sunt redate.

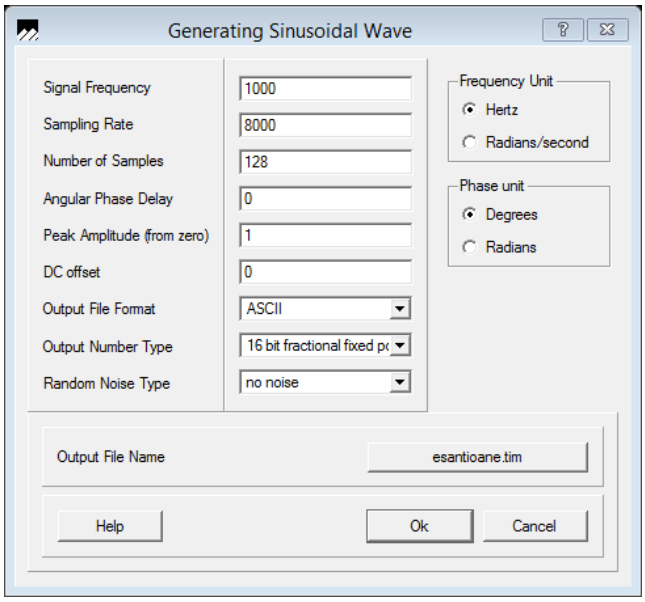
Se compilează programul si descarcă pe sistemul de dezvoltare.

Vizualizarea semnalelor de la sistemul de dezvoltare se face cu ajutorul programului Visual Analyser care se găsește pe Desktop.

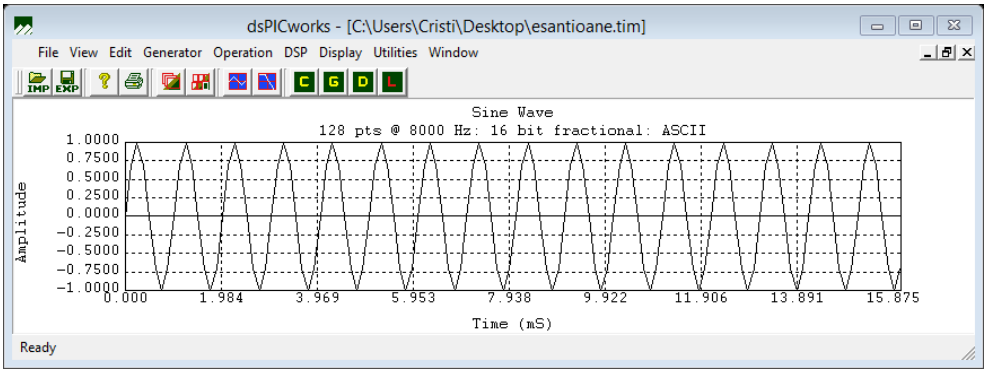
2. Generator de semnal sinusoidal

Se va genera un semnal sinusoidal de frecvență fixă, 1*kHz*. Pentru a realiza această aplicație se vor folosi eșantioanele unui semnal sinusoidal de frecvență 1*kHz*, obținute cu ajutorul programului *dsPICworks v2.0*. Acestea vor fi salvate într-un vector care va fi citit în mod repetat.

Pentru generarea eșantioanelor se deschide programul *dsPICworks v2.0*. Se selectează *Generator* → *Sinusoidal*..., și se introduc datele din imagine:



In dreptul paragrafului *Output File Name* se va apăsa pe butonul *Click to Select File Name* unde se va alege directorul in care se salvează fişierul generat. Forma de undă generată:



Fişierul astfel salvat se deschide cu aplicaţia *Notepad*, iar valorile se vor copia în programul principal.

La fel ca la aplicaţia anterioara se vor adăuga proiectului fişierele *WM8510CodecDrv.h* şi *WM8510CodecDrv.c*, care conţin declaraţiile şi codul funcţiilor necesare pentru generarea semnalului audio folosind codecul audio.

Programul principal folosit:

```
#include "..\h\p33FJ256GP506.h"
#include "..\h\WM8510CodecDrv.h"

_FGS(GWRP_OFF & GCP_OFF);//Faraprotectie la scriere si
faraprotectia codului
_FOSCSEL(FNOSC_FRC);//Se foloseste oscilatorul RC rapid (Fast
RC)
_FOSC(FCKSM_CSECMD & OSCIOFNC_ON & POSCMD_NONE);//Se
activeaza comutarea ceasurilor, OSC2 ca pin digital, se
dezactiveaza oscilatorul primar
_FWDT(FWDTEN_OFF);//Se opreste cronometrul watchdog

#define FRAME_SIZE 128
intcodecBuffer[WM8510DRV_DRV_BUFFER_SIZE];

//Esantioanele unui semnal de 1kHz esantionat cu frecventa de
8kHz
intsamples[FRAME_SIZE]={0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F
,0x8001,
0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,
0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,
```

```
0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,
0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,
0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,
0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,0x0000,
0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,
0x8001,0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,
0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,
0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,
0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,
0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,
0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,
0x0000,0xA57F,0x8001,0xA57F};

//Instantierea driverului
WM8510Handle codec;
WM8510Handle *codecHandle=&codec;

//Programul principal
intmain(){
        //Configurarea oscilatorului astfel incat
dispozitivul sa opereze la 40MHz
        //Fosc= Fin*M/(N1*N2), Fcy=Fosc/2
        //Fosc= 7.37M*40/(2*2)=80Mhz pentru tact intrare de
7.37M
        PLLFBD=41;                      //M=39
        CLKDIVbits.PLLPOST=0;           //N1=2
        CLKDIVbits.PLLPRE=0;            //N2=2
        OSCTUN=0;

        __builtin_write_OSCCONH(0x01); //Initializare
oscilator la FRC (Fast RC) cu PLL
        __builtin_write_OSCCONL(0x01);
        while(OSCCONbits.COSC!=0b01);//Astepta realizarea
comutarii
        while(!OSCCONbits.LOCK);

        // Initializari
        WM8510Init(codecHandle,codecBuffer);//Initializeaza
codecul
        WM8510Start(codecHandle);//Porneste codecul

        //Bucla de program
        while(1){
                //Citirea esantioanelor semnalului
                //Asteapta ca WM8510 sa devina disponibil
        while(WM8510IsWriteBusy(codecHandle));
        //Redare semnal inregistrat
        WM8510Write(codecHandle,samples,FRAME_SIZE);
        }}
```

Programul se compilează si descarcă pe sistemul de dezvoltare după care va fi rulat. Semnalul se va vizualiza cu aplicația Visual Analyser.

3. Generator haotic pe baza ecuației logistice

Există multe aplicații în care este necesară generarea numerelor aleatoare. Una dintre ele o reprezintă generarea unui semnal de zgomot.

Un exemplu tipic de generator haotic în timp discret este cel descris de ecuația logistică:

$$x_n = r \cdot x_{n-1}\left(1 - x_{n-1}\right)$$

Acesta este un proces iterativ prin care eșantionul curent este calculat în funcție de eșantionul precedent. Pentru $r = 0,7$, relația devine:

$$x_n = 0,7 \cdot x_{n-1}\left(1 - x_{n-1}\right)$$

Programul principal este următorul:

```
#include "..\h\p33FJ256GP506.h"

_FGS(GWRP_OFF & GCP_OFF);//Faraprotectie la scriere si
faraprotectia codului
_FOSCSEL(FNOSC_FRC);//Se foloseste oscilatorul RC rapid (Fast
RC)
_FOSC(FCKSM_CSECMD & OSCIOFNC_ON & POSCMD_NONE);//Se
activeaza comutarea ceasurilor, OSC2 ca pin digital, se
dezactiveaza oscilatorul primar
_FWDT(FWDTEN_OFF);//Se opreste cronometrul watchdog

#define FRAME_SIZE 128

//Programul principal
intmain(){

        //Configurarea oscilatorului astfel incat
dispozitivul sa opereze la 40MHz
        //Fosc= Fin*M/(N1*N2), Fcy=Fosc/2
        //Fosc= 7.37M*40/(2*2)=80Mhz pentru tact intrare de
7.37M
        PLLFBD=41;                      //M=39
        CLKDIVbits.PLLPOST=0;           //N1=2
        CLKDIVbits.PLLPRE=0;            //N2=2
        OSCTUN=0;
```

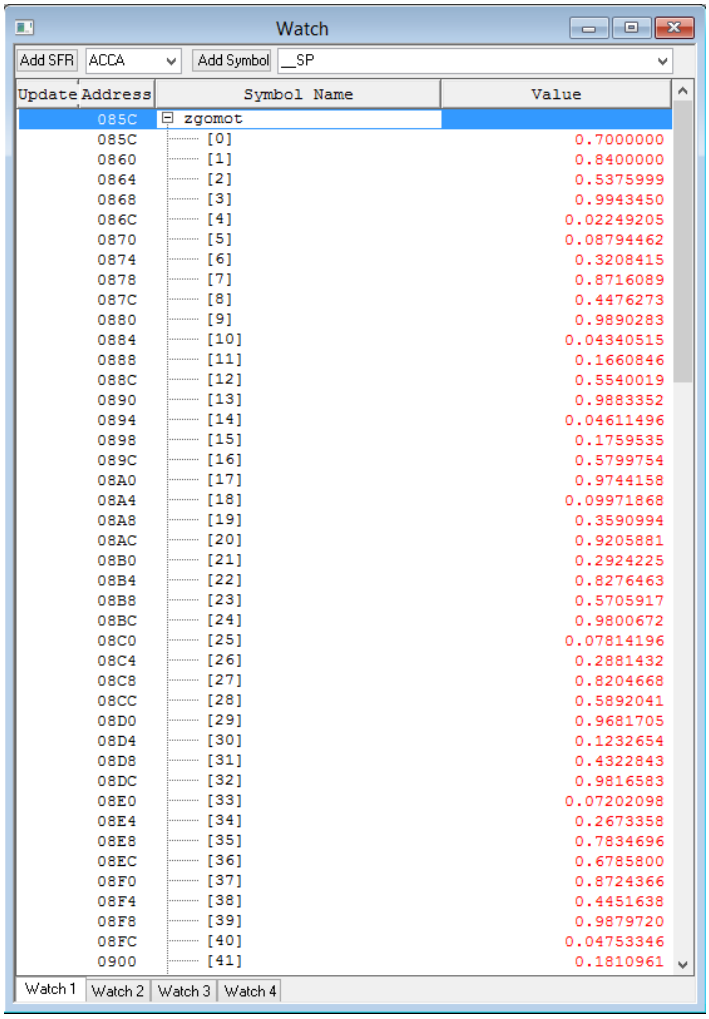
```
        __builtin_write_OSCCONH(0x01); //Initializare
oscilator la FRC (Fast RC) cu PLL
        __builtin_write_OSCCONL(0x01);
        while(OSCCONbits.COSC!=0b01);//Astepta realizarea
comutarii
        while(!OSCCONbits.LOCK);

        //Generare numere aleatoare folosind ecuatia
logisticii
        float zgomot[FRAME_SIZE];
        zgomot[0]=0.7;
        int j =1;
        int r =4;

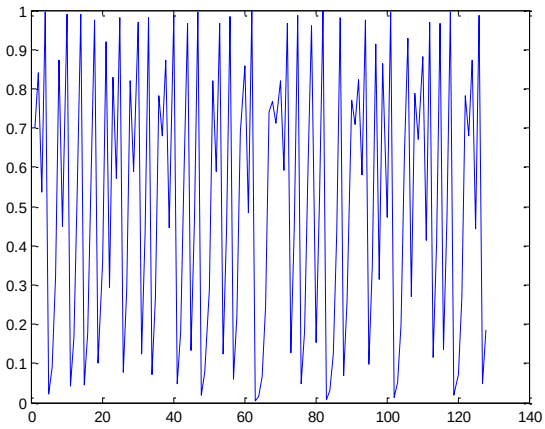
        while(j<=FRAME_SIZE){
                zgomot[j]= r*zgomot[j-1]*(1-zgomot[j-1]);
                j++;
        }

        //Bucla de program
        while(1){
        //Instructiuni...
        }
}
```

Pentru a vizualiza semnalul generat, se poate observa conținutul vectorului *zgomot*. Pentru aceasta se selectează *Debugger → Select Tool → 11 Starter Kits*. Se alege *Project → BuildConfiguration → Debug*. După compilare, se adaugă vectorul *zgomot* în fereastra de observație: *View → Watch → Selectați zgomot → AddSymbol*. Se programează microcontrolerul și se rulează programul. După câteva secunde se poate opri execuția programului pentru a vizualiza conținutul vectorului *zgomot*:



Se execută click dreapta pe *zgomot*, se alege *Export Table…*, se apasă *OK*, se alege *Single Column Output*și se alege numele fișierului în care se va salva conținutul vectorului. Acesta se va deschide cu aplicația *Notepad*, se va copia conținutul său și se va realiza un grafic folosind instrucțiunea *plot([*vector_date*])*, din MATLAB.
Semnalul generat:



Din experiența programelor anterioare, modificați programul astfel incat, semnalul sa fie generat pe ieșirea sistemului de dezvoltare.

Realizați următoarele aplicații:

a) Realizați un program prin care să se dubleze amplitudinea unui semnal de intrare. Semnalul de ieșire se va genera folosind codul audio.

b) Modificați programul prin care procesorul de semnal generează un semnal sinusoidal astfel încât semnalul de ieșire să aibă frecvența de 30*Hz*.

c) Modificați același program astfel încât semnalul de ieșire să fie de 500*Hz*și frecvența de eșantionare de 48*kHz*.

d) Generați un semnal format din două sinusoide cu frecvențele de 100*Hz*, respectiv 300*Hz*, iar frecvența de eșantionare fiind de 48*kHz*.

e) Realizați un program astfel încât semnalul de ieșire să respecte următoarea relație:

$$y[n] = x[x] + x[n-1] + x[n-2]$$

f) Realizați un program prin care să generați un semnal triunghiular, cu frecvența de 500*Hz*și frecvența de eșantionare de 48*kHz*.

3 Implementarea de algoritmi rapizi pentru transformata Fourier

3.1 Scopul lucrării

Lucrarea are ca scop studiul semnalelor în domeniul spectral. Se vor reprezenta secvențe numerice folosind seria Fourier și transformata Fourier discretă. De asemenea, vor fi prezentați algoritmi rapizi de calcul al transformatei Fourier, ce pot fi aplicați pe procesoare digitale de semnal.

3.2 Noțiuni teoretice

În mediul de programare MATLAB, calculul transformatei Fourier se realizează cu ajutorul următoarelor funcții:

X = *fft*(*x*)	**Calculează coeficienții transformatei Fourier discrete**
X = *ifft*(*x*)	**Calculează inversa transformatei Fourier discrete**
x – reprezintă secvența de intrare	
X – reprezintă coeficienții transformatei Fourier discrete	

Calculul **seriei Fourier discrete** se realizează după următoarele relații:

$$c_k = \frac{1}{N}\sum_{n=0}^{N-1}\tilde{x}[n]\cdot e^{-jk\frac{2\pi}{N}n}$$ Analiza secvenței $x[n]$

$$\tilde{x}[n] = \sum_{k=0}^{N-1}c_k\cdot e^{+jk\frac{2\pi}{N}n}$$ Sinteza secvenței $x[n]$

unde $x[n]$ este o secvență reală periodică.

Transformata Fourier discretă (TFD) **Transformata Fourier discretă inversă (TFDI)**

$$X[k]=\sum_{n=0}^{N-1}x[n]e^{-j2\pi kn/N}=\sum_{n=0}^{N-1}x[n]W_N^{kn} \quad x[n]=\frac{1}{N}\sum_{k=0}^{N-1}X[k]e^{j2\pi kn/N}=\frac{1}{N}\sum_{k=0}^{N-1}X[k]W_N^{-kn}$$

Reprezentarea matriceală este $X=Wx\leftrightarrow x=\frac{1}{N}W^*X$, unde $W_N^{nk}=e^{-j2\pi kn/N}$.

Coeficienții TFD $\{X[k]\}$ sunt variabile complexe caracterizate de modul şi fază:

$$X[k]=|X(k)|e^{j\phi(k)}$$

unde amplitudinea spectrală este:

$$|X(k)|=\sqrt{\{\mathrm{Re}\{X[k]\}\}^2+\{\mathrm{Im}\{X[k]\}\}^2}$$

iar spectrul fazei este:

$$\phi[k]=\arctan\frac{\mathrm{Im}\{X[k]\}}{\mathrm{Re}\{X[k]\}}$$

Aplicațiile TFDnecesită operații cu un volum mare de date, deoarece mărea număruluiNal datelor,precizia de calcul crește. Rezultă căvolumul de calcule crește în mod corespunzător. Astfel, pentru determinarea unui coeficient $X[k]$ alTFD sunt necesare *N* înmulțiri și *N-1* adunări în complex. Determinarea tuturor coeficienților ($k=0,1,...,N-1$) implică N^2 înmulțiri și $N(N-1)$ adunări în complex. În aplicații, Nia valori de ordinul, cel puțin, $N=2^{10}=1024$, ceea ce conduce la valori inacceptabile pentru N^2.

În vederea reducerii volumului de calcule s-au elaborat metode prin care secvența de *N* valori x_n este împărțită în subsecvențe tot mai mici, în care operațiile de calcul sunt de același tip, datorită simetriei și periodicității

exponențialelor w_N^{nk}. *A* fost elaborată o mare varietate de algoritmi de calcul a TFD[3], cunoscuți sub denumirea *transformata Fourier rapidă (TFR) – fast Fourier transform (FFT).*

Diferitele variante au performanțe comparabile și operează în general cu $N = 2^m$, ceea ce permite divizarea continuă cu 2, a secvențelor de date.

Există două clase de algoritmi pentru calculul TFD**:** cu decimare în timp (*DIT*) și cu decimare în frecvență.

3.2.1 TFR cu decimare în timp

Termenul de „*decimare*" în timp presupune *divizarea în subsecvențe mai scurte a secvenței* $\{x_n\}$*, care conține* $N = 2$ *eșantioane astfel încât secvența* $X[k]$ *să rezulte din combinarea transformatelor corespunzătoare.* Acest lucru este posibil datorită următoarelor proprietăți ale secvenței $\{W_N^{kn}\}$, care se verifică imediat ținând seama că $W_N = e^{-j\frac{2\pi}{N}}$:

$$W_N^{2k} = W_{\frac{N}{2}}^{2k}$$

$$W_N^{k(N-n)} = \left(W_N^{kn}\right)^*$$

$$W_N^{kn} = W_N^{(k+N)n} = W_N^{k(N+n)}$$

Ultimele două egalități ilustrează *simetria*, respectiv *periodicitatea* secvențelor W_N^{kn}.

În cazul algoritmului TFR cu decimare în timp,secvența de intrare $\{x[n]\}$, $n = \overline{0, N-1}$ se împarte în două secvențe: pară $\{g[n]\}$ și impară $\{h[n]\}$, separând eșantioanele cu indice par de cele cu indice impar:

$$\left.\begin{aligned} g[n] &= x[2n] \\ h[n] &= x[2n+1] \end{aligned}\right\}, \quad n = \overline{0, \frac{N}{2}-1}$$

Separând secvența pară de cea impară în expresia pentru $X[k]$, se obține:

$$X[k] = \sum_{n=0}^{N/2-1}\left\{x[2n]W_N^{2nk} + x[2n+1]W_N^{(2n+1)k}\right\}, \qquad k = \overline{0,\ N-1}$$

Avem că:

[3]Mateescu, Ad., ș.a., *Prelucrarea numerică a semnalelor*, Editura Tehnică, București, 1997

$$W_N^{2nk} = e^{-j2\pi \cdot 2nk/N} = e^{-j\frac{2\pi}{N/2}nk} = W_{\frac{N}{2}}^{nk}$$

şi rezultă:

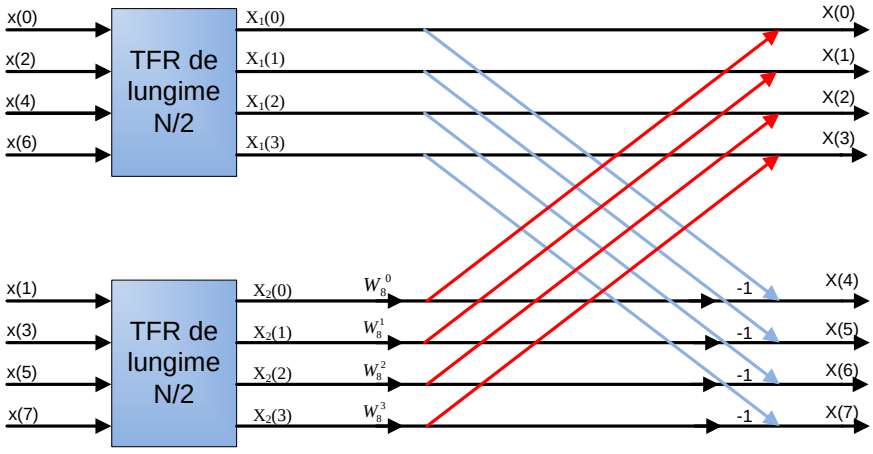
$$X[k] = \sum_{n=0}^{N/2-1} x[2n] W_{\frac{N}{2}}^{nk} + W_N^k \sum_{n=0}^{N/2-1} x[2n+1] W_{\frac{N}{2}}^{nk}, \qquad k = \overline{0,\ N-1}$$

Se observă că acestea reprezintă TFD a secvenței pare, respectiv impară. Avem:

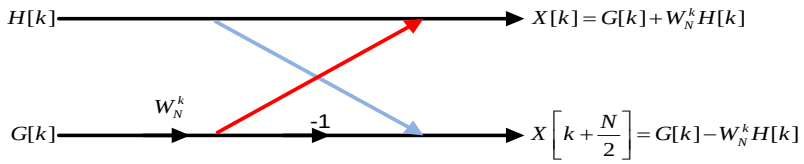
$$G[k] = \sum_{n=0}^{N/2-1} g[n] W_{\frac{N}{2}}^{nk}, \quad H[k] = W_N^k \sum_{n=0}^{N/2-1} h[n] W_{\frac{N}{2}}^{nk}, \quad k = \overline{0,\ \frac{N}{2}-1}$$

Secvențele $\{G[k]\}$ și $\{H[k]\}$ au perioada $\frac{N}{2}$,și prin urmare:

$$G[k] = G\left[k + \frac{N}{2}\right], \quad H[k] = H\left[k + \frac{N}{2}\right]$$



Descompunerea unei TFD de lungime *N*, în două TFD de lungime $\frac{N}{2}$, $N = 8$,utilizând algoritmul de decimare în timp



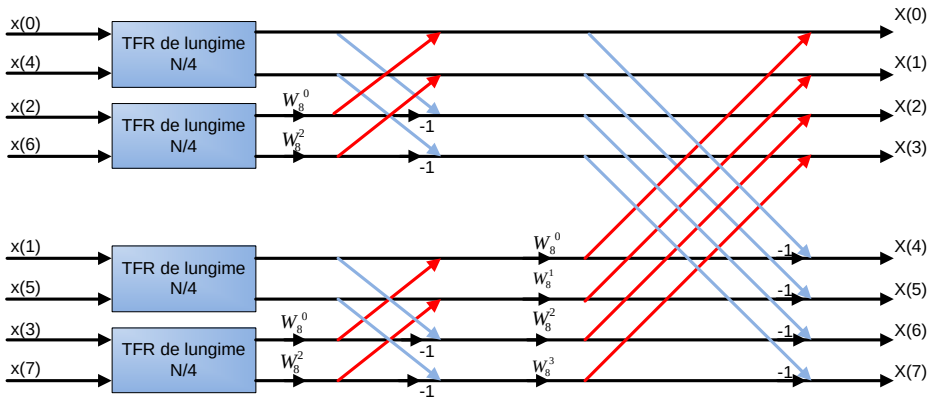
Reprezentarea grafică a unei etape de divizare (structură *butterfly*)

Numărul total de înmulțiri în complex este acum $2\left(\frac{N}{2}\right)^2 + N = N^2\left[\frac{1}{2}+\frac{1}{N}\right] \cong \frac{N^2}{2}$ la N mare.

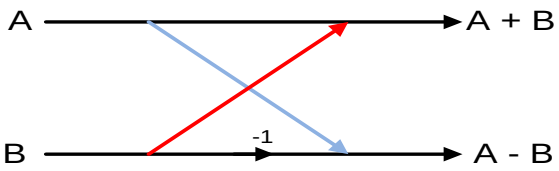
Relațiile de mai sus reprezintă esența algoritmului TFR cu decimare în timp, ilustrând un modul de combinații elementare, care reduce numărul de înmulțiri la jumătate. Acest modul este numit *etapă de divizare*sau *fluture* (*butterfly*). Factorul W_N^{nk} se numește *factor de rotație* (sau *coeficient de rotație*). Diviziunea cu 2 a secvențelor din domeniul timp este continuată. $\{G[k]\}$ și $\{H[k]\}$ se obțin din subsecvențe de lungime $\frac{N}{4}$. Decimarea în timp poate fi repetată de m ori, în ultima etapă de divizare cu doi obținându-se o TFDcu $N=2$, care necesită o adunare și o scădere, deoarece aici factorul de rotație este $W_N^{\frac{N}{2}} = e^{-j\pi} = -1$. În total, pentru determinarea secvenței $\{X[k]\}$ sunt necesare următoarele operații în complex:

$$\frac{N}{2} \times m = \frac{N}{2}\log_2 N \text{ înmulțiri și } N \times m = N\log_2 N \text{ adunări.}$$

Reducerea volumului de calcule este spectaculoasă pentru N mare. De exemplu, pentru $N = 2^{10} = 1024$, rezultă o reducere ceva mai mare de 100 de ori.



Graful TFR cu decimare în timp pentru $N = 8$



Graful pentru TFR cu $N = 2$

Sunt evidente câteva caracteristici generale ale TFR cu decimare în timp:

- ordinea datelor de intrare $\{x[n]\}$, $n = \overline{0, N-1}$ nu este cea naturală, în timp ce datele de ieșire $\{X[k]\}$ apar în ordine naturală. Ordinea intrărilor nu este întâmplătoare, ci urmează „*legea de inversiune binară*”, în care ordonarea se face „*cu inversarea biților*”. În această ordonare poziția *n*, ocupată de eșantionul de intrare cu indicele *n*, are în cod binar numărul *n* reprezentat în ordinea inversată a biților. În tabelul următor se ilustrează inversarea biților în cazul $N = 8$.

Ordinea eșantioanelor la intrare pentru TFR cu decimare în timp, *N*=8

Indexul eșantionului de intrare		**Indexul eșantionului cu inversarea biților**	
Zecimal	Binar	Binar	Zecimal
0	000	000	0
1	001	100	4
2	010	010	2
3	011	110	6
4	100	001	1
5	101	101	5
6	110	011	3
7	111	111	7

3.2.2 TFR cu decimare în frevență

Algoritmul TFR cu decimare în frecvențăeste echivalent din punct de vedere al volumului de calcule cu cel precedent.Secvența de intrare $\{x[n]\}$ a datelor de intrare se împarte în două subsecvențe în modul următor:

$$\left.\begin{aligned} x[n]^1 &= x[n] \\ x[n]^2 &= x\left[n+\frac{N}{2}\right] \end{aligned}\right\}, \quad n=\overline{0,\ \frac{N}{2}-1}$$

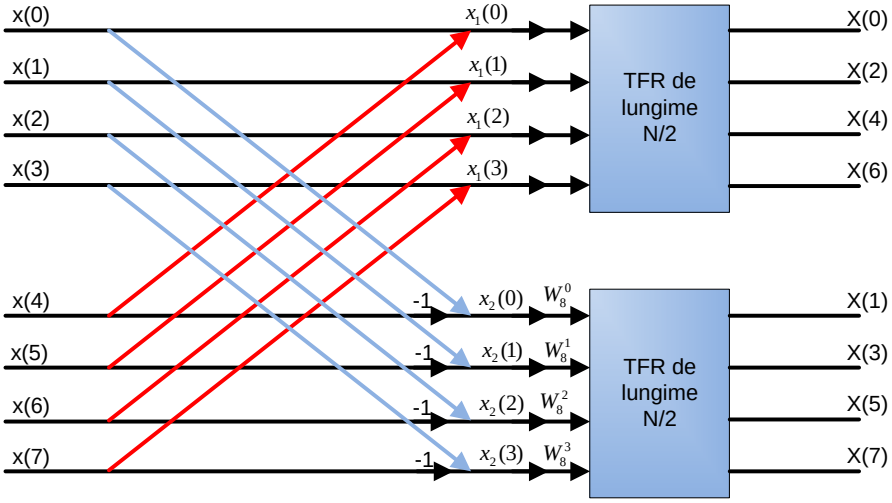
Relația pentru **TFR**, devine:

$$X[k]=\sum_{n=0}^{N/2-1} x[n]W_N^{nk}+\sum_{n=\frac{N}{2}}^{N/2-1} x[n]W_N^{nk}, \qquad k=\overline{0,\ N-1}$$

$$X[k]=\sum_{n=0}^{N/2-1} x[n]^1 W_N^{nk}+\sum_{n=0}^{N/2-1} x[n]^2 W_N^{\left(n+\frac{N}{2}\right)k}, \qquad k=\overline{0,\ N-1}$$

Se realizează acum decimarea în frecvență, separând eșantioanele $X[k]$ de ordin par, $k=2r$, de cele de ordin impar, $k=2r+1$:

$$\left.\begin{aligned} X[2r] &= \sum_{n=0}^{\frac{N}{2}-1}\left[x[n]^1 W_{\frac{N}{2}}^{nr}+x[n]^2 W_{\frac{N}{2}}^{nr}\right] \\ X[2r+1] &= \sum_{n=0}^{\frac{N}{2}-1}\left[x[n]^1 W_{\frac{N}{2}}^{nr}+x[n]^2 W_{\frac{N}{2}}^{\left(nr+\frac{N}{2}\right)}\right] \end{aligned}\right\} \quad r=\overline{0,\ \frac{N}{2}-1}$$

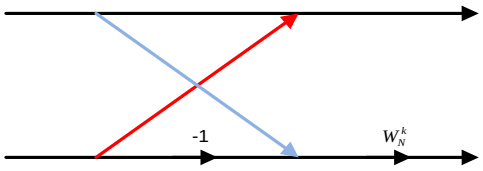


Descompunerea unei TFD de lungime *N*, în două TFD de lungime $\frac{N}{2}$, $N=8$, utilizând algoritmul de decimare în frecvență

Relațiile precedente pot fi simplificate sub forma:

$$\left.\begin{aligned} X[2r] &= \sum_{n=0}^{\frac{N}{2}-1}\left[x[n]^1 + x[n]^2\right]W_{\frac{N}{2}}^{nr} \\ X[2r+1] &= \sum_{n=0}^{\frac{N}{2}-1}\left[x[n]^1 - x[n]^2\right]W_N^r W_{\frac{N}{2}}^{nr} \end{aligned}\right\} \quad r = \overline{0,\ \frac{N}{2}-1}$$

Se obțin astfel TFD ale unor secvențe de lungime $\frac{N}{2}$.



***Fluture* (*butterfly*) pentru TFD cu decimare în frecvență**

3.3 Aplicații

3.3.1 Implementarea de algoritmi pentru TF

1. Să se realizeze descompunerea în *SFTD* a unei serii de impulsuri periodice cu factorul de umplere 0,5. Se vor folosi 64 de eșantioane. Să se verifice apoi proprietățile generale ale coeficienților descompunerii în *SFTD* a unei secvențe periodice, cu valori reale, pentru care:

$$c(1)=\frac{1}{N}\sum_{n=0}^{N-1}x[n+1] \qquad c\left(\frac{N}{2}\right)=\frac{1}{N}\sum_{n=0}^{N-1}x[n+1](-1)^n \qquad c_{N-k}=c_k^*$$

Programul MATLAB:

```
x=[ones(1,32),zeros(1,32)];N=64;
for k=0:N-1
    c(k+1)=0;
for n=0:N-1
        c(k+1)=c(k+1)+x(n+1)*...
exp(-j*pi*2*k/N*n);
end
    c(k+1)=c(k+1)/N
end
stem(abs(c(1:N/2))); grid
```

2. Se realizează secvența MATLAB pentru simularea algoritmului TFR cu decimare în frecvență și compararea rezultatului obținut cu acest algoritm, cu cel obținut utilizând funcția *fft* din MATLAB.

```
%simulare RFT
clear, clc;
%generarea semnalului pentru care se calculeaza spectru
%frecventa de esantionare
fe=10e3;
%lungime RFT
N=256;
t=0:1/fe:(N-1)/fe;
%frecventa sinusoida
fs=1e3;
n=0:length(t)-1;
x=sin(2*pi*fs/fe*n);
%reprezentarea in timp a semnalului
subplot(311);
```

```
plot(t(1:100),x(1:100));grid;
title('Reprezentarea semnalului in timp')
xlabel('Timp [s]');
ylabel('Amplitudine')
a=fft(x,N);
%reprezentare spectru semnal
subplot(312);
plot(abs(a));grid;
title('Reprezentarea spectrului semnalului(calculat cu
functia "fft" din Matlab)');
xlabel('Numar esantioane');
ylabel('Amplitudine');
%algoritm pentru calculul TFR utilizand decimarea in
frecventa
offset = N;
xi=x;
etape=0;
% for pentru numarul de etape
for et=1:(log2(N));
    offset = offset/2;
    nr_grupuri = N/(2*offset);
    x = xi;
    m=-1;
    etape=etape+1;
% for pentru numarul de grupuri dintr-o etapa
for l=1:nr_grupuri
        k=0;
        m=m+2;
aux=((m-1)*offset)+1;
%for pentru numarul de butterfly dintr-un grup
for j=aux:((offset*m))
            xi(j) = (x(j)+x(j+offset));
xa = (x(j)-x(j+offset));
            w(k+1) = exp((-i*2*pi*k)/N);
xi(j+offset) = xa*w(k+1);
            k=k+2^(etape-1);
end
end
end
%ordonareesantioane la iesire
y=xi;
poz = 0:N-1;
for i=1:N
poz_bin{i} = dec2bin(poz(i), log2(N));
end
for j=1:N
for i=1:log2(N)
poz_bin_ordonat{j}(i)=poz_bin{j}(log2(N)+1-i);
```

```
end
end
poz_ordonat = bin2dec(poz_bin_ordonat);
for i=1:N
    y(poz_ordonat(i)+1)=xi(i);
end
%repreyentare grafica
subplot(313);
plot(abs(y));grid;
title('Reprezentarea spectrului semnalului (calculat
utilizand algoritmul de decimare in frecventa)');
xlabel('Numar esantioane');
ylabel('Amplitudine');
```

3. Secvența MATLAB următoare analizează în domeniile timp și frecvență un semnal *chirp*, a cărui frecvență instantanee variază între 0 și 5*MHz*, pe o durată $T = 10\mu s$. Frecvența de eșantionare utilizată este $F_e = 50MHz$.

```
f0=0;ff=5e6; T=1e-5; beta=(ff-f0)*pi/T;
Fe=5e7;t=0:1/Fe:T;
x=cos(2*pi*f0*t+beta*t.^2);
subplot(211),plot(t,x),
xlabel('timp [s]'); ylabel('amplitudine [V]');
title('semnal temporal')
X=abs(fft(x,1024));
subplot(212), plot([0:99]/Fe,X(1:100))
xlabel('frecventa [Hz]'); ylabel('amplitudine [V]');
title('spectrul semnalului')
```

4. Să se elaboreze o funcție MATLAB pentru calculul TFTD a unei secvențe finite, de lungime *N*, pentru *N* frecvențe echidistante pe cercul unitate. Să se utilizeze funcția creată pentru calculul TFTD a secvenței $x[n] = 0,88^n$ în $N = 128$ puncte.

```
function [H,W]=tftd(h,N)
W=(2*pi/N)*[0:N-1]'; mid=ceil(N/2)+1;
W(mid:N)=W(mid:N)-2*pi; W=fftshift(W);
H=fftshift(fft(h,N));
subplot(211),plot(W/2/pi,abs(H));grid;
xlabel('frecventa normalizata')
ylabel('amplitudine [V]')
subplot(212),plot(W/2/pi,180/pi*angle(H));grid;
xlabel('frecventa normalizata')
ylabel('faza [grade]');end
```

5. Reprezentați densitatea spectrală de putere a unui semnal format din două sinusoide de 50*Hz*și 120*Hz*și un zgomot aditiv, alb, gaussian, de medie nulă și dispersie 4.

```
t=0:0.001:0.8;
x=sin(2*pi*50*t)+sin(2*pi*120*t)+2*randn(1,length(t));
subplot(211);plot(x);
title('Semnalul x(t)');
grid;
X=fft(x,512);
Px=X.*conj(X)/512;
f=1000*(0:255)/512;
subplot(212);plot(f,Px(1:256));
title('Densitatea spectrala de putere');
grid
```

6. Să se verifice proprietățile de translație (deplasare, decalaj) în timp și de modulație ale transformatei Fourier. Se va considera semnalul:

$$x[n]=\begin{cases} n, n\in[1,8] \\ 0, n\in[9,16] \end{cases}$$

```
figure
w = -pi:2*pi/255:pi;
wo = 0.4*pi; D = 10;
num = [1 2 3 4 5 6 7 8 9];

h1 = freqz(num, 1, w);
h2 = freqz([zeros(1,D) num], 1, w);

subplot(2,2,1)
plot(w/(2*pi),abs(h1));grid
title('Spectrul de amplitudine al secventeiinitiale')
subplot(2,2,2)
plot(w/(2*pi),abs(h2));grid
title('Spectrul de amplitudine al secventei decalate')
subplot(2,2,3)
plot(w/(2*pi),angle(h1));grid
title('Spectrul de faza al secventeiinitiale')
subplot(2,2,4)
plot(w/(2*pi),angle(h2));grid
title('Spectrul de faza al secventei decalate')

figure

w = -pi:2*pi/255:pi; wo = 0.4*pi;
```

```
num1 = [1 3 5 7 9 11 13 15 17];
L = length(num1);
h1 = freqz(num1, 1, w);
n = 0:L-1;
num2 = exp(wo*1i*n).*num1;
h2 = freqz(num2, 1, w);

subplot(2,2,1)
plot(w/(2*pi),abs(h1));grid
title('Spectrul de amplitudine al secventeiinitiale')
subplot(2,2,2)
plot(w/(2*pi),abs(h2));grid
title('Spectrul de amplitudine al secventei modulate')
subplot(2,2,3)
plot(w/(2*pi),angle(h1));grid
title('Spectrul de faza al secventeiinitiale')
subplot(2,2,4)
plot(w/(2*pi),angle(h2));grid
title('Spectrul de faza al secventei modulate')
```

7. Să se studieze influența lungimii semnalului asupra spectrului său, estimat prin intermediul TFD. Se va considera un semnal sinusoidal de frecvență 10*Hz*, eșantionat la 100*Hz*. Să se studieze apoi efectul procedeului de *zero-padding*și cazurile în care numărul de perioade este întreg sau nu.

```
figure(1)
t = 0:0.01:0.5-0.01;
x = cos(20*pi*t);
N = length(x) ; X = fft(x,N);
fp = (0:N-1)/N/0.01; fp = fp - 1/2/0.01;
stem(fp,fftshift(abs(X)));grid ;
axis([ -1/(2*0.01) 1/(2*0.01) 0 25]);
xlabel('frecventa (Hz)')
ylabel('spectru de amplitudine')

figure(2)
X = fft(x,20*N); N = length(X);
fp = (0:N-1)/N/0.01;
fp = fp - 1/2/0.01;
plot(fp,abs(fftshift(X)));grid ;
axis([ -1/(2*0.01) 1/(2*0.01) 0 25]);
xlabel('frecventa (Hz)')
ylabel('spectru de amplitudine')

figure(3)
t = 0:0.01:0.5-0.01-1/20;
x = cos(20*pi*t); N = length(x);
```

```
X = fft(x,N);
fp = (0:N-1)/N/0.01;
fp = fp - 1/2/0.01;
plot(fp,abs(fftshift(X)));grid ;
axis([ -1/(2*0.01) 1/(2*0.01) 0 25]);
xlabel('frecventa (Hz)'); ylabel('spectru de
amplitudine')
```

3.3.2 Influența tipului ferestrei asupra spectrului estimat cu TFD

a) Să se genereze o sinusoidă de amplitudine 1*V*, de frecvență 50*Hz*, eșantionată la 256*Hz*, în 32 de puncte. Să se calculeze TFD în 1024 de puncte considerând succesiv o fereastră dreptunghiulară, apoi triunghiulară, Hamming, Hanningși Blackman. Să se concluzioneze asupra rezoluțiilor spectrale și dinamice.

```
t=(1:32);f1=50;Fe=256;Nfft=1024 ;
y1=sin(2*pi*f1/Fe*t);sig=y1.*boxcar(32)' ;
y_rect=abs(fftshift((fft(sig,Nfft))));
sig=y1.*triang(32)';
y_tria=abs(fftshift((fft(sig,Nfft))));
sig=y1.*hamming(32)' ;
y_hamm=abs(fftshift((fft(sig,Nfft))));
sig=y1.*hanning(32)' ;
y_hann=abs(fftshift((fft(sig,Nfft))));
sig=y1.*blackman(32)';
y_blac=abs(fftshift((fft(sig,Nfft))));
f=[-Fe/2:Fe/Nfft:(Fe/2-Fe/Nfft)];
subplot(511)
semilogy(f(513:1024),y_rect(513:1024));
axis([0 128 1e-3 100]);grid
legend('fereastra dreptunghiulara',-1)
subplot(512);semilogy(f(513:1024),y_tria(513:1024));
axis([0 128 1e-3 100]);grid
legend('fereastra triunghiulara',-1)
subplot(513);semilogy(f(513:1024),y_hamm(513:1024));
grid;ylabel('amplitudine spectrala')
axis([0 128 1e-4 100]);legend('fereastra Hamming',-1)
subplot(514);semilogy(f(513:1024),y_hann(513:1024));
axis([0 128 1e-3 100]);grid ;
legend('fereastra Hanning',-1)
subplot(515);semilogy(f(513:1024),y_blac(513:1024));
axis([0 128 1e-3 100]);grid
legend('fereastra Blackman',-1)
xlabel('frecventa (Hz)')
```

b) Să se genereze un semnal compus din suma a două sinusoide de frecvențe 100*Hz*și 94*Hz*și de amplitudine 1*V*. Se va considera aceeași lungime a semnalului de 32 puncte, iar TFD va fi calculată tot în 1024 de puncte.

```
t=(1:32);
f1=94;f2=100 ;
Fe=256;Nfft=1024 ;
y1= sin(2*pi*f1/Fe*t)+ sin(2*pi*f2/Fe*t);
sig=y1.*boxcar(32)' ;
y_rect=abs(fftshift((fft(sig,Nfft))));
sig=y1.*triang(32)';
y_tria=abs(fftshift((fft(sig,Nfft))));
sig=y1.*hanning(32)' ;
y_hann=abs(fftshift((fft(sig,Nfft))));
sig=y1.*hamming(32)' ;
y_hamm=abs(fftshift((fft(sig,Nfft))));
sig=y1.*blackman(32)';
y_blac=abs(fftshift((fft(sig,Nfft))));
f=[-Fe/2:Fe/Nfft:(Fe/2-Fe/Nfft)];
subplot(511)
plot(f(513:1024),y_rect(513:1024));
grid;axis([0 128 0 20])
legend('fereastra dreptunghiulara',-1)
subplot(512);
plot(f(513:1024),y_tria(513:1024));grid;
axis([0 128 0 7])
legend('fereastra triunghiulara',-1)
subplot(513);
plot(f(513:1024),y_hann(513:1024));
grid;
axis([0 128 0 7])
legend('fereastra Hanning   ',-1)
ylabel('amplitudine spectrala')
subplot(514);
plot(f(513:1024),y_hamm(513:1024));
grid;
axis([0 128 0 7])
legend('fereastra Hamming ',-1)
subplot(515);
plot(f(513:1024),y_blac(513:1024));
grid;
axis([0 128 0 6])
legend('fereastra Blackman',-1)
xlabel('frecventa (Hz)')
```

c) Să se genereze semnalul următor în 32 de puncte:

$$x[n]=\sin\left(2\pi\frac{100}{256}n\right)+0{,}1\sin\left(2\pi\frac{74}{256}n\right)$$

Reprezentați spectrul său utilizând diverse ponderări și interpretați rezultatele.

```
t=(1:32);f1=74;f2=100 ;Fe=256;Nfft=1024 ;
y1= 0.1*sin(2*pi*f1/Fe*t)+ sin(2*pi*f2/Fe*t);
sig=y1.*boxcar(32)';
y_rect=abs(fftshift(fft(sig,Nfft)));
sig=y1.*triang(32)';
y_tria=abs(fftshift(fft(sig,Nfft)));
sig=y1.*hanning(32)';
y_hann=abs(fftshift(fft(sig,Nfft)));
sig=y1.*hamming(32)';
y_hamm=abs(fftshift(fft(sig,Nfft)));
sig=y1.*blackman(32)';
y_blac=abs(fftshift(fft(sig,Nfft)));
f=[-Fe/2:Fe/Nfft:(Fe/2-Fe/Nfft)];
subplot(511)
plot(f(513:1024),y_rect(513:1024));grid;axis([0 128 0
20])
legend('fereastra dreptunghiulara',-1)
subplot(512);plot(f(513:1024),y_tria(513:1024));grid;
axis([0 128 0 10])
legend('fereastra triunghiulara',-1)
subplot(513);plot(f(513:1024),y_hann(513:1024));grid;
axis([0 128 0 10])
legend('fereastra Hanning',-1)
ylabel('amplitudine spectrala')
subplot(514);plot(f(513:1024),y_hamm(513:1024));grid;
axis([0 128 0 10])
legend('fereastra Hamming ',-1)
subplot(515);plot(f(513:1024),y_blac(513:1024));grid;
axis([0 128 0 8])
legend('fereastra Blackman',-1); xlabel('frecventa
(Hz)')
```

8. Să se calculeze convoluția ciclică a două secvențe, *x* și *h*, de aceeași perioadă *N*, utilizând TFD_{1D}.

```
x=[1,2]; h=[3,4];
X=fft(x);H=fft(h);Y=X.*H;
y=abs(ifft(Y))
```

3.3.3 Analiza în domeniul frecvenţă a unui semnal folosind microcontrolerul dsPIC33FJ256GP506

9. Să se determine spectrul unui semnal audio folosind microcontrolerul dsPIC33FJ256GP506, de pe placa de dezvoltare *DSC Starter Kit*.

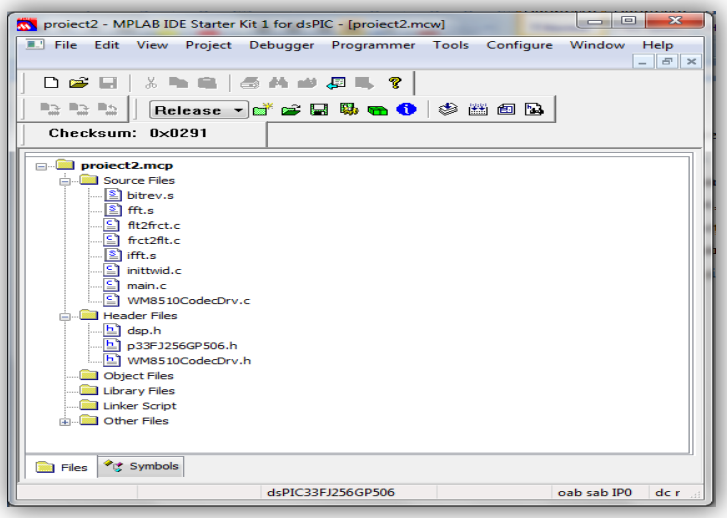
Semnalul audio analizat este sub forma unui vector preluat de la convertorul analog-digital. Frecventa de esantionare este de 8*kHz*.

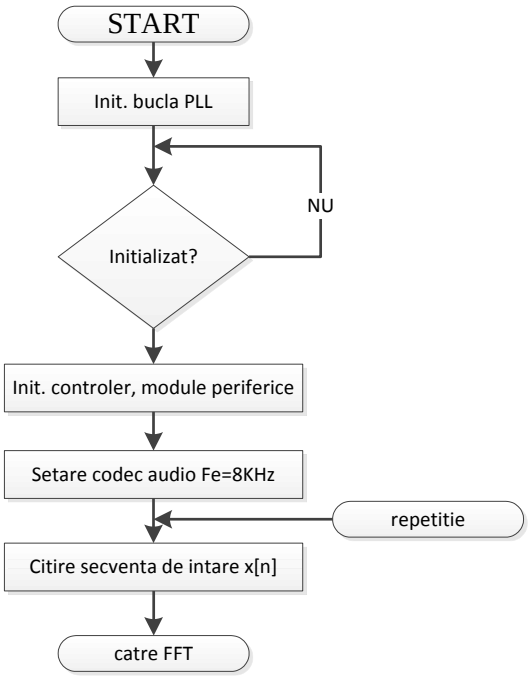
Algoritmul pentru determinarea spectrului unui semnal implică următorii paşi:

1. Iniţializarea coeficienţilor folosiţi în algoritmul TFR;
2. Calculul transformatei Fourier;
3. Inversarea biţilor rezultatelor obţinute, deoarece rutina folosită la pasul anterior furnizează rezultatul cu biţii în ordine inversă;
4. Calculul modulului numerelor complexe obţinute anterior.
5. Vizualizarea rezultatului in modul Debug.

MPLAB cuprinde rutine de program scrise în limbaj de asamblare şi în limbaj C, prin care se poate calcula spectrul unui semnal parcurgând paşii prezentaţi. Mai jos este prezentată o imagine captură în care sunt prezente toate părţile programului realizat.

Organigrama programului principal a fost împărţită în patru părţi şi anume partea de iniţializare a plăcii a driverelor, partea de transformată Fourier rapidă, aplicaţia ce se doreşte a fi implementă şi partea de transformată Fourier rapidă inversă.





După cum reiese şi din imaginea alăturata prima etapă consta in configurarea oscilatorului după care se testează dacă este pornit iar dacă este continuăm prin iniţializarea plăcii şi a driverelor după care prin pornirea funcţiei de prelucrare a intrări şi ieşiri audio, se trece în continuare la configurarea codecului audio pentru a funcţiona la frecvenţă de eşantionare de 8khz iar după aceste iniţializări începe execuţia buclei principale în care se face TFR, APLICAŢIA şi TFRI. Mai departe se testează codecul WM8510 dacă este ocupat, daca nu este, se trece la obţinerea eşantioanelor audio după care la aplicarea transformatei Fourier rapide.

În program dimensiunea fiecărui cadru audio, lungimea blocului de TFR şi numărul etapelor de tip fluture sunt adăugate cu următoarele comenzi:

```
#define FRAME_SIZE
#define FFT_BLOCK_LENGTH
#define LOG2_BLOCK_LENGTH
```

Instanţierea structurii driverelor şi crearea legăturilor se face în modul următor:

```
WM8510Handle codec;
WM8510Handle * codecHandle = &codec;
```

Pentru a iniţializa placa şi driverele, pentru a porni funcţiile audio de intrare şi ieşire şi pentru configurarea codecului audio să funcţioneze la 8kHz se folosesc următoarele linii de cod:

```
WM8510Init(codecHandle,codecBuffer);
WM8510Start(codecHandle);
WM8510SampleRate8KConfig(codecHandle);
```

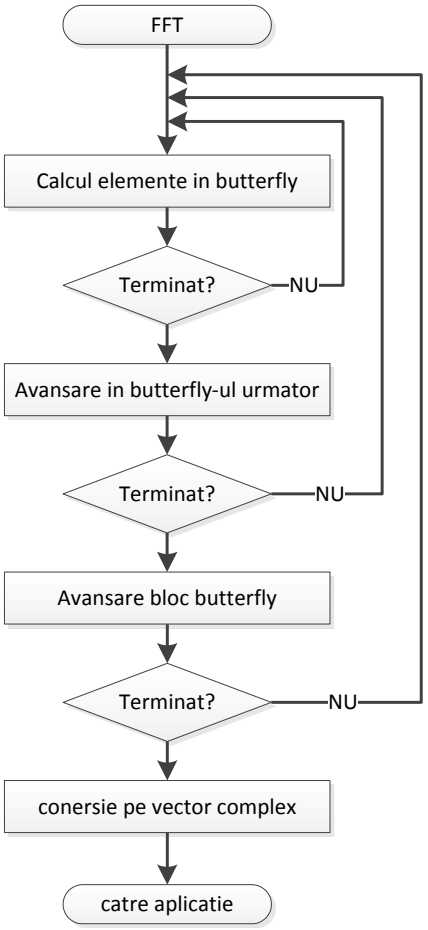
Factorii de rotire în program sunt iniţializa şi în bucla principală de procesare, executată pentru fiecare cadru de intrare în modul următor:

```
TwidFactorInit (LOG2_BLOCK_LENGTH, &twiddleFactors[0], 0);
TwidFactorInit (LOG2_BLOCK_LENGTH, &twiddleFactorsc[0], 1)
```

Liniile de cod pentru obţinerea eşantioanelor audio sunt următoarele:

```
while(WM8510IsReadBusy(codecHandle));
WM8510Read(codecHandle,samples,FRAME_SIZE);
```

Schema din figura alăturata reprezintă organigrama software a Transformatei Fourier Rapide în care testăm printr-un contor dacă lungimea blocului de lungime FFT a fost depăşită în caz că da se face toată partea imaginară zero şi se retestează iar în caz că nu a fost depăşită se trece la calcularea factorilor de rotire (twiddle factors). În continuare se testează dacă am depăşit jumătate din lungimea blocului de lungime FFT iar dacă am depăşit transformăm matricea eşantioanelor de la intrare într-o matrice complexă şi reluăm testarea iar în caz că nu mergem spre executarea Transformatei Fourier Inverse (IFFT).



Conversia matricei de eşantioane reale de la intrare într-o matrice de eşantioane complexe pentru lungimea blocului de FFT s-a realizat într-o instrucţiune de tip for:

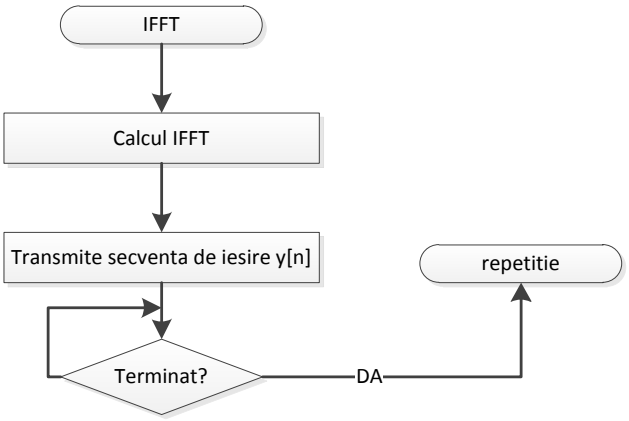
```
     for   (   i   =   0;   i   <
FFT_BLOCK_LENGTH; i++ )
      {
            sigCmpx[i].real    =
samples[i]*2;
      sigCmpx[i].imag          =
0x0000;
      }
```

Pentru trecerea în complex, biţii reversivi şi transformata Fourier rapidă se scriu următoarele linii de cod:

```
FFTComplexIP (LOG2_BLOCK_LENGTH, &sigCmpx[0],
&twiddleFactors[0],COEFFS_IN_DATA);
BitReverseComplex (LOG2_BLOCK_LENGTH , &sigCmpx[0]);
```

Convertirea matricei de eşantioane reale de la intrare într-o matrice de eşantioane complexe pentru lungimea blocului de FFT/2 s-a realizat într-o instrucţiune de tip for:

```
for ( i = 0; i < FFT_BLOCK_LENGTH/2; i++ ) //
Converteste matricea reala de esantioane de la intrare la
matricea complexa de esantioane de la intrare
{
      schimb = sigCmpx[i];
      sigCmpx[i] = sigCmpx[i+FFT_BLOCK_LENGTH/2];
      sigCmpx[i+FFT_BLOCK_LENGTH/2] =schimb ;
}
```



Organigrama Transformatei Fourier Rapide Inverse este asemnănătoare cu caclulul Transformatei Fourier rapide şi se începe prin calcularea factorilor de rotire şi a biţilor reversivi, testăm dacă am depăşit lungimea blocului de FFT/2, dacă da convertim matricea eşantioanelor de la intrare într-o matrice complexă şi retestăm. Dacă nu a fost depăşită mergem şi testăm dacă codecul audio WM8510 este ocupat. Dacă acesta este ocupat, testăm până când acesta va fi liber, iar dacă nu este ocupat mergem şi scriem în codecul audio WM8510 după care ne întoarcem în bucla principală.

Transformata Fourier Rapidă Inversă în program are următoarea linie de cod:

```
IFFTComplexIP (LOG2_BLOCK_LENGTH, &sigCmpx[0],
&twiddleFactorsc[0],COEFFS_IN_DATA);
```

Convertirea matricei de eşantioane reale de la intrare într-o matrice de eşantioane complexe pentru lungimea blocului de FFT s-a realizat într-o instrucţiune de tip for unde de asemeni în samples avem şi semnalul nostru rezultat.

```
for ( i = 0; i < FFT_BLOCK_LENGTH; i++ )
 {
       samples[i] = (10*sigCmpx[i].real * 40);
 }
```

Pentru a trimite secventa de date la codecul audio sunt necesare urmatoarele linii de cod:

```
  /* Asteapta pâna când codecul este disponibil pentru un
nou cadru*/
  while(WM8510IsWriteBusy(codecHandle));
  WM8510Write (codecHandle,samples,FRAME_SIZE);
```

```
În program avem _FFTComplexIP: Complex în locul DIF FFT.
Operatii:
F(k) = 1/N*sum_n (f(n)*WN(kn)), WN(kn) = exp[-
(j*2*pi*k*n)/N],
n in {0, 1,... , N-1}, si
k in {0, 1,... , N-1}, cu N = 2^m.
Intrare:
w0 = numar de etape în FFT (log2N)
w1 = pointer la sursa vectoriala complexa (srcCV)
w2 = pointer la factorul de rotatie complex
(twidFactors)
w3 = COEFFS_IN_DATA, sau pagina programului memoriei cu
factorul de rotatie .
Întoarce:
w0 = pointer la vectorul sursa (srcCV)
```

Funcţiile folosite sunt:

```
externfractcomplex*TwidFactorInit(int log2N,
fractcomplex*twidFactors, intconjFlag);
```

Inițializează coeficienții de rotație folosiți în cadrul algoritmului TFR. Aceștia sunt calculați ca numere reale, după care se realizează conversia lor în numere fracționare. Acceptă ca argumente $\log_2 N$, unde *N* reprezintă numărul de puncte în care se calculează transformata Fourier, pointer-ul către vectorul factorilor de rotație (**twidFactors* – *twiddlefactors*) și un indicator care specifică dacă dorim generarea conjugatelor complexe ale coeficienților (*conjFlag*; 0 semnifică se dorește acest lucru).

```
externfractcomplex*FFTComplexIP(int                    log2N,
fractcomplex*srcCV,              fractcomplex*twidFactors,
intfactPage);
```

Calculează transformata Fourier a unei secvențe, înlocuind valorile inițiale cu cele obținute prin calcul (*IP – In Place*). Astfel, se utilizează mai eficient memoria disponibilă. Acceptă ca argumente $\log_2 N$, unde *N* reprezintă numărul de puncte în care se calculează transformata Fourier, vectorul complex pentru care se calculează transformata Fourier (*srcCV – source complex vector*), pointer-ul către vectorul factorilor de rotație (practic, adresa primului element din vector) și o variabilă care indică locația în care sunt stocați coeficienții de rotație, în memoria de date *X* (caz în care, se introduce constanta COEFS_IN_DATA, definită în *dsp.h*) sau în memoria de program (în acest caz se introduce numărul paginii la care se află aceștia). Vectorul returnat de această funcție conține componentele spectrale, având valorile în format fracționar complexși cu biții inversați.

```
externfractcomplex*BitReverseComplex(int log2N,
fractcomplex*srcCV) ;
```

Efectuează operația de inversare a biților unui număr fracționar complex. Acceptă ca argumente $\log_2 N$, unde *N* reprezintă numărul de puncte în care se calculează transformata Fourier, și pointer-ul către vectorul pentru care se aplică funcția (*srcCV*).

```
externfractional*SquareMagnitudeCplx(intnumelems,
fractcomplex*srcV, fractional*dstV);
```

Calculează modulul numerelor complexe fracționare, acestea fiind elemente ale unui vector. Trebuie precizat vectorul pentru care se aplică funcția (*srcV – source vector*), dimensiunea sa (*numelems – number of elements*) și vectorul în care se stochează rezultatele, ca numere fracționare reale (*dstV – destination vector*).

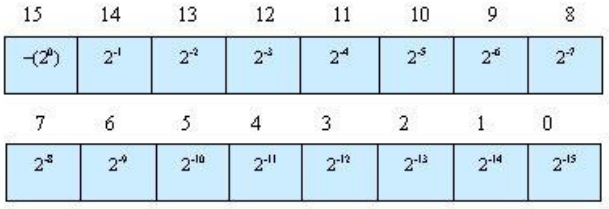
```
externfractional Float2Fract (floataVal);
```

Realizează conversia dintr-un număr real, de tip *float*, într-un număr fracționar, în gama $[-1,1)$.

```
externfloat Fract2Float (fractionalaVal).
```

Realizează conversia dintr-un număr fracționar, în gama $[-1,1)$, într-un număr real, de tip *float*.

În general, algoritmii implementați pe procesoare de semnal sunt proiectați astfel încât să lucreze cu numere fracționare, care au gama de reprezentare $[-1,1)$. Unul din motivele pentru care se folosește acest format este că produsul a două numere fracționare este mai mic decât acestea[4]. În aplicația noastră, se folosește formatul *1.15*, care utilizează un bit de semn și 15 biți fracționari, așa cum se observă mai jos:



De exemplu, numărul 0011 0000 00000000, sau 0x3000, în format fracționar, reprezintă valoarea 0,375 (0,25+0,125).

Precizia limitată a formatului *1.15* produce erori de cuantizare. De exemplu, cea mai apropiată reprezentare a numărului 0,2, în formatul *1.15*, este 0x199A, echivalentă însă cu 0,20001220703125, rezultând o eroare de cuantizare de $1,220703125 \cdot 10^{-5}$.

Scalarea valorilor eșantioanelor în intervalul $[-0,5;0,5)$ se face printr-o deplasare la dreapta cu un bit, adică o împărțire la doi.

Fișierele menționate mai sus vor fi copiate în proiectul creat, în directorul *src*. Directorul *h* va conține fișierele antet. Utilizarea celor două directoare este utilă la identificarea fișierelor în cazul în care proiectul cuprinde mai multe.

Așadar, în directorul *h* se vor copia:

- *dspcommon.inc* – conține declarații ale unor regiștrișsi configurări pentru calcule cu numere fracționare, fiind utilizat în rutinele de asamblare; se găsește în *C:\Microchip Starter Kits\dsPIC Starter Kit - 1\MPLAB C30\src\dsp\inc*;
- *dsp.h* – conține declarațiile funcțiilor folosite pentru algoritmul TFR; se găsește în *C:\Microchip Starter Kits\dsPIC Starter Kit - 1\MPLAB C30\src\dsp\include*.

În directorul *src* se vor copia:

[4]Kester, W., *Mixed-signaland DSP design techniques*, Elsevier Science, 2003

- *bitrev.s* – rutina de inversare a biților;
- *cplxsqrmag.s* – rutina de calcul a modului valorilor complexe;
- *fft.s* – rutina de calcul a transformatei Fourier;
- *flt2frct.c* –funcția de conversie din numere reale de tip *float* în numere fracționare;
- *frct2flt.c* – funcția de conversie din numere fracționare în numere reale de tip *float*;
- *initwid.c* – rutina de generare a coeficienților de rotație.

Toate aceste fișiere se găsesc în *C:\Microchip Starter Kits\dsPIC Starter Kit - 1\MPLAB C30\src\dsp\asm*, respectiv *C:\Microchip Starter Kits\dsPIC Starter Kit - 1\MPLAB C30\src\dsp\src*.

Pentru a nu exista probleme la compilare, se adaugă în arborele de căutare locația *D:\Projects\h*, iar în arborele de căutare pentru fișiere scrise în limbaj de asamblare (*Assembler Include SearchPath*), locația *D:\Projects\src*.

Programul principal folosit:

```
#include "..\h\p33FJ256GP506.h"
#include "..\h\dsp.h"

_FGS(GWRP_OFF & GCP_OFF);//Faraprotectie la scriere si
faraprotectia codului
_FOSCSEL(FNOSC_FRC);//Se foloseste oscilatorul RC rapid (Fast
RC)
_FOSC(FCKSM_CSECMD & OSCIOFNC_ON & POSCMD_NONE);//Se
activeaza comutarea ceasurilor, OSC2 ca pin digital, se
dezactiveaza oscilatorul primar
_FWDT(FWDTEN_OFF);//Se opreste cronometrul watchdog

#define FRAME_SIZE 128
#define LOG2_BLOCK_LENGTH 7

//Coeficientii de rotatie sunt salvati in memoria de date X
fractcomplextwiddleFactors     [FRAME_SIZE/2] __attribute__
((section(".xbss, bss, xmemory"), aligned(FRAME_SIZE*2)));
//Rezultatul TFR este salvat in memoria de date Y
fractcomplexsigCmpx            [FRAME_SIZE] __attribute__
((far)) _YBSS(FRAME_SIZE *2*2);
fractional     magsig                  [FRAME_SIZE/2];

//Programul principal
intmain(){

        //Configurarea oscilatorului astfel incat
dispozitivul sa opereze la 40MHz
```

```
        //Fosc= Fin*M/(N1*N2), Fcy=Fosc/2
        //Fosc= 7.37M*40/(2*2)=80Mhz pentru tact intrare de
7.37M
        PLLFBD=41;                      //M=39
        CLKDIVbits.PLLPOST=0;           //N1=2
        CLKDIVbits.PLLPRE=0;            //N2=2
        OSCTUN=0;

        __builtin_write_OSCCONH(0x01);        //Initializare
oscilator la FRC (Fast RC) cu PLL
        __builtin_write_OSCCONL(0x01);
        while(OSCCONbits.COSC!=0b01);  //Astepta realizarea
comutarii
        while(!OSCCONbits.LOCK);

        //Declararea variabilelor
        intsamples[FRAME_SIZE]={0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0x
        A57F,0x8001,
        0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,
        0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,
        0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,
        0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,
        0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,
        0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,0x0000,
        0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,
        0x8001,0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,
        0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,
        0x0000,0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,
        0x5A81,0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,
        0x7FFF,0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,
        0x5A81,0x0000,0xA57F,0x8001,0xA57F,0x0000,0x5A81,0x7FFF,0x5A81,
        0x0000,0xA57F,0x8001,0xA57F};

        int j;
        float spectru [FRAME_SIZE/2];//Jumatate din spectrul
calculat, pentru optimizarea memoriei folosite
        //Datorita simetriei, cealaltajumatate este in
oglinda
        floattemp;

        //Initializari
        TwidFactorInit(LOG2_BLOCK_LENGTH,&twiddleFactors[0],0
);//Initializareacoeficientilorfolositi in algoritmul TFR

        //Bucla de program
        while(1){
                //Scalarea datelor in intervalul [-0,5, 0,5)
                for(j=0; j<FRAME_SIZE; j++){   //Crearea unui
vector de numere complexe, cu partea imaginara nula
                        temp=(float)(samples[j]);
                        sigCmpx[j].real= Float2Fract(temp)>>1;
```

```
                        sigCmpx[j].imag=0x0000;
                }


        FFTComplexIP(LOG2_BLOCK_LENGTH,&sigCmpx[0],&twiddleFa
ctors[0],COEFFS_IN_DATA);

        BitReverseComplex(LOG2_BLOCK_LENGTH,&sigCmpx[0]);

        SquareMagnitudeCplx(FRAME_SIZE/2,&sigCmpx[0],&magsig[
0]);//magsig = |FFT|

                for(j=0;j<FRAME_SIZE/2;j++)
                        spectru[j]= Fract2Float(magsig[j]);

                while(1);//Ramane in bucla
        }
}
```

Spectrul obținut:

Frecvența de eșantionare este de $8kHz$, iar transformata Fourier a fost determinată în 128 de puncte. Rezultă un pas de:

$$\frac{8000}{128} = 62,5Hz$$

Deci, componenta spectrală determinată se află la frecvența:

$$62,5 \cdot 17 = 1062,5Hz$$

3.4 Desfăşurarea lucrării

1. Se vor rula în MATLAB toate aplicațiile descrise in breviarul teoretic şi se va urmări execuția lor.
2. Să se genereze şi să se reprezinte în timp şi frecvență următoarele semnale:

- semnal sinusoidal cu frecvența 8*kHz* eşantionat la 30*kHz*;
- semnalul format din suma a două semnale sinusoidale de 2*kHz*, respectiv 3*kHz*, eşantionate la 13*kHz*;
- semnal dreptunghiular cu perioada de 125μs, eşantionat la 15*kHz*;

3. Să se realizeze modulația în amplitudine având semnalul modulator $x[n]=\sin(2\pi 0,01\cdot n)+\cos(2\pi 0,03\cdot n)$ şi semnalul modulat $s[n]=\sin(2\pi 0,2\cdot n)$, unde $n\in N$ şi $0\le n\le 300$. Gradul de modulație *m* este 0,6. Să se reprezinte semnalele din punct de vedere spectral.
4. Să se genereze şi să se reprezinte în timp şi frecvență un semnal *chirp*, a cărui frecvență instantanee variază între 10*kHz*şi 10*MHz*, pe o durată $T=10\mu s$. Frecvența de eşantionare utilizată este $F_e=70MHz$.
5. Folosind funcția MATLAB pentru calculul TFTD prezentată în lucrare să se calculeze TFTD şi să se reprezinte grafic modulul şi faza pentru următoarele semnale:

- $x[n]=\sin(0,3n)$ în 64 de puncte;
- $x[n]=\log(0,2n)$ în 256 de puncte.

4 Aplicaţii ale sistemelor în timp discret

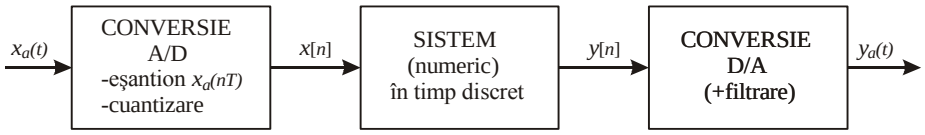
4.1 Scopul lucrării

Sistemele în timp discret procesează secvenţe discrete in timp aplicate la intrare şi generează alte secvenţe discrete în timp la ieşire având o serie de proprietăţi simple. Scopul acestei lucrări este de a realiza simulări ale sistemelor discrete în timp şi verificarea proprietăţilor acestora. Simulările vor fi realizate în MATLAB, şi folosind microcontrolerul dsPIC33FJ256GP506.

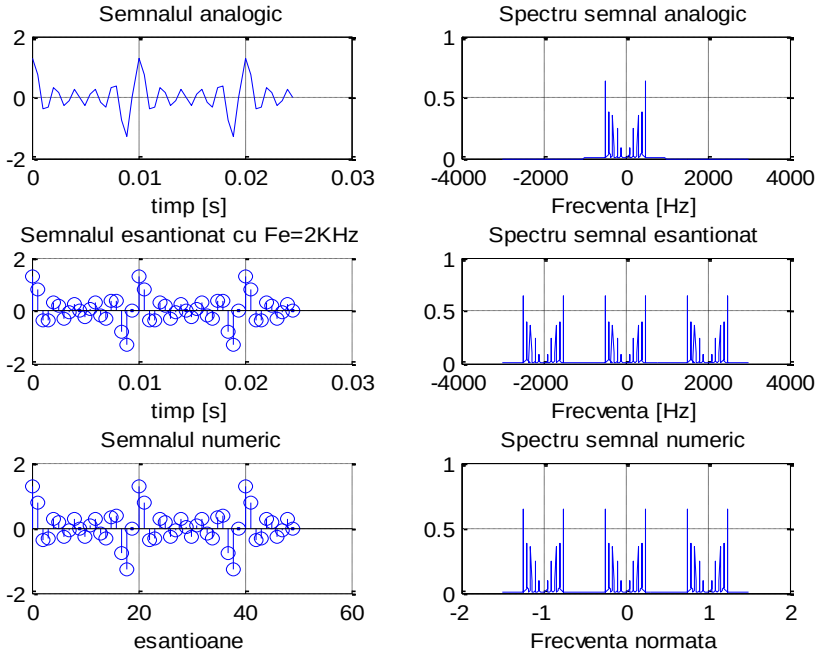
4.2 Noţiuni teoretice

4.2.1 Prelucrarea semnalelor în timp discret

Sistemul reprezintă „o colecţie de elemente între care există anumite relaţii de interdependenţă, subordonate unui anumit scop”[5]. Cu referire la sistemul electric, un sistem se poate defini ca „o mulţime de circuite interconectate astfel încât se pot identifica perechi de borne numite porţi de intrare şi perechi de borne numite porţi de ieşire”[6].
În figura de mai jos se prezintă locul unui sistem de prelucrare numerică în cadrul unui lanţ de prelucrare a unor semnale analogice.

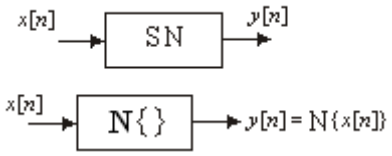


În continuare, sunt ilustrate, în domeniile timp şi frecvenţă, etapele de conversie a unui semnal analogic într-un semnal numeric (frecvenţa maximă din spectrul semnalului analogic este 500*Hz*, iar frecvenţa de eşantionare este 2*kHz*):



4.2.2 Modelarea matematică a sistemelor numerice

Vom nota cu SN (sistem numeric) un procesor de semnale numerice, reprezentat în figura de mai jos cu o intrare și o ieșire și prin $N\{\ \}$, un operator de prelucrare în timp discret.



4.2.3 Proprietăți generale ale sistemelor numerice

Pentru un sistem în timp discret cu secvențe de intrare $x_1[n]$ și $x_2[n]$ care generează secvențele de ieșire $y_1[n]$, respectiv $y_2[n]$, este ***liniar*** dacă pentru o

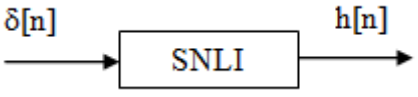
secvență de intrare $x[n]=\alpha\cdot x_1[n]+\beta\cdot x_2[n]$ se obține la ieșire secvența $y[n]=\alpha\cdot y_1[n]+\beta\cdot y_2[n]$.
Proprietatea de superpoziție din relația secvenței de ieșire trebuie să se păstreze pentru orice constante arbitrare α și β, și pentru orice secvență de intrare $x_1[n]$ și $x_2[n]$, pentru ca sistemul să rămână liniar.

Un sistem numeric al cărui răspuns este $y[n]$ pentru o secvență de intrare $x[n]$ este ***invariant*** dacă răspunsul său este $y[n-n_0]$ când la intrarea sa se aplică secvența $x[n-n_0]$. Variabila n_0 este un număr întreg pozitiv sau negativ. Dacă sistemul nu respectă relația de invarianță, atunci spunem că sistemul este variant în timp.

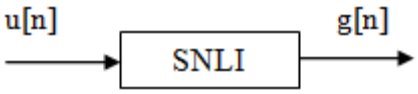
Un sistem numeric este *liniar și invariant în timp* dacă respectă ambele proprietăți de liniaritate și de invarianță.

Un sistem numeric discret în timp este ***cauzal*** (sau ***nonanticipativ***) dacă răspunsul său, $y[n]$ pentru orice *n*, depinde doar de valoarea semnalului de la intrare, $x[n]$, la acel moment și, eventual, la momentele *n-k* anterioare acestuia. Putem spune ca sistemul numeric este cauzal dacă satisface condiția $h[n]=0, n<0$.

Un sistem timp discret care are o secvență finită la intrare $x[n]$ și generează o secvență finită la ieșire, $y[n]$ (se găsește sub denumirea de *BIBO - Bounded Input Bounded Output*) este ***stabil*** dacă secvența de intrare are valori finite $|x[n]|<B_x$, iar pentru secvența de ieșire valorile sunt de asemenea finite $|y[n]|<B_y$ pentru orice valoare a lui *n*. B_x și B_y sunt constante cu valori finite.
Răspunsul unui sistem numeric în timp discret la o secvența impuls unitate $\delta[n]$ se numește ***răspunsul la impuls*** (sau ***funcția pondere***) și este notat cu $h[n]$.



Răspunsul unui sistem numeric în timp discret o secvența treaptă unitate $u[n]$ se numește ***răspunsul la treaptă unitate*** (sau ***funcția indicială***) și este notat cu $g[n]$.



Răspunsul unui sistem liniar și invariant în timp, $y[n]$, caracterizat de funcția pondere $h[n]$ la o secvență de intrare $x[n]$, este dat de relația:

$$y[n]=\sum_{k=-\infty}^{\infty} x[n-k]h[k]$$

care poate fi scris și sub forma:

$$y[n]=\sum_{k=-\infty}^{\infty} x[n]h[n-k]$$

Aceste sume se numesc convoluția a două secvențe $x[n]$ și $h[n]$, și sunt reprezentate prin operatorul „*”. Convoluția a două secvențe este notată sub forma: $y[n]=x[n]*h[n]$.

Dacă funcția pondere a unui sistem liniar și invariant timp este $h[n]=h_1[n]*h_2[n]$, rezultatul conectării în serie a două sisteme numerice liniare și invariante în timp cu funcțiile $h_1[n]$ și $h_2[n]$, iar dacă $h_1[n]*h_2[n]=\delta[n]$, atunci spunem că funcția pondere $h_1[n]$ este inversa funcției $h_2[n]$, și reciproc.

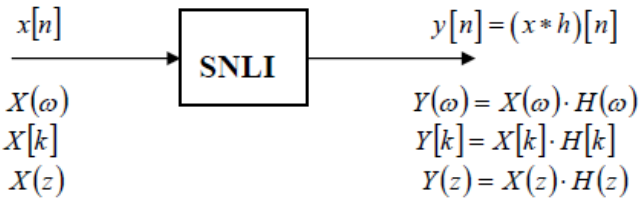
Un sistem numeric liniar și invariant de tip BIBO, stabil, cu funcția pondere $h[n]$ este ***absolut sumabilă*** dacă:

$$\sum_{n=-\infty}^{\infty} |h[n]| < \infty$$

Un sistem numeric liniar și invariant în timp este numit ***sistem cu răspuns finit impuls finit*** (*FIR – Finite Impulse Response*) dacă funcția pondere (răspunsul său la impuls) are lungime finită. În caz contrar, este un ***sistem cu impuls infinit la impuls*** (*IIR – Infinite Impuls Response*).

4.2.4 Analiza sistemelor numerice liniare şi invariante în timp

În general, un sistem numeric liniar şi invariant în timp poate fi caracterizat de secvenţele de intrare-ieşire şi transformatele lor, aşa cum se prezintă în figura de mai jos:



În domeniul timp, corespondenţa intrare-ieşire este definită de relaţii liniare între $x[n]$ şi $y[n]$:

$$y[n]=\sum_{i=0}^{M} b_i \cdot x[n-i]-\sum_{i=1}^{N} a_i \cdot y[n-i]=\{b_0 \cdot x[n]+b_1 \cdot x[n-1]+...+b_M \cdot x[n-M]\}-$$
$$-\{a_1 \cdot y[n-1]+a_2 \cdot y[n-2]+...+a_N \cdot y[n-N]\}$$

Funcţia de transfer $H(z)$ a sistemului este definită astfel:

$$H(z)=\frac{Y(z)}{X(z)}=\frac{b_0+b_1 z^{-1}+b_2 z^{-2}+...+b_M z^{-M}}{a_0+a_1 z^{-1}+a_2 z^{-2}+...+a_N z^{-N}}\Bigg|_{a_0=1}=\frac{\sum_{i=0}^{M} b_i z^{-i}}{1+\sum_{i=0}^{N} a_i z^{-i}}$$

Răspunsul pondere al sistemelor numerice liniare şi invariante în timp la excitaţia particulară

Considerăm că la intrarea unui sistem numeric se aplică un impuls Dirac. Rezultă că:

$$h[n]=N\{\delta[n]\}$$

Datorită proprietăţii de invarianţă, rezultă că:

$$h[n-k]=N\{\delta[n-k]\}$$

Deoarece o secvenţă oarecare se poate reprezenta prin:

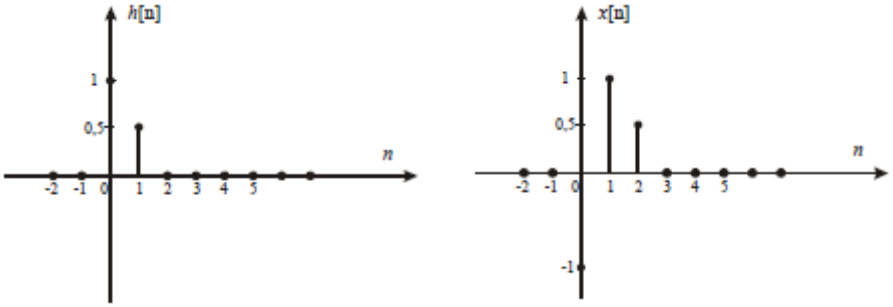
$$x[n]=\sum_k x[k]\delta[n-k]$$

rezultă că răspunsul sistemului la o secvență oarecare $x[n]$ este date de relația:

$$y[n]=N\{x[n]\}=N\left\{\sum_k x[k]\delta[n-k]\right\}=\sum_k x[k]N\{\delta[n-k]\}=$$
$$=\sum_k x[k]h[n-k]=\sum_k x[n-k]h[k]=(x*h)[n]$$

De exemplu, fie un sistem numeric cu funcția pondere $h[n]$, respectiv excitația $x[n]$ aplicată la intrarea sa:

$$h[n]=\begin{cases}1, & n=0\\ 0,5, & n=1\\ 0, & n>1\end{cases} \qquad x[n]=\begin{cases}-1, & n=0\\ 1, & n=1\\ 0,5, & n=2\\ 0, & n\geq 3\end{cases}$$



Conform relației generale $y[n]=\sum_{k=0}^{\infty}h[k]x[n-k]$, obținem:

$$n=0\Rightarrow y[0]=h[0]\cdot x[0]=-1$$
$$n=1\Rightarrow y[1]=h[0]\cdot x[1]+h[1]\cdot x[0]=1-0,5=0,5$$
$$n=2\Rightarrow y[1]=h[0]\cdot x[2]+h[1]\cdot x[1]=1\cdot 0,5+1\cdot 0,5=1$$
$$n=3\Rightarrow y[3]=h[1]\cdot x[2]=0,5\cdot 0,5=0,25$$
$$n\geq 4\Rightarrow y[n]=0$$

Răspunsul pondere al sistemelor numerice liniare şi invariante în timp la excitaţia particulară

Dacă la intrarea unui sistem numeric liniar şi invariant în timp se aplică o secvenţă treaptă unitate $u[n]$, rezultă la ieşirea sa răspunsul indicial $g[n]$:

$$g[n] = N\{u[n]\}$$

Dar $u[n] = \sum_{k=0}^{\infty} \delta[n-k]$. Rezultă că răspunsul indicial va fi:

$$g[n] = N\left\{\sum_{k=0}^{\infty} \delta[n-k]\right\} = \sum_{k=0}^{\infty} N\{\delta[n-k]\} = \sum_{k=0}^{\infty} h[n-k]$$

Cum $\delta[n] = u[n] - u[n-1]$, rezultă că răspunsul pondere $h[n]$ poate fi scris sub forma:

$$h[n] = N\{\delta[n]\} = N\{u[n] - u[n-1]\} = N\{u[n]\} - N\{u[n-1]\} = g[n] - g[n-1]$$

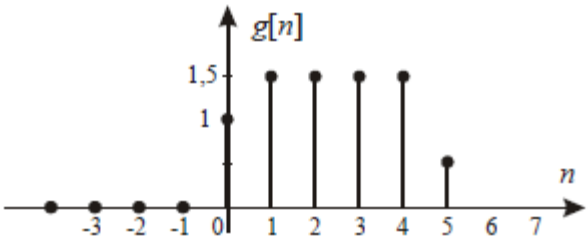
De exemplu, fie un sistem numeric cu funcţia pondere $h[n]$:

$$h[n] = \begin{cases} 1, & n = 0 \\ 0,5, & n = 1 \\ 0, & n > 1 \end{cases}$$

Rezultă că răspunsul indicial $g[n] = \sum_{k=0}^{\infty} h[n-k]$ are valorile:

$$n = 0 \Rightarrow g[0] = h[0] = 1$$
$$n = 1 \Rightarrow g[1] = h[1] + h[0] = 1,5$$
$$n = 2 \Rightarrow g[2] = h[2] + h[1] + h[0] = 1,5$$
$$n = 3 \Rightarrow g[3] = h[3] + h[2] + h[1] + h[0] = 1,5$$

Reprezentând grafic, obţinem:



4.2.5 Exemple de sisteme numerice simple

Circuitul de întârziere

Este caracterizat în domeniile analogic şi numeric de relaţiile:



Funcţia de transfer a circuitului de întârziere este:

$$H(z)=\frac{Y(z)}{X(z)}=z^{-1},$$

iar în domeniul frecvenţă:

$$H\left(e^{j\omega}\right)=e^{-j\omega}=\cos\omega-j\sin\omega\Rightarrow\left|H\left(e^{j\omega}\right)\right|^2=1$$

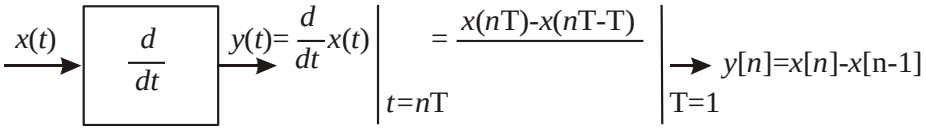
Secvenţa în MATLAB pentru reprezentarea caracteristicii de frecvenţă şi fază:

```
num=[0 1];      % coeficientii numaratorului functiei de
                % transfer H(z) in ordine descrescatoare
den=[1];        % coeficientii numitorului functiei de
                % transfer H(z) in ordine descrescatoare
[H,W]=freqz(num,den); % "freqz" calculeaza raspunsul
                      % in frecventa pentru H(z) dat?
                      % prin num si den
subplot(121);
plot(W/(2*pi),abs(H));grid;
title('|H(e^{j\omega})|');
ylabel('amplitudine');
xlabel('Frecventa normata');
subplot(122);
plot(W/(2*pi),angle(H)*180/pi);grid;
title('Caracteristica de faza');
```

```
ylabel('Faza [grade]');
xlabel('Frecventa normata');
```

Circuitul de derivare

Este caracterizat în domeniile analogic şi numeric de relaţiile:



Conform relaţiei $Y(z)=X(z)-z^{-1}X(z)$, rezultă funcţia de transfer a circuitului de derivare:

$$H(z)=\frac{Y(z)}{X(z)}=1-z^{-1}\Rightarrow H(e^{j\omega})=1-e^{-j\omega}=1-\cos\omega+j\sin\omega\Rightarrow\left|H(e^{j\omega})\right|^2=2(1-\cos\omega)$$

Secvenţa în MATLAB pentru reprezentarea caracteristicii de frecvenţă şi fază:

```
num=[1 -1];
den=1;
[H,W]=freqz(num,den);
subplot(121);
plot(W/(2*pi),abs(H));grid
title('|H(e^{j\omega})|');
ylabel('amplitudine');
xlabel('Frecventa normata');
subplot(122);
plot(W/(2*pi),angle(H)*180/pi);grid;
title('Caracteristica de faza');
ylabel('Faza [grade]');
xlabel('Frecventa normata');
```

Circuitul de integrare

Este caracterizat în domeniile analogic şi numeric de relaţiile:



Cum $x(t)=\frac{dy(t)}{dt}=\frac{y(nT)-y(nT-T)}{T}$, rezultă că $x[n]=y[n]-y[n-1]$. Astfel, conform relației $X(z)=Y(z)-z^{-1}Y(z)$, rezultă funcția de transfer a circuitului de derivare:

$$H(z)=\frac{Y(z)}{X(z)}=\frac{1}{1-z^{-1}}$$

Se obține:

$$H(e^{j\omega})=\frac{1}{1-e^{-j\omega}}=\frac{1}{1-\cos\omega+j\sin\omega}\Rightarrow\left|H(e^{j\omega})\right|^2=\frac{1}{2(1-\cos\omega)}$$

Secvența în MATLAB pentru reprezentarea caracteristicii de frecvență și fază:

```
num=[1];
den=[1 -1];
[H,W]=freqz(num,den);
subplot(121);
plot(W/(2*pi),abs(H));grid;
title('|H(e^{j\omega})|');
ylabel('amplitudine');
xlabel('Frecventa normata');
subplot(122);
plot(W/(2*pi),angle(H)*180/pi);grid;
title('Caracteristica de faza');
ylabel('Faza [grade]');
xlabel('Frecventa normata');
```

Convoluția a două secvențe discrete

Să se calculeze pas cu pas convoluția a două secvențe discrete. Originea celor două funcții este presupusă la $n=0$. Pentru a vedea cum se realizează pas cu pas operația de convoluție, după rularea programului se va apăsa orice tastă de mai multe ori. Pentru reluarea simulării se execută din nou programul.

```
x=ones(3,1); h=exp(-[1:10]);
Ly=length(x)+length(h)-1;
lh=length(h);lx=length(x); y=zeros([Ly,1]);
xf=fliplr(x);
disp(['Apasati pe orice tasta pentru a calcula convolutia']);
for i=1:1:Ly
    indlo=max(0,i-lx); indhi=min(i-1,lh-1);
    for j=indlo:indhi
        y(i)=h(j+1)*x(i-j)+y(i);
```

```
    end
    subplot(311);stem(-lx+i:1:i-1,xf);
    ylabel('x in oglinda');
    lim1= max(lh,lx) ;lim2= min(min(min(x),min(h)),0) ;
    lim3= max(max(x),max(h)) ;
    axis([-lx,lim1,lim2,lim3]);
    subplot(312);stem(0:1:lh-1,h);ylabel('h');
    axis([-lx,lim1,lim2,lim3]);
    subplot(313);stem(0:1:Ly-1,y);ylabel('y');
    pause
end
```

Convoluția, corelația și filtrajul adaptat

Să se scrie un program care să arate că operația de convoluție realizată de un filtru adaptat este echivalentă cu corelația dintre semnalul cu care acesta este adaptat și semnalul de intrare.
Se va considera semnalul de intrare:

$$x\{n\} = \{1, -2, 3, -4, 3, 2, 1\}$$

și filtrul adaptat caracterizat prin funcția pondere:

$$h\{n\} = \{3, 2, 1, -2, 1, 0, -4, 0, 3\}$$

Programul MATLAB:

```
h = [3 2 1 -2 1 0 -4 0 3];
x = [1 -2 3 -4 3 2 1];
xa = fliplr(h);
y1 = conv(h,x) ;
y2 = filter(h,1,[x zeros(1,length(x)+1)]);
y3 = xcorr(x,xa);
subplot(311);stem(y1); ylabel('Amplitudine');
title('Iesire obtinuta prin convolutie'); grid;
subplot(312);stem(y2); ylabel('Amplitudine');
title('Iesire obtinuta prin filtraj'); grid;
subplot(313);stem(y3)
xlabel('Index temporal n'); ylabel('Amplitudine');
title('Iesire obtinuta prin corelatie'); grid;
```

4.3 Aplicații

4.3.1 Sisteme numerice liniare 1D

1. Să se verifice că sistemul numeric reprezentat prin funcția de transfer:

$$H(z)=\frac{2,2403+2,4908z^{-1}+2,2403z^{-2}}{1-0,4z^{-1}+0,75z^{-2}}$$

este liniar și invariant în timp.

a) Verificarea liniarității sistemului în MATLAB:

```
clear; n = 0:40; a = 2;b = -3;
x1 = cos(2*pi*0.1*n); x2 = cos(2*pi*0.4*n);
x = a*x1 + b*x2;
num = [2.2403 2.4908 2.2403]; den = [1 -0.4 0.75];
ic = [0 0];
% functia filter implementeaza un filtru cu functia de
% transfer data prin coeficientii numaratorului si
% numitorului si realizeaza filtrarea semnalului x1
y1 = filter(num,den,x1,ic); y2 = filter(num,den,x2,ic);
y = filter(num,den,x,ic); yt = a*y1 + b*y2; d = y - yt;
figure
subplot(3,1,1); stem(n,y);grid; ylabel('Amplitudine');
title('Iesirea pt intrarea ponderata: a\cdotx_{1}[n] + b
\cdot x_{2}[n]');
subplot(3,1,2); stem(n,yt);grid; ylabel('Amplitudine');
title('Iesire: a \cdot y_{1}[n] + b \cdot y_{2}[n]');
subplot(3,1,3); stem(n,d);grid;
xlabel('Index temporal n'); ylabel('Amplitudine');
title(' Semnal diferenta');
```

b) Verificarea că sistemul este invariant, în MATLAB:

```
clear; n = 0:40; D = 10;a = 3.0;b = -2;
x = a*cos(2*pi*0.1*n) + b*cos(2*pi*0.4*n);
xd = [zeros(1,D) x];
num = [2.2403 2.4908 2.2403]; den = [1 -0.4 0.75];
ic = [0 0];
y = filter(num,den,x,ic); yd = filter(num,den,xd,ic);
d = y - yd(1+D:41+D);
figure
subplot(3,1,1);stem(n,y); ylabel('Amplitudine');
```

```
title('Iesire y[n]'); grid;
subplot(3,1,2);stem(n,yd(1:41)); ylabel('Amplitudine');grid;
title(['Iesire pt intrarea decalata x[n -', num2str(D),']']);
subplot(3,1,3); stem(n,d);
xlabel('Index temporal n'); ylabel('Amplitudine');
title(' Semnal diferenta'); grid;
```

4.3.2 Funcția de transfer, polii și zerourile unui sistem numeric

2. Fie un sistem numeric liniar și invariant în timp caracterizat în domeniul timp discret de ecuația:

$$y[n]-0,334y[n-1]=0,334x[n]+0,334x[n-1]$$

Să se determine funcția de transfer, polii și zerourile acestui sistem.

Funcția de transfer se deduce ușor din ecuația sistemului numeric:

$$H(z)=\frac{Y(z)}{X(z)}=\frac{0,344+0,344z^{-1}}{1-0,344z^{-1}}$$

Determinarea polilor și zerourilor în MATLAB:

```
clear;
b=[0.334,0.334]; a=[1.0,-0.334];
figure(1) ;
% se afiseaza polii si zerourile folosind functia zplane
% care are ca parametrii coeficientii numaratorului
% si numitorului
zplane(b,a) ;
figure(2) ;
% se reprezinta grafic in modul si faza functia de
% transfer folosind functia freqz
freqz(b,a);
```

4.3.3 Funcția pondere a unui sistem numeric

3. a) Să se determine funcția pondere pentru un sistem numeric liniar și invariant în timp definit prin funcția de transfer:

$$H(z)=\frac{2,2403+2,4908z^{-1}+2,2403z^{-2}}{1-0,4z^{-1}+0,75z^{-2}}$$

b) Să se compare funcțiile pondere pentru două filtre trece jos de tip Butterworth și Cauer de ordinul 5, având frecvența de tăiere normată de 0,25.

a) Programul MATLAB:

```
clear; N = 40;
num=[2.2403 2.4908 2.2403];
den= [1 -0.4 0.75];
% functia pondere se determina folosind functia impz ce
% are ca parametri coeficientii functiei de transfer si
% numarul de puncte in care sa se calculeze functia
y = impz(num,den,N); figure
stem(y);xlabel('Index temporal n');
ylabel('Amplitudine'); title('Functia pondere'); grid;
```

b) Programul MATLAB:

```
clear;[b,a]=butter(5,0.5);h=impz(b,a,50);
figure(1);subplot(211);stem(h);
grid; title('butterworth ')
subplot(212);[H,W]=freqz(b,a);
plot(W/(2*pi), abs(H));grid;
title('Caracteristica de transfer');
figure(2);subplot(211);
[b,a]=ellip(5,1,20,0.5);h=impz(b,a,50);
stem(h); grid; title('cauer ')
subplot(212);[H,W]=freqz(b,a);
plot(W/(2*pi), abs(H));grid;
title('Caracteristica de transfer');
```

4.3.4 Stabilitatea sistemelor numerice

4. Să se arate că sistemul numeric caracterizat prin funcția de transfer:

$$H(z)=\frac{1-0,8z^{-1}}{1+1,5z^{-1}+0,9z^{-2}}$$

este stabil.

Pentru a determina dacă un sistem numeric este stabil există două variante:
- se demonstrează că răspunsul la impuls tinde spre zero;
- dacă toți polii sistemului numeric se află în interiorul cercului unitate, atunci sistemul numeric este stabil.

Programul MATLAB:

```
clear; num = [1 -0.8]; den = [1 1.5  0.9];
N = 200; h = impz(num,den,N+1);
```

```
parsum = 0;
for k = 1:N+1;
    parsum = parsum + abs(h(k));
    if abs(h(k)) < 10^(-6),
        break,
    end
end
figure
n = 0:N;stem(n,h); grid;
xlabel('Index temporal n'); ylabel('Amplitudine');
figure
zplane(num,den); disp('Sum =');disp(abs(h(k)));
```

Conectarea în cascadă a sistemelor numerice

5. Fie două sisteme numerice definite prin funcțiile de transfer următoare:

$$H_1(z)=\frac{0,3-0,2z^{-1}+0,4z^{-2}}{1+0,9z^{-1}+0,8z^{-2}} \qquad H_2(z)=\frac{0,2-0,5z^{-1}+0,3z^{-2}}{1+0,7z^{-1}+0,85z^{-2}}$$

Arătați că funcția de transfer a sistemului numeric obținut prin conectarea în cascadă a celor două sisteme este egală cu produsul funcțiilor de transfer ale acestora.

Să se calculeze produsul dintre cele două funcții de transfer și să se verifice coeficienții trecuți mai jos la *num* și *den*.

Programul MATLAB:

```
clear; x = [1 zeros(1,40)];n = 0:40;
den = [1 1.6 2.28 1.325 0.68];
num = [0.06 -0.19 0.27 -0.26 0.12];
y = filter(num,den,x);
num1 = [0.3 -0.2 0.4];den1 = [1 0.9 0.8];
num2 = [0.2 -0.5 0.3];den2 = [1 0.7 0.85];
y1=filter(num1,den1,x);y2=filter(num2,den2,y1);d=y-y2;
figure;subplot(3,1,1); stem(n,y);ylabel('Amplitudine');
title('Iesirea sistemului rezultat'); grid;
subplot(3,1,2); stem(n,y2);ylabel('Amplitudine');
title('Iesirea celor doua sisteme conectate in cascada');
grid;
subplot(3,1,3); stem(n,d)
xlabel('Index temporal n');ylabel('Amplitudine');
title(' Semnal diferenta'); grid;
```

4.3.5 Efectul zerourilor şi polilor asupra funcţiei de transfer a unui sistem numeric

6. a) Fie sistemul reprezentat prin funcţia de transfer $H(z)=1-0,5z^{-1}$, prezentând un singur zero la $z=0,5$. Să se studieze variaţia lui $\left|H\left(e^{j\omega}\right)\right|$ la apropierea zeroului de cercul unitate.

b) Procedaţi în acelaşi mod pentru sistemul reprezentat prin funcţia de transfer $H(z)=\dfrac{1}{1-0,5z^{-1}}$, care are un singur pol, la $z=0,5$.

a) Programul MATLAB:

```
b(1,1:2)=[1 -0.4];b(2,1:2)=[1 -0.6];
b(3,1:2)=[1 -0.8];b(4,1:2)=[1 -1];
figure;hold on;tipcul=['b';'r';'g';'k'];
for k=1:4
   H=freqz(b(k,:),1); plot(abs(H),tipcul(k,:))
end
axis([0 512 0 2.2]);grid on
legend('zerou=.4','zerou=.6','zerou=.8','zerou=1',2)
title('Influenta zerourilor asupra functiei de transfer');
```

b) Programul MATLAB:

```
a(1,1:2)=[1 -0.5];a(2,1:2)=[1 -0.6];
a(3,1:2)=[1 -0.7];a(4,1:2)=[1 -0.8];
figure;hold on
for k=1:4
   H=freqz(1,a(k,:));plot(abs(H),tipcul(k,:))
end
axis([0 512 0 5.5]);grid on;
legend('pol=.5','pol=.6','pol=.7','pol=.8');
title('Influenta polilor asupra functiei de transfer');
```

4.3.6 Implementarea unui sistem numeric pe microcontrolerul dsPIC33FJ256GP506

7. Se consideră un sistem liniar şi invariant în timp caracterizat de relaţia:

$$y[n]=\frac{1}{4}\left(x[n]+x[n-1]+x[n-2]+x[n-3]\right)$$

Acesta se mai numește filtru cu medie alunecătoare de ordinul 4. Să se implementeze acest sistem numeric pe microcontrolerul dsPIC33FJ256GP506. Se vor realiza două secvențe sinusoidale, $x_1[n]$ și $x_2[n]$, cu frecvențele de 10*Hz*, respectiv 40*Hz*, cu frecvența de eșantionare de 100*Hz* și cu lungimea 100 de eșantioane. Se va efectua suma acestora, obținând secvența $x[n]=x_1[n]+x_2[n]$. Aplicați funcția pondere ce caracterizează sistemul din relația de mai sus. În final, trasați grafic toate semnalele: $x_1[n]$, $x_2[n]$, $x[n]$ și $y[n]$ obținute în urma operațiilor efectuate. Se va folosi și programul MATLAB.

Frecvențele unghiulare în timp discret ale celor două secvențe sinusoidale sunt calculate prin relația:

$$\omega = 2\pi \frac{f_{semnal}}{f_{esantionare}}$$

Relațiile care le caracterizează sunt:

$$x_1[n] = \sin(2\pi 0{,}1 \cdot n)$$

$$x_2[n] = \sin(2\pi 0{,}4 \cdot n)$$

Funcția pondere se determină prin aplicarea la intrare a semnalului impuls unitate:

$$h\{n\} = \left\{\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}\right\}$$

Programul MATLAB:

```
N=100;
n = 0:N-1;
y = zeros(1,N);
x1 = sin(2*pi*0.1*n); % prima secventa
x2 = sin(2*pi*0.4*n); % a doua secventa
x = x1+x2;
% Implementarea filtrului cu medie alunecatoare
for k=4:N
    y(k)=(x(k)+x(k-1)+x(k-2)+x(k-3))/4;
end
% Afisarea tuturor semnalelor
clf;
subplot(2,2,1);
stem(n, x1);
axis([0, 100, -2, 2]);
xlabel('n'); ylabel('Amplitudine');
title('Secventa x1');
subplot(2,2,2);
stem(n, x2);
```

```
axis([0, 100, -2, 2]);
xlabel('n'); ylabel('Amplitudine');
title('Secventa x2');
subplot(2,2,3);
stem(n, x);
axis([0, 100, -2, 2]);
xlabel('n'); ylabel('Amplitudine');
title('Secventa x');
subplot(2,2,4);
stem(n, y);
axis([0, 100, -2, 2]);
xlabel('n'); ylabel('Amplitudine');
title('Secventa y');
axis;
```

Liniile de program:

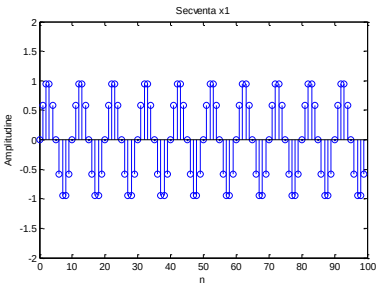
```
for k=4:N
    y(k)=(x(k)+x(k-1)+x(k-2)+x(k-3))/4;
end
```

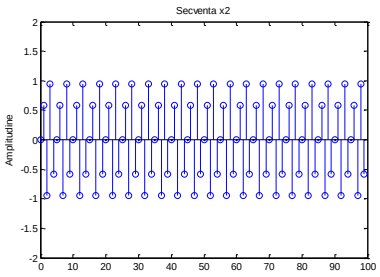
pot fi înlocuite cu:

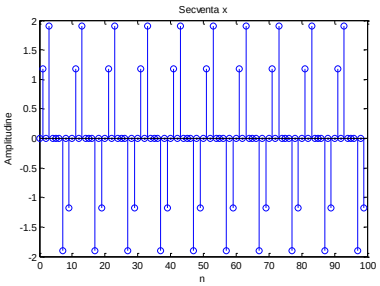
```
num = [1/4 1/4 1/4 1/4];
y = filter(num,1,x);
```

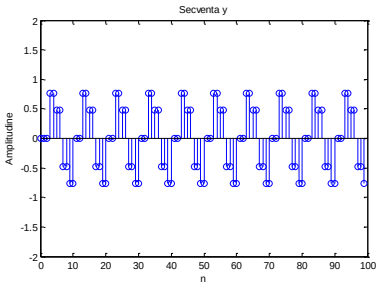
prin utilizarea funcției *filter*.

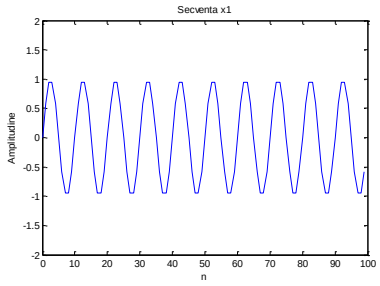
Rezultatele simulării:

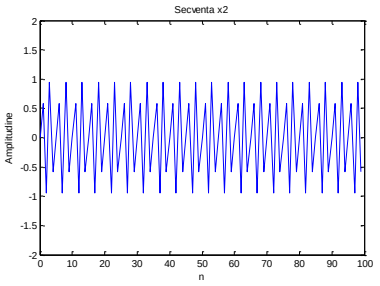

Secventa x1
Amplitudine
n

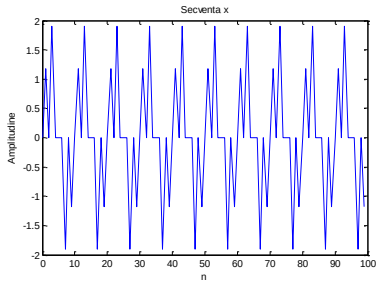

Secventa x2
Amplitudine
n

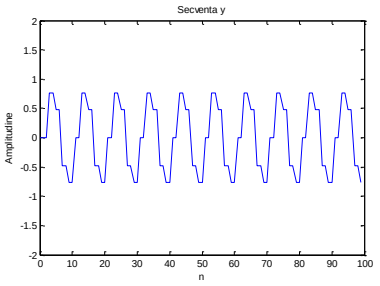

Secventa x
Amplitudine
n


Secventa y
Amplitudine
n

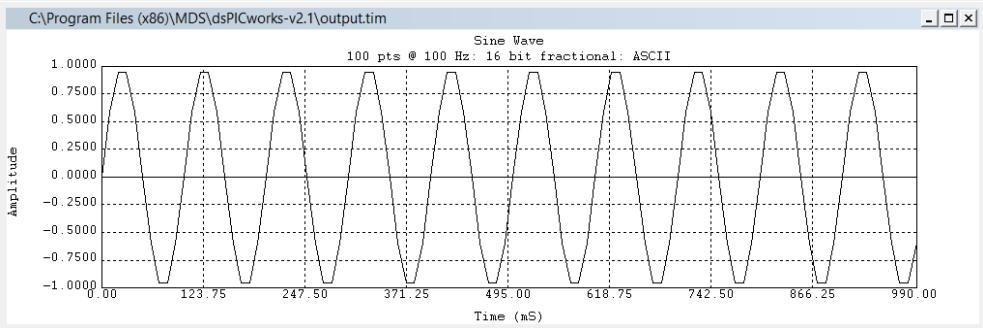

Secventa x1
Amplitudine
n

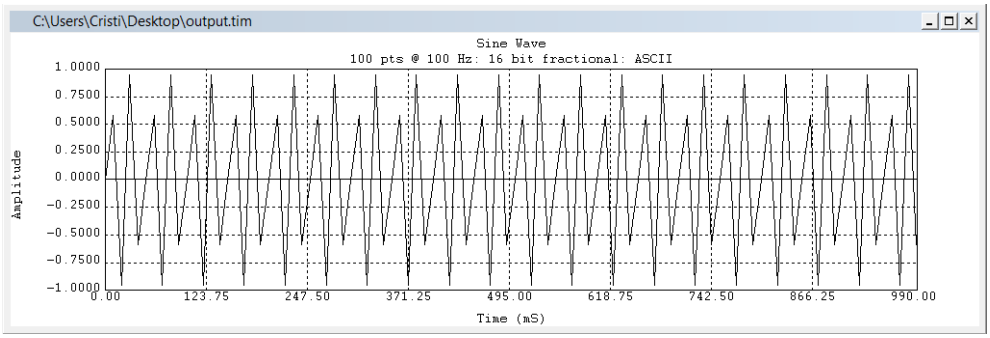

Secventa x2
Amplitudine
n


Secventa x
Amplitudine
n


Secventa y
Amplitudine
n

Pentru simularea pe microcontroler a sistemului numeric am generat cele două secvențe $x_1[n]$ și $x_2[n]$, folosind programul *dsPICworks v2.1*. Procedura de realizare a fost descrisă în laboratorul anterior. Secvențele $x_1[n]$, respectiv $x_2[n]$:





Programul principal folosit:

```
#include "..\h\p33FJ256GP506.h"

_FGS(GWRP_OFF & GCP_OFF); //Fara protectie la scriere si fara
protectia codului
_FOSCSEL(FNOSC_FRC); //Se foloseste oscilatorul RC rapid
(Fast RC)
_FOSC(FCKSM_CSECMD & OSCIOFNC_ON & POSCMD_NONE); //Se
activeaza comutarea ceasurilor, OSC2 ca pin digital, se
dezactiveaza oscilatorul primar
_FWDT(FWDTEN_OFF); //Se opreste cronometrul watchdog

#define FRAME_SIZE 100

//Programul principal
int main (){
```

```
        //Configurarea oscilatorului astfel incat
dispozitivul sa opereze la 40MHz
        //Fosc= Fin*M/(N1*N2), Fcy=Fosc/2
        //Fosc= 7.37M*40/(2*2)=80Mhz pentru tact intrare de
7.37M
        PLLFBD=41;                                          //M=39
        CLKDIVbits.PLLPOST=0;          //N1=2
        CLKDIVbits.PLLPRE=0;           //N2=2
        OSCTUN=0;

        __builtin_write_OSCCONH(0x01);        //Initializare
oscilator la FRC (Fast RC) cu PLL
        __builtin_write_OSCCONL(0x01);
        while (OSCCONbits.COSC != 0b01);      //Astepta
realizarea comutarii
        while(!OSCCONbits.LOCK);

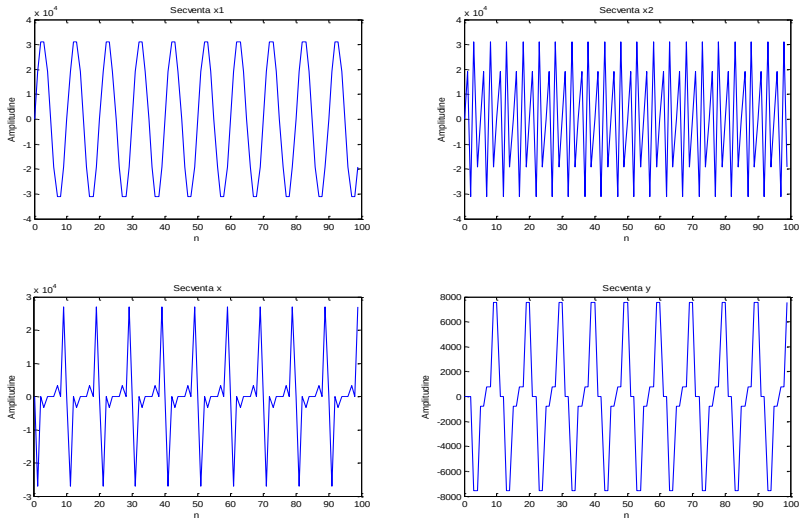
        //Declararea variabilelor
        //Semnalele de intrare - sinusoidale, esantionate cu
frecventa de 100Hz
        int x1[FRAME_SIZE] =
{0x0000,0x4B3B,0x79BB,0x79BB,0x4B3B,0x0000,0xB4C5,0x8645,0x86
45,0xB4C5,
0x0000,0x4B3B,0x79BB,0x79BB,0x4B3B,0x0000,0xB4C5,0x8645,0x864
5,0xB4C5,
0x0000,0x4B3B,0x79BB,0x79BB,0x4B3B,0x0000,0xB4C5,0x8645,0x864
5,0xB4C5,
0x0000,0x4B3B,0x79BB,0x79BB,0x4B3B,0x0000,0xB4C5,0x8645,0x864
5,0xB4C5,
0x0000,0x4B3B,0x79BB,0x79BB,0x4B3B,0x0000,0xB4C5,0x8645,0x864
5,0xB4C5,
0x0000,0x4B3B,0x79BB,0x79BB,0x4B3B,0x0000,0xB4C5,0x8645,0x864
5,0xB4C5,
0x0000,0x4B3B,0x79BB,0x79BB,0x4B3B,0x0000,0xB4C5,0x8645,0x864
5,0xB4C5,
0x0000,0x4B3B,0x79BB,0x79BB,0x4B3B,0x0000,0xB4C5,0x8645,0x864
5,0xB4C5,
0x0000,0x4B3B,0x79BB,0x79BB,0x4B3B,0x0000,0xB4C5,0x8645,0x864
5,0xB4C5
};
        int x2[FRAME_SIZE] =
{0x0000,0x4B3B,0x8645,0x79BB,0xB4C5,0x0000,0x4B3B,0x8645,0x79
BB,0xB4C5,
0x0000,0x4B3B,0x8645,0x79BB,0xB4C5,0x0000,0x4B3B,0x8645,0x79B
B,0xB4C5,
```

```
0x0000,0x4B3B,0x8645,0x79BB,0xB4C5,0x0000,0x4B3B,0x8645,0x79B
B,0xB4C5,
0x0000,0x4B3B,0x8645,0x79BB,0xB4C5,0x0000,0x4B3B,0x8645,0x79B
B,0xB4C5,
0x0000,0x4B3B,0x8645,0x79BB,0xB4C5,0x0000,0x4B3B,0x8645,0x79B
B,0xB4C5,
0x0000,0x4B3B,0x8645,0x79BB,0xB4C5,0x0000,0x4B3B,0x8645,0x79B
B,0xB4C5,
0x0000,0x4B3B,0x8645,0x79BB,0xB4C5,0x0000,0x4B3B,0x8645,0x79B
B,0xB4C5,
0x0000,0x4B3B,0x8645,0x79BB,0xB4C5,0x0000,0x4B3B,0x8645,0x79B
B,0xB4C5,
0x0000,0x4B3B,0x8645,0x79BB,0xB4C5,0x0000,0x4B3B,0x8645,0x79B
B,0xB4C5,
0x0000,0x4B3B,0x8645,0x79BB,0xB4C5,0x0000,0x4B3B,0x8645,0x79B
B,0xB4C5
};
        int x[FRAME_SIZE];
        float y[FRAME_SIZE];
        int i;

        //Initializari
        y[0] = 0;
        y[1] = 0;
        y[2] = 0;

        //Bucla de program
        while(1){
                //Calculul secventei suma
                for (i=0;i<FRAME_SIZE;i++)
                        x[i] = x1[i] + x2[i];
                //Filtrarea
                for (i=3;i<FRAME_SIZE;i++)
                y[i] = (float)(0.25*(x[i]+x[i-1]+x[i-2]+x[i-
3]));
                while(1); //Ramane in bucla dupa ce parcurge
secventa de intrare
        }
}
```

Secvențele au fost citite din fereastra *Watch*, utilizând modul *Debug* al programului MPLAB IDE. Rezultatele simulării, ilustrate folosind funcția *plot* din MATLAB:



4.4 Desfăşurarea lucrării

1. Se vor rula în MATLAB toate aplicaţiile descrise în breviarul teoretic şi se va urmări execuţia lor.

2. Să se verifice că sistemele numerice reprezentate prin următoarele funcţii de transfer:

$$H_1(z)=\frac{0,3-0,2z^{-1}+0,4z^{-2}}{1+0,9z^{-1}+0,8z^{-2}} \qquad H_2(z)=\frac{0,2-0,5z^{-1}+0,3z^{-2}}{1+0,7z^{-1}+0,85z^{-2}}$$

sunt sisteme liniare şi invariante în timp, folosind MATLAB.

3. Să se calculeze funcţiile de transfer ale sistemelor numerice descrise de următoarele ecuaţii diferenţiale:

$$y[n]=5x[n]-5x[n-1]+0,4x[n-2]+0,32x[n-3]-$$
$$-0,5y[n-1]+0,34y[n-2]+0,08y[n-3]$$

$$y[n]=5x[n]-10,5x[n-1]+11,7x[n-2]+0,3x[n-3]-4,4x[n-4]-$$
$$-0,9y[n-1]+0,76y[n-2]+0,016y[n-3]+0,096y[n-4]$$

Să se reprezinte grafic în modul și fază funcțiile de transfer calculate și diagramele poli-zerouri. Să se determine răspunsul sistemelor la semnalul impuls unitate și treaptă unitate. Să se precizeze dacă sistemele numerice sunt stabile. Se vor folosi programele MATLAB. Implementați funcțiile de transfer și pe microcontrolerul dsPIC33FJ256GP506.

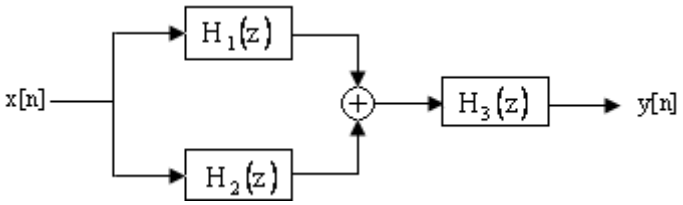
4. Determinați răspunsul la impuls $h[n]$ pentru următoarele funcții de transfer:

$$H(z)=\frac{-0,1z^{-1}+2,19z^{-2}}{\left(1-0,8z^{-1}+0,41z^{-2}\right)\left(1+0,3z^{-1}\right)}$$

$$H(z)=\frac{6-z^{-1}}{1+0,5z^{-1}}+\frac{2}{1-0,4z^{-1}}$$

Să se reprezinte diagramele poli-zerouri, răspunsul sistemelor la semnalul impuls unitate și treaptă unitate și să se specifice dacă sistemele numerice sunt stabile. Se vor folosi programele MATLAB.

5. Se consideră următoarea structură digitală:



unde:

$$H_1(z)=\frac{2}{3}+\frac{2}{5}z^{-1}+\frac{4}{7}z^{-2}$$

$$H_2(z)=\frac{4}{3}+\frac{8}{5}z^{-1}+\frac{3}{7}z^{-2}$$

$$H_1(z)=3+2z^{-1}+4z^{-2}$$

Să se determine funcția de transfer $H(z)$ a sistemului compus. Să se reprezinte grafic această funcție în modul și fază, folosind MATLAB.

5 Proiectarea şi implementarea filtrelor FIR

5.1 Scopul lucrării

Lucrarea are ca scop studiul, respectiv proiectarea filtrelor FIR (*Finite Impulse Response*) cu ajutorul programelor MATLAB şi dsPIC FD Lite. Se va realiza şi implementarea filtrelor pe dsPIC33FJ256GP506.

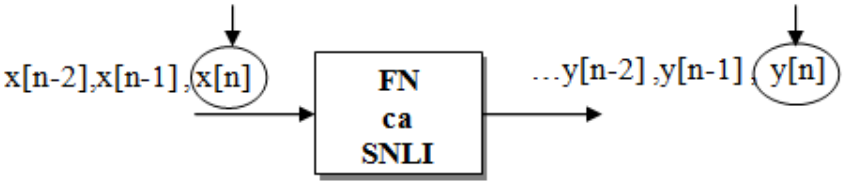
5.2 Noţiuni teoretice

Filtrul numeric este un sistem (în timp) discret caracterizat printr-un *algoritm numeric* cu ajutorul căruia o secvenţă de intrare se transformă într-o secvenţă (filtrată) de ieşire.

Algoritmul (de filtrare) poate să reprezinte: o filtrare trece jos, trece sus, trece bandă, opreşte bandă, sau o operaţie de diferenţiere, integrare etc.

5.2.1 Funcţia de transfer a unui filtru numeric

Să considerăm schema generală de filtrare a unei secvenţe de intrare $x[n]$ într-o secvenţă de ieşire $y[n]$:



Un FN poate fi caracterizat de ecuaţia în timp discret:

$$a_0 y[n] + a_1 y[n-1] + a_2 y[n-1] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2]$$

Dacă $a_0 = 1$, rezultă că secvenţa de ieşire este dată de relaţia:

$$y[n] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2] - a_1 y[n-1] - a_2 y[n-1]$$

sau:

$$y_n = b_0 x_n + b_1 x_{n-1} + b_2 x_{n-2} - a_1 y_{n-1} - a_2 y_{n-2}$$

Aplicând transformata *Z* ambilor membri rezultă că:

$$Y(z) = b_0 X(z) + b_1 z^{-1} X(z) + b_2 z^{-2} X(z) - a_1 z^{-1} Y(z) - a_2 z^{-2} Y(z)$$

Funcția de transfer a filtrului numeric considerat are expresia:

$$H(z) = \frac{Y(z)}{X(z)} = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}$$

5.2.2 Avantajele filtrelor numerice

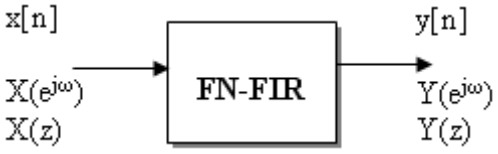
- flexibilitate prin modificarea algoritmului;
- ușor de proiectat, testat și implementat pe un microprocesor sau DSP;
- stabilitate ridicată în raport cu timpul și temperatura;
- siguranță în funcționare, dată de tehnologiile integrate;
- modularitate;
- versatilitate în prelucrarea semnalelor;
- aplicabilitate în joasă frecvență și radiofrecvență.

5.2.3 Etapele proiectării unui filtru numeric

1. Rezolvarea problemei aproximării, care conduce la obținerea funcției de transfer $H\left(e^{j\omega}\right)$;
2. Alegerea schemei de realizare;
3. Cuantizarea coeficienților funcției $H(z)$, corespunzătoare unei lungimi finite de reprezentare;
4. Cuantizarea variabilelor: semnalul de intrare, de ieșire și a semnalelor intermediare;
5. Verificarea prin simulare (*offline*) a modului în care filtrul proiectat satisface exigențele impuse.
   De regulă, etapa 5 conduce la reluarea etapelor 2-4.

5.3 FN cu răspuns finit la impuls

Să considerăm schema generală a unui filtru numeric de tip FIR:



În timp discret, putem nota:

$$y[n]=b_0x[n]+b_1x[n-1]+...+b_Mx[n-M]=\sum_{i=0}^{M}b_ix[n-i]$$

Aplicând transformata *Z* ambilor membri ai ecuației de mai sus se obține:

$$Y(z)=\sum_{i=0}^{M}b_iz^{-i}X(z)$$

În planul variabilei *z*, funcția de transfer a FN de tip FIR este:

$$H_{FIR}(z)=\frac{Y(z)}{X(z)}=\frac{Z\{y[n]\}}{Z\{x[n]\}}=\sum_{i=0}^{M}b_iz^{-i}$$

iar răspunsul pondere este dat de relația:

$$h_{FIR}[n]=Z^{-1}\{H_{FIR}(z)\}=\sum_{i=0}^{M}b_iZ^{-1}\{z^{-i}\}=\sum_{i=0}^{M}b_i\delta[n-i]=$$
$$=b_0\delta[n]+b_1\delta[n-1]+...+b_M\delta[n-M]$$

5.3.1 Proiectarea FN-FIR prin metoda ferestrelor (sau metoda seriei Fourier)

Algoritm:

1. Se dă răspunsul dorit pentru FN-FIR, notat cu $H_d(e^{j\omega})$, care este o funcție periodică în ω, cu perioada 2π;
2. Se determină răspunsul pondere al FN:

$$h_\infty[n]=TFTD^{-1}\{H_d(e^{j\omega})\}=\frac{1}{2\pi}\int_{2\pi}H_d(e^{j\omega})\cdot e^{j\omega n}d\omega$$

3. Se trunchiază răspunsul pondere $h_\infty[n]$ pentru a obține răspunsul pondere al unui FN-FIR:

$$h[n]=h_\infty[n]\cdot w[n]$$

unde $w[n]$ este o funcție fereastră de lungime *N*.

4. Se calculează funcția de transfer a FN-FIR:

$$H\left(e^{j\omega}\right) = TFTD\left\{h[n]\right\} = TFTD\left\{h_{\infty}[n]\cdot w[n]\right\}$$

5. Se compară funcția de transfer obținută $H\left(e^{j\omega}\right)$ cu funcția de transfer dorită $H_d\left(e^{j\omega}\right)$.

5.3.2 Proiectarea FN-FIR prin metoda eșantionării în frecvență

În proiectarea filtrelor FIR prin metoda eșantionării în frecvență se specifică răspunsul în frecvență dorit $H_d(\omega)$ într-un set de *M* frecvențe uniform distanțate în intervalul $[0, 2\pi]$, fie acestea:

$$\omega_k = \frac{2\pi}{M}(k+\alpha), \quad k = 0,1,\ldots(M-1)/2, \;\; M \;\; impar$$

$$k = 0,1,\ldots\left[(M/2)-1\right], \quad M \quad par$$

$$\alpha = \quad 0 \quad sau \quad 1/2$$

și din aceste specificații se calculează răspunsul la impuls al filtrului sintetizat, care să îndeplinească cerințele de proiectare. Suportul teoretic al acestei metode îl constituie transformata Fourier discretă.

Se impune identitatea între răspunsul filtrului dorit și cel sintetizat la frecvențele specificate și se calculează răspunsul la impuls $h[n]$.

5.3.3 Proiectarea FN-FIR prin optimizare

Fie:

- caracteristica dorită a unui FN:

$$H_d\left(e^{j\omega}\right) = H_d(\omega)$$

- și caracteristica unui FN tip FIR:

$$H_{FIR}\left(e^{j\omega}\right) = H_{FIR}(\omega) = H_{FIR}\left(\omega, \bar{b}\right)$$

unde $\bar{b}$ – coeficienți necunoscuți.

Pentru $\omega = \omega_i, \; i = \overline{1, M}$:

- *eroarea locală de aproximare* va fi:

$$\varepsilon\left(\omega_i, \bar{b}\right) = H_d(\omega_i) - H_{FIR}\left(\omega_i, \bar{b}\right)$$

- *eroarea globală de aproximare* va fi:

$$E\left(\bar{b}\right)=\sum_{i=1}^{M}\varepsilon^{2}\left(\omega_{i},\bar{b}\right)=\min$$

Dorim să minimizăm eroarea globală de aproximare prin alegerea corespunzătoare a coeficienților $\bar{b}$.

5.4 Aplicații FN-FIR în MATLAB

Comenzi MATLAB pentru proiectarea FN-FIR

Comandă	Descriere
fir1	metoda ferestrelor
fir2	metoda ferestrelor cu caracteristică de frecvență prescrisă
kaiserord	estimează parametrii pentru fir1 cu fereastra Kaiser
firls	utilizează metoda celor mai mici pătrate
remez	utilizează algoritmul Remez pentru proiectarea FIR
remezord	estimează ordinul FIR pentru utilizarea algoritmului Remez
fircls	utilizează metoda celor mai mici pătrate *cu constrângeri*
fircls1	utilizează metoda celor mai mici pătrate pentru FTJ și FTS
cremez	proiectează FIR cu fază neliniară echiriplu
firrcos	proiectează un FIR cu caracteristică tip cosinus ridicat

5.4.1 Implementarea FN-FIR prin metoda ferestrelor

Exemplu de proiectare a unui FN-FIR cu $n=51$ și $\omega t=0,4rad/s$ (se folosește o fereastră dreptunghiulară):

```
h=0.4*sinc (0.4*(-25:25));
```

```
subplot(211);
stem(h);grid
[H,w]=freqz (h,1,512,2);
subplot(212);
plot(w/2,abs (H)), grid
```

Exemple de proiectare a FN-FIR prin metoda ferestrelor

Se foloseşte funcţia `b=fir1(n,wn,'ftype',window)` unde:
n+1 – reprezintă ordinul filtrului;
wn – frecvenţa de tăiere (are valori cuprinse între 0 şi 1, unde 1 corespunde jumătăţii frecvenţei de eşantionare – frecvenţa Nyquist);
'*ftype*' – tipul filtrului ('*high*', '*stop*');
window – tip fereastră.

Exemplu de FN-FIR de tipul 1 şi 2

a) calculul răspunsurilor pondere şi vizualizarea lor prin intermediul a trei tipuri de ferestre: *dreptunghiulară, Hamming, Blackman***:**

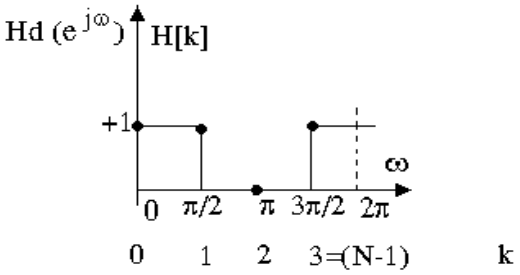
```
figure(1);
fd1=fir1(8,0.5,boxcar(9));fd2=fir1(9,0.5,boxcar(10));
subplot(321);stem(fd1);grid;subplot(322);stem(fd2);grid
;
fh1=fir1(8,0.5,hamming(9));fh2=fir1(9,0.5,hamming(10));
subplot(323);stem(fh1);grid;subplot(324);stem(fh2);grid
;
fb1=fir1(8,0.5,blackman(9));fb2=fir1(9,0.5,blackman(10)
);
subplot(325);stem(fb1);grid;subplot(326);stem(fb2);grid
;
```

b) calculul caracteristicilor de frecvenţă şi vizualizarea lor:

```
figure(2);
f=0:1/64:1-1/64;
Fd1=abs(fft(fd1,64)); Fd2=abs(fft(fd2,64));
subplot(321);plot(f,Fd1);grid;
subplot(322);plot(f,Fd2);grid;
Fh1=abs(fft(fh1,64));Fh2=abs(fft(fh2,64));
subplot(323);plot(f,Fh1);grid;
subplot(324);plot(f,Fh2);grid;
Fb1=abs(fft(fb1,64)); Fb2=abs(fft(fb2,64));
subplot(325);plot(f,Fb1);grid;
subplot(326);plot(f,Fb2);grid;
```

5.4.2 Implementarea FN-FIR prin metoda eşantionării în frecvenţă

Dacă funcția de transfer dorită este $H[k]=[1 \quad 1 \quad 0 \quad 1]$:



funcția de transfer este:

$$H(z)=\frac{1-z^{-N}}{N}\sum_{k=0}^{N-1}\frac{H[k]}{1-e^{jk\frac{2\pi}{N}}z^{-1}}$$

```
r=[1,1,0,1];N=4;
for k=0:N-1
    ka=k+1;p(ka)=exp (j*k*2*pi/N);K(ka)=0;
end
[b,a]=residuez (r,p,k);
bb (1:N)=b (1:N);bbb=[1,0,0,0,-1];
B=conv (bbb,bb);A=N.*a;
H=deconv (B,A);freqz (B,A);
```

Proiectarea unui FN-FIR tip trece jos cu $N=30$ cu caracteristica de frecvență prescrisă:

$$f=[0 \quad 0,6 \quad 0,6 \quad 1] \text{ și } m=[1 \quad 1 \quad 0 \quad 0]$$

Se folosește funcția **`b = fir2(n, f, m)`** unde:

n+1 – reprezintă ordinul filtrului;

f – este un vector cu valorile frecvenței normate cu valorile între 0 și 1, unde 1 corespunde frecvenței Nyquist; prima componentă trebuie să fie 0, iar ultima 1.

m – este un vector cu valorile dorite ale amplitudinii funcției de transfer la frecvențele normate corespunzătoare.

```
f=[0 0.6 0.6 1]; m=[1 1 0 0];
b=fir2(30,f,m);
```

```
subplot(211);stem(b);grid;
[h,w]=freqz(b,1,128);
subplot(212);plot(f/2,m,w/(2*pi),abs (h));grid;
```

Proiectarea unui FN-FIR tip trece sus prin metoda eșantionării în frecvență, plecând de la următoarele date de proiectare:

- ordinul $N = 24$;
- frecvența limită de trecere $f_z = 6kHz$;
- frecvența de eșantionare $f_e = 20kHz$.

```
N=23;ft=6e3;fe=2e4;wn=ft/fe;inc=0.01;f=[0:inc:1];
Hd=[zeros(1,(2*wn)/inc+1)  ones(1,(1-2*wn)/inc)];
subplot(313);
plot(f/2,Hd);grid;
b=fir2(N,f,Hd);[H,W]=freqz (b,1,fe);
subplot (311);
plot(W/(2*pi), 20*log10 (abs(H)));grid;
subplot(312);
plot(W/(2*pi),unwrap(angle(H))*180/pi);grid;
```

Proiectarea FN-FIR tip cosinus ridicat

Se folosește funcția **b=firrcos(n,F$_0$,df,Fs)**, unde:
n+1 – reprezintă ordinul filtrului;
F_0 – frecvența de tăiere [*Hz*];
df – banda de tranziție [*Hz*];
Fs – frecvența de eșantionare.

Exemplu: un FN-FIR cosinus ridicat de ordinul $N = 20$ cu $f_0 = 0,25$ (frecvența de tăiere).

```
h=firrcos (20, 0.25,0.025,1);H=fft(h,1024);
plot((0:511)/1024,abs(H(1:512)));grid;
```

5.4.3 Implementarea FN-FIR prin optimizare folosind algoritmul Remez elaborat de Parks-McClellan

Se folosește funcția **`b=remez(n,f,a,w)`** unde:
n+1 – reprezintă ordinul filtrului;
f, a – caracteristica de frecvență impusă;
w – vector pondere;

Exemplu: proiectarea unui FTB numeric de gradul $N=16$ specificat prin:

```
f=[0 0.3 0.4 0.6 0.7 1];
a=[0 0 1 1 0 0];
b=remez(16,f,a);
[h,w]=freqz (b,1,512);
plot (f,a,w/pi,abs(h));grid;
```

Proiectarea FN-FIR prin optimizare folosind minimizarea erorilor pătratice

Se folosește funcția **`b=firls(n,f,a,w,'ftype')`** unde:
n+1 – reprezintă ordinul filtrului;
f, a – perechi de vectori care specifică caracteristica de frecvență prescrisă a filtrului;
w – vector pondere;
'ftype' – 'hilbbert', 'differentiator'.

Exemplu: se proiectează același filtru proiectat prin metoda remez și se compară rezultatele:

```
clear,clc;
f=[0 0.3 0.4 0.6 0.7 1];
a=[0 0 1 1 0 0];
b=firls(17,f,a);
[h,w]=freqz(b,1,512);
plot(f,a,w/pi,abs(h));grid;
```

Implementarea unui filtru FIR pe dsPIC33FJ256GP506

Relația în timp discret care caracterizează structura unui filtru FIR este:

$$y[n]=\sum_{i=0}^{N} x[n-i]\cdot h[i]$$

unde:

$y[n]$ – este semnalul de la ieșirea filtrului;

$x[n]$ – este semnalul de la intrarea filtrului;

$h[i]$ – sunt coeficienții filtrului;

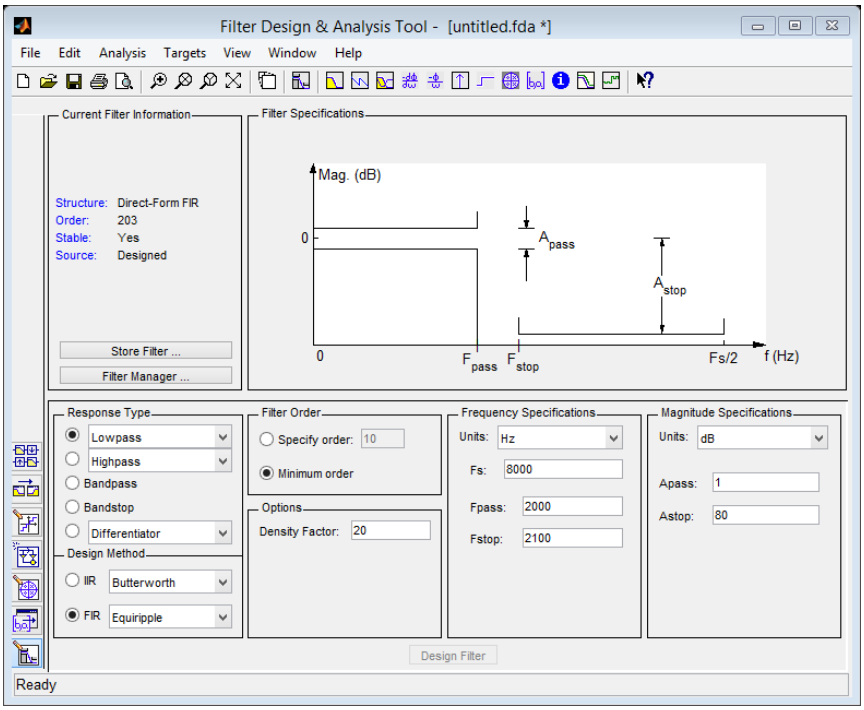
N – este ordinul filtrului.

Așa cum se poate observa, eșantionul semnalului de la ieșire la momentul de timp curent se calculează prin combinația liniară a eșantioanelor semnalului de la intrare. Deci, la fiecare moment de timp, trebuie calculată o sumă de produse, adică trebuie executate $N+1$ înmulțiri și N adunări, operanzii constând în istoria semnalului de la intrare pe de o parte și coeficienții filtrului pe de altă parte.

Se va implementa un filtru trece jos FIR, având frecvența de tăiere la $1kHz$. Pentru comparație, se va proiecta filtrul atât în MATLAB, cât și în dsPIC FD Lite.

Pentru a proiecta filtre în MATLAB, se folosește aplicația *Filter Design and Analysis*, care se poate accesa folosind comanda `>> fdatool`. Aceasta permite proiectarea filtrelor numerice, oferind o diversitate de opțiuni, cum ar fi: alegerea tipului filtrului, a ordinului său, stabilirea frecvențelor limită, a structurii filtrului, a modului de generare a coeficienților ș.a. De asemenea, prezintă grafic răspunsul filtrului proiectat, dar și polii/zerourile funcției de transfer.

După deschiderea aplicației, se realizează configurările din figură:



Înainte de a apăsa pe butonul *Design Filter*, se accesează *Set quantization parameters*, de pe coloana din stânga, și se alege *Filter Arithmetic – Fixed-point*. După ce se dă comanda *Design Filter*, se va putea observa răspunsul filtrului în funcție de frecvență.

Fișierul de coeficienți se obține astfel: *File → Export → Export To – Coefficient File (ASCII) → Format – Hexadecimal → Export*, și se alege locația în care se salvează.

Următorul pas îl reprezintă generarea fișierului de coeficienți în formatul utilizat de funcția *FIR* a bibliotecii dedicată pentru aplicații de procesare digitală de semnal a MPLAB-ului. Se va folosi funcția disponibilă în anexă:

```
function [] = filter_file(filter_type,coeffs_File,memory)
```

Funcția are următorii parametri:

- *filter_type* – tipul filtrului proiectat (’*ftj*’, ’*fts*’, ’*ftb*’, ’*fob*’);
- *coeffs_file* – numele fișierului de coeficienți generat anterior (de exemplu, ’*untitled.fcf*’);
- *memory* – memoria în care vor fi stocați coeficienții (memoria RAM - ’*ram*’, sau memoria de program - ’*program*’).

Se va apela funcția astfel:

```
>> filter_file('ftj','untitled.fcf','program')
```

Se va genera fișierul *lowpass_psv.s*, care se va copia în proiectul nostru, în directorul *src*.

Proiectul va conține fișierele antet *dsp.h* și *dspcommon.inc*, și fișierele sursă *fir.s*, *firdelay.s* și cel furnizat de programul MATLAB. Fișierele *.s* se găsesc în *C:\Microchip Starter Kits\dsPIC Starter Kit - 1\MPLAB C30\src\dsp\asm*.

Rutinele scrise în limbaj de asamblare operează cu o structură numită *FIRStruct*, descrisă în *dsp.h*. Aceasta conține următoarele variabile:

- *numCoeffs* – numărul de coeficienți ai filtrului proiectat;
- *coeffsBase* – pointer către vectorul de coeficienți;
- *coeffsEnd* – pointer către ultimul element din vectorul de coeficienți;
- *coeffsPage* - variabilă care indică locația în care sunt stocați coeficienții filtrului, în memoria de date *X* (caz în care, se introduce constanta COEFS_IN_DATA, definită în *dsp.h*) sau în memoria de program (în acest caz se introduce numărul paginii la care se află aceștia);
- *delayBase* – pointer către buffer-ul de întârziere;
- *delayEnd* – pointer către ultimul element din buffer-ul de întârziere;
- *delay* – valoarea curentă a pointer-ului de întârziere.

În procesul de filtrare trebuie să se țină cont de eșantioanele filtrate și cele care urmează să fie filtrate. Acest lucru se realizează folosind o secvență adițională, numită *delay* (întârziere). Înainte de filtrare, secvența respectivă conține eșantioanele unei secvențe filtrate anterior. După operația de filtrare, secvența conține ultimele eșantioane filtrate. Pentru operare corectă, trebuie ca această secvență să fie inițializată cu 0. Acest lucru se realizează folosind funcția

FIRDelayInit, care acceptă ca argument pointer-ul către structura FIR definită cu funcția *FIRStruct*.

Funcția care realizează efectiv filtrarea este *FIR*, și are următoarele argumente:

- *numSamps* – numărul de eșantioane de intrare (care trebuie filtrate);
- *dstSamps* – pointer către vectorul de ieșire, ce va conține eșantioanele semnalului rezultat în urma operației de filtrare;
- *srcSamps* – pointer către vectorul de intrare, ce conține eșantioanele semnalului supus filtrării;
- *filter* – pointer către structura FIR definită cu funcția *FIRStruct*.

Pentru această aplicație se va filtra un semnal dreptunghiular, de frecvență 1*kHz*, eșantionat cu frecvența de 20*kHz*, generat folosind programul dsPICworks. Pentru a nu exista probleme la compilare, se adaugă în arborele de căutare locația *D:\Projects\h*, iar în arborele de căutare pentru fișiere scrise în limbaj de asamblare (*Assembler Include Search Path*), locația *D:\Projects\src*.

Programul principal:

```
#include "..\h\p33FJ256GP506.h"
#include "..\h\dsp.h"

_FGS(GWRP_OFF & GCP_OFF); //Fara protectie la scriere si fara
protectia codului
_FOSCSEL(FNOSC_FRC); //Se foloseste oscilatorul RC rapid
(Fast RC)
_FOSC(FCKSM_CSECMD & OSCIOFNC_ON & POSCMD_NONE); //Se
activeaza comutarea ceasurilor, OSC2 ca pin digital, se
dezactiveaza oscilatorul primar
_FWDT(FWDTEN_OFF); //Se opreste cronometrul watchdog

#define BLOCK_LENGTH 256 //Marimea secventei care va fi
filtrata
#define LOWPASSFILTER
//#define HIGHPASSFILTER
//#define BANDPASSFILTER
//#define BANDSTOPFILTER
#define FILTERCOEFFS_IN_PROGMEM //Se comenteaza aceasta linie
daca coeficientii filtrului sunt stocati in memoria RAM

//Secventa de intrare - semnal dreptunghiular, de frecventa
1kHz, esantionat cu frecventa de 20kHz
```

```
fractional square1k[256] =
{0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001
};

//Secventa de iesire (secventa de intrare filtrata)
fractional FilterOut[BLOCK_LENGTH];

#ifdef LOWPASSFILTER
        #ifdef FILTERCOEFFS_IN_PROGMEM
                extern FIRStruct lowpass_psvFilter;
//Structura de filtru trece jos, cu coeficientii salvati in
memoria de program
        #else
                extern FIRStruct lowpassFilter;  //Structura
de filtru trece jos, cu coeficientii salvati in memoria RAM
        #endif
#endif
```

```
#ifdef HIGHPASSFILTER
        #ifdef FILTERCOEFFS_IN_PROGMEM
                extern FIRStruct highpass_psvFilter;
//Structura de filtru trece sus, cu coeficientii salvati in
memoria de program
        #else
                extern FIRStruct highpassFilter; //Structura
de filtru trece sus, cu coeficientii salvati in memoria RAM
        #endif
#endif
#ifdef BANDPASSFILTER
        #ifdef FILTERCOEFFS_IN_PROGMEM
                extern FIRStruct bandpassexample_psvFilter;
//Structura de filtru trece banda, cu coeficientii salvati in
memoria de program
        #else
                extern FIRStruct bandpassexampleFilter;
//Structura de filtru trece banda, cu coeficientii salvati in
memoria RAM
        #endif
#endif
#ifdef BANDSTOPFILTER
        #ifdef FILTERCOEFFS_IN_PROGMEM
                extern FIRStruct bandstop_psvFilter;
//Structura de filtru opreste banda, cu coeficientii salvati
in memoria de program
        #else
                extern FIRStruct bandstopFilter; //Structura
de filtru opreste banda, cu coeficientii salvati in memoria
RAM
        #endif
#endif

int main(void) {

                //Configurarea oscilatorului astfel incat
dispozitivul sa opereze la 40MHz
                //Fosc= Fin*M/(N1*N2), Fcy=Fosc/2
                //Fosc= 7.37M*40/(2*2)=80Mhz pentru tact
intrare de 7.37M
                PLLFBD=41;                      //M=39
                CLKDIVbits.PLLPOST=0;           //N1=2
                CLKDIVbits.PLLPRE=0;            //N2=2
                OSCTUN=0;

                __builtin_write_OSCCONH(0x01);
        //Initializare oscilator la FRC (Fast RC) cu PLL
                __builtin_write_OSCCONL(0x01);
```

```
                while (OSCCONbits.COSC != 0b01);
        //Astepta realizarea comutarii
                while(!OSCCONbits.LOCK);

        //Realizarea filtrarii
        #ifdef LOWPASSFILTER
                #ifdef FILTERCOEFFS_IN_PROGMEM
                //Initializarea secventei delay
                        FIRDelayInit(&lowpass_psvFilter);
                //Filtrarea secventei de intrare

FIR(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&lowpass_psvFilte
r);
                #else
                //Initializarea secventei delay
                        FIRDelayInit(&lowpassexampleFilter);
                //Filtrarea secventei de intrare

FIR(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&lowpassFilter);
                #endif
               #endif
               #ifdef HIGHPASSFILTER
                #ifdef FILTERCOEFFS_IN_PROGMEM
                //Initializarea secventei delay
                        FIRDelayInit(&highpass_psvFilter);
                //Filtrarea secventei de intrare

FIR(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&highpass_psvFilt
er);
                #else
                //Initializarea secventei delay
                        FIRDelayInit(&highpassFilter);
                //Filtrarea secventei de intrare

FIR(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&highpassFilter);
                #endif
               #endif
        #ifdef BANDPASSFILTER
                #ifdef FILTERCOEFFS_IN_PROGMEM
                //Initializarea secventei delay
                        FIRDelayInit(&bandpass_psvFilter);
                //Filtrarea secventei de intrare

FIR(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&bandpass_psvFilt
er);
                #else
                //Initializarea secventei delay
                        FIRDelayInit(&bandpassFilter);
```

```
                                //Filtrarea secventei de
intrare

FIR(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&bandpassFilter);
                #endif
               #endif
               #ifdef BANDSTOPFILTER
                #ifdef FILTERCOEFFS_IN_PROGMEM
                //Initializarea secventei delay
                        FIRDelayInit(&bandstop_psvFilter);
                //Filtrarea secventei de intrare

FIR(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&bandstop_psvFilt
er);
                #else
                //Initializarea secventei delay
                        FIRDelayInit(&bandstopFilter);
                //Filtrarea secventei de intrare

FIR(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&bandstopFilter);
                #endif
        #endif

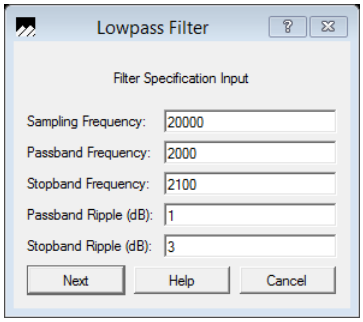
        //Bucla de program
        while (1);}
```

Se vor vizualiza secvențele de intrare și de ieșire.

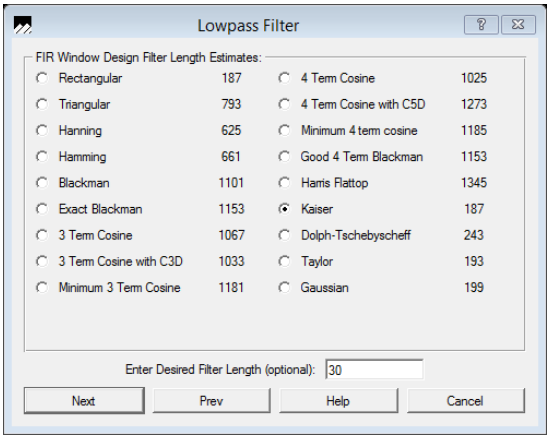
Programul principal este conceput utilizând directive de preprocesare, astfel încât să permită folosirea tuturor tipurilor de filtre. De exemplu, dacă se dorește utilizarea unui filtru trece bandă, atunci se urmează pașii:

- se proiectează filtrul în MATLAB sau dsPIC FD Lite și se generează fișierul de coeficienți în forma necesară funcțiilor din MPLAB;
- se adaugă fișierul respectiv proiectului;
- se comentează liniile *#define LOWPASSFILTER, #define HIGHPASSFILTER* și *#define BANDSTOPFILTER*;
- se scoate de sub comentariu linia *#define BANDPASSFILTER*;
- dacă se dorește salvarea coeficienților în memoria RAM, atunci se comentează și linia *#define FILTERCOEFFS_IN_PROGMEM*.

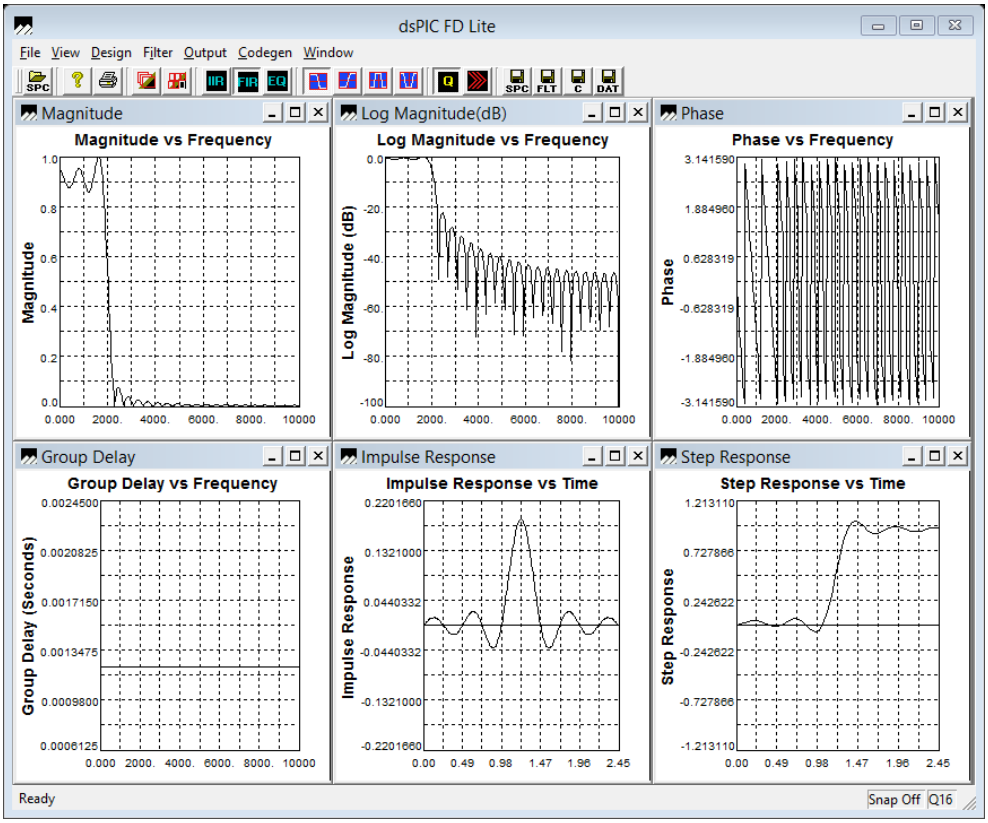
În continuare este prezentată procedura de proiectare a unui filtru folosind programul dsPIC FD Lite. După deschidere, se urmează secvența: *Design → FIR Window Design → Lowpass* și se introduc datele, ca în figura de mai jos:



Se alege tipul ferestrei și/sau se poate specifica ordinul filtrului. Am ales ordinul filtrului 30.



Următoarea fereastră prezintă grafic răspunsul filtrului și variația altor parametri în funcție de frecvență:



Pentru generarea fișierului cu coeficienți se selectează: *Codegen* → *Microchip* → *dsPIC30* → *Program Space* → se debifează *C Header File and Sample Calling Sequence (.h)* și *C Wrapper*. Se va salva fișierul cu denumirea *lowpass_psv*. Dacă coeficienții ar fi fost stocați în RAM, se aleagea denumirea *lowpass*. În mod asemănător, se procedează și pentru alte tipuri de filtre, cu mențiunea că se folosesc denumirile *highpass*, *passband*, respectiv *stopband*.

Fișierul generat de programul dsPIC FD Lite se adaugă proiectului, la fel cum s-a procedat în cazul proiectării folosind MATLAB.

5.5 Desfăşurarea lucrării

1. Se vor rula în MATLAB toate aplicațiile descrise în breviarul teoretic și se va urmări execuția lor.

2. Se va implementa pe dsPIC33FJ256GP506 un FTB cu banda $[800,1000]Hz$, frecvența de eșantionare 8*kHz*, gradul 511, folosind metoda ferestrelor.

3. Să se implementeze pe procesorul dsPIC33FJ256GP506 un FTB cu banda $[2000,3000]Hz$, frecvența de eșantionare 32*kHz*, gradul 399, folosind metoda ferestrelor.

4. Să se proiecteze un FTJ cu frecvența de tăiere 2000*Hz*, frecvența de eșantionare 18,9*kHz*, gradul 300, folosind metoda eșantionării în frecvență. Să se implementeze pe dsPIC33FJ256GP506.

5. Să se proiecteze un FTS cu frecvența de tăiere 10*kHz*, frecvența de eșantionare 48*kHz*, gradul 250, prin optimizare folosind algoritmul *Remez*.

6 Proiectarea şi implementarea filtrelor IIR

6.1 Scopul lucrării

Lucrarea are ca scop studiul, respectiv proiectarea filtrelor IIR (*Infinite Impulse Response*) cu ajutorul programelor MATLAB şi dsPIC FD Lite. Se va realiza şi implementarea filtrelor pe dsPIC33FJ256GP506.

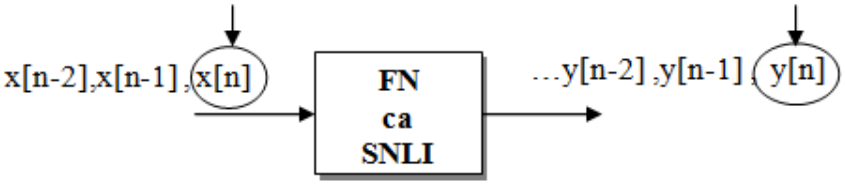
6.2 Noţiuni teoretice

Filtrul numeric este un sistem (în timp) discret caracterizat printr-un *algoritm numeric* cu ajutorul căruia o secvenţă de intrare se transformă într-o secvenţă (filtrată) de ieşire.

Algoritmul (de filtrare) poate să reprezinte: o filtrare trece jos, trece sus, trece bandă, opreşte bandă, sau o operaţie de diferenţiere, integrare etc.

6.2.1 Funcţia de transfer a unui filtru numeric

Să considerăm schema generală de filtrare a unei secvenţe de intrare $x[n]$ într-o secvenţă de ieşire $y[n]$:



Un FN poate fi caracterizat de ecuaţia în timp discret:

$$a_0 y[n] + a_1 y[n-1] + a_2 y[n-1] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2]$$

Dacă $a_0 = 1$, rezultă că secvenţa de ieşire este dată de relaţia:

$$y[n] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2] - a_1 y[n-1] - a_2 y[n-1]$$

sau:

$$y_n = b_0 x_n + b_1 x_{n-1} + b_2 x_{n-2} - a_1 y_{n-1} - a_2 y_{n-2}$$

Aplicând transformata *Z* ambilor membri rezultă că:

$$Y(z) = b_0 X(z) + b_1 z^{-1} X(z) + b_2 z^{-2} X(z) - a_1 z^{-1} Y(z) - a_2 z^{-2} Y(z)$$

Funcția de transfer a filtrului numeric considerat are expresia:

$$H(z) = \frac{Y(z)}{X(z)} = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}$$

6.3 FN cu răspuns infinit la impuls

Să considerăm schema generală a unui filtru numeric de tip IIR:



În timp discret, putem nota:

$$y[n] = b_0 x[n] + b_1 x[n-1] + ... + b_M x[n-M] - a_1 y[n-1] - ... - a_N y[n-N]$$

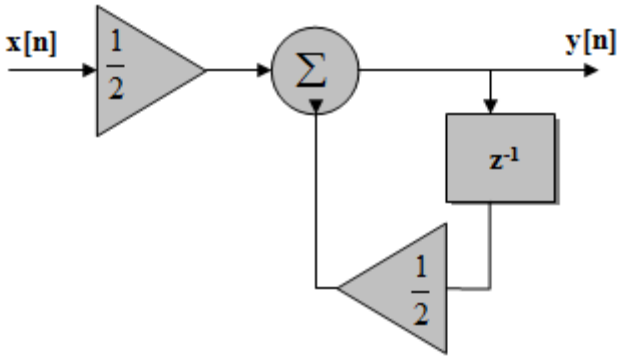
În planul variabilei *z*, funcția de transfer a FN de tip FIR este:

$$H_{FIR}(z) = \frac{Y(z)}{X(z)} = \frac{Z\{y[n]\}}{Z\{x[n]\}} = \frac{\sum_{i=0}^{M} b_i z^{-i}}{1 + \sum_{i=0}^{N} a_i z^{-i}}$$

Răspunsul pondere $h_{IIR}[n]$ și funcția de transfer $H_{IIR}(z)$ se obțin prin aplicarea transformatei *Z* directe/inversă:

$$h_{IIR}[n] \overset{Z}{\leftrightarrow} H_{IIR}(z)$$

Exemplu de FN de tip IIR



Rezultă că $y[n]=\frac{1}{2}x[n]+\frac{1}{2}y[n-1]$, astfel că $Y(z)=\frac{1}{2}X(z)+\frac{1}{2}z^{-1}Y(z)$.

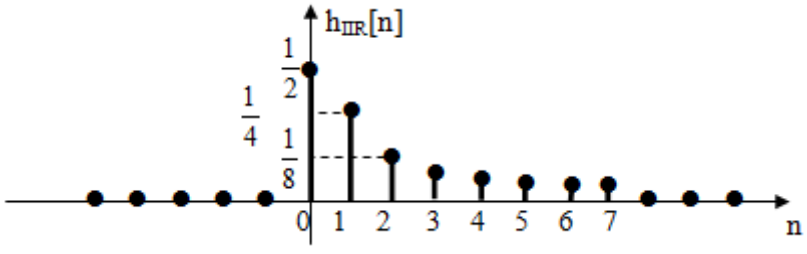
Funcția de transfer a FN-IIR din figură are expresia:

$$H_{FIR}(z)=\frac{Y(z)}{X(z)}=\frac{\frac{1}{2}}{1-\frac{1}{2}z^{-1}}=\frac{1}{2-z^{-1}}$$

iar răspunsul pondere este:

$$h_{IIR}[n]=Z^{-1}\{H_{IIR}(z)\}=Z^{-1}\left\{\frac{\frac{1}{2}}{1-\frac{1}{2}z^{-1}}\right\}=\frac{1}{2}\left(\frac{1}{2}\right)^{n}\cdot u[n]=\left(\frac{1}{2}\right)^{n+1}\cdot u[n]$$

Reprezentarea grafică a răspunsului pondere este:



6.3.1 Proiectarea FN-IIR prin metoda aproximării numerice a ecuației diferențiale ce caracterizează un FA

Plecând de la funcția de transfer a FA, se obține, prin transformare de frecvență, funcția de transfer $H_{IIR}(z)$ a FN:

$$H_a(s)\big|_{s\to\frac{1-z^{-1}}{T}} \to H_{IIR}(z)$$

6.3.2 Proiectarea FN-IIR prin metoda invarianței la impuls unitate

Algoritm:

1. Se calculează $H_a(s)$ a FA prototip;
2. Se determină $h_a(t)=TL^{-1}\{H_a(s)\}$;
3. Se determină răspunsul pondere a FN astfel încât: $h_a(nT)\big|_{T=1}=h[n]$;
4. Se calculează funcția de transfer a FN: $H_d(z)=Z\{h[n]\}$.

6.3.3 Proiectarea FN-IIR prin metoda transformării biliniare

Algoritm:

1. Se determină $H_a(s)$ a FA prototip;
2. Se calculează caracteristica de frecvență a FA:

$$|H_a(j\Omega)| \text{ sau } a(\Omega)=20\lg|H_a(j\Omega)|^{-1}$$

3. Se determină $H_{IIR}(s)$ a FN utilizând transformarea biliniară:

$$H_{IIR}(z)=H_a(s)\Big|_{s\to\frac{2}{T}\frac{1-z^{-1}}{1+z^{-1}}}$$

4. Se calculează caracteristica de frecvență a FN, care se compară cu cea a FA prototip (și cu datele prescrise FN).

6.3.4 Proiectarea FN-IIR prin metode de optimizare

Fie funcția de transfer a unui FN-IIR de ordinul întâi:

$$H(z)=\frac{b_0+b_1z^{-1}}{1+a_1z^{-1}}$$

Valorile parametrilor b_0 ,b_1 ,a_1 se vor determina prin minimizarea funcției criteriu global:

$$\sum_{m=1}^{NF}\left\{\left|H(e^{j\omega_m})\right|-\left|H_{dorit}(e^{j\omega_m})\right|\right\}^2=\min$$

6.4 Aplicații FN-IIR în MATLAB

Comenzi MATLAB pentru proiectarea FN-IIR

$$\left.\begin{matrix}[b,a]\\[z,p,k]\end{matrix}\right\}=\text{butter}(n,Wn,options)$$

$$\left.\begin{matrix}[b,a]\\[z,p,k]\end{matrix}\right\}=\text{cheby1}(n,Rp,Wn,options)$$

$$\left.\begin{matrix}[b,a]\\[z,p,k]\end{matrix}\right\}=\text{cheby2}(n,Rp,Rs,Wn,options)$$

$$\left.\begin{matrix}[b,a]\\[z,p,k]\end{matrix}\right\}=\text{ellip}(n,Rp,Rs,Wn,options)$$

unde $[b,a]$ sunt coeficienții funcției de transfer, iar $[z, p, k]$ sunt zerourile (z), polii (p), respectiv amplificarea (k).

Funcții MATLAB pentru determinarea ordinului (n) și frecvența de tăiere (W_n) pentru un filtru:

$$[n,Wn]=\left\{\begin{matrix}buttord\\cheb1ord\\cheb2ord\\ellipord\end{matrix}\right\}(Wp,Ws,Rp,Rs)$$

unde:

- W_p este frecvența de tăiere pentru banda de trecere; este un scalar sau un vector de două componente cu valorile cuprinse între 0 și 1, unde 1 corespunde frecvenței Nyquist;

- W_s este frecvența de tăiere pentru banda de oprire; este un scalar sau un vector de două componente cu valorile cuprinse între 0 și 1, unde 1 corespunde frecvenței Nyquist;
- R_p – riplul maxim în banda de trecere în *dB*;
- R_s – atenuarea minimă în banda de oprire în *dB*.

Exemplu de proiectare a unui FA prototip

Proiectați un filtru analogic trece bandă (FA-FTB) Chebyshev tip I, de ordinul $n=10$ cu $R_P=3dB$, $F_1=0,1$ ($\Omega_1=2\pi F_1=0,2\pi$) și $F_2=0,5$ ($\Omega_2=\pi$).

```
% determinarea coeficientilor functiei de transfer
% a FTJ analogic cu frecventa de taiere unitate
[b,a]=cheby1(10,3,1,'s');
% reprezentare grafica a modulului si fazei
figure(1);
freqs(b,a);
w1=0.2*pi;w2=pi;Bw=w2-w1;w0=sqrt(w1*w2);
% translatare de frecventa (vezi HELP lp2bp)
[bt,at]=lp2bp(b,a,w0,Bw);
w=linspace(0.01,1,500)*2*pi;
H=freqs(bt,at,w);
figure(2);
% reprezentare grafica a modulului functiei de transfer
semilogy(w/2/pi,abs(H)); grid;
title('Caracteristica de transfer');
xlabel('Frecventa normata');
ylabel('Amplitudine');
```

6.4.1 Implementarea FN prin aproximare în planul z

a) Proiectați un FN-FIR de tip trece bandă cu aproximare Butterworth, care să satisfacă următoarele specificații:

- $R_P=1dB$ pentru $300<f<500Hz$;
- $A_s=20dB$ pentru $f<200Hz$ și $f>650Hz$;
- frecvența de eșantionare este $f_s=1600\text{Hz}$;

Alegeți cel mai mic grad care satisface aceste condiții.

Secvența MATLAB:

```
Fs=1600; %frecventa de esantionare
Fn=Fs/2; %frecventa Nyquist
%frecventele normate
Wp = [300 500]/Fn; Ws = [200 650]/Fn;
Rp = 1; Rs = 20;
%determinare ordin minim filtru
[n,Wn] = buttord(Wp,Ws,Rp,Rs);
%determinarea coeficientilor filtrului
[b,a] = butter(n,Wn);
%raspunsul la impuls
h=impz(b,a);
stem(h);grid
title('Raspunsul la impuls');
%caracteristici de transfer
figure
[H,W]=freqz(b,a);
subplot(211)
plot(W/(2*pi)*Fs, abs(H));grid
title('Caracteristica de transfer');
xlabel('Frecventa [Hz]');
ylabel('Amplitudine')
subplot(212)
plot(W/(2*pi)*Fs, unwrap(angle(H))*180/pi);grid
title('Caracteristica de faza');
xlabel('Frecventa [Hz]');
ylabel('Faza [grade]');
```

Proiectați un FN tip FTB cu banda de trecere $[1000, 2000]Hz$, benzile de blocare $[0,500]Hz$ și $[2500,-]Hz$, iar $R_P = 1dB$, $R_S = 60dB$, cu $f_e = 10kHz$.

```
fe=10e3; %frecventa de esantionare
fn=fe/2; %frecventa Nyquist
Wp=[1000, 2000]/fn; %banda de trecere
Ws=[500,2500]/fn; %frecventele de taiere
Rp=1; Rs=60;
% determinarea ordinului filtrului
% si frecventa de taiere
[n,Wn]=buttord(Wp, Ws, Rp, Rs);
%calculul coeficientilor filtrului
[b,a]=butter(n,Wn);
% (sau:=)
%[n,Wn]=ellipord(Wp, Ws, Rp, Rs)
%[b,a]=ellip(n,Rp,Rs,Wn)
[H,W]=freqz(b,a);
subplot(211);
plot(W/(2*pi), abs(H));grid;
```

```
title('Caracteristica de transfer');
xlabel('Frecventa normata');
ylabel('Amplitudine');
subplot(212);
plot(W/(2*pi), unwrap(angle(H))*180/pi);grid;
title('Caracteristica de faza');
xlabel('Frecventa normata');
ylabel('Faza [grade]');
```

Să se ruleze programul și pentru filtru de tip eliptic și să se precizeze diferențele.

6.4.2 Implementarea FN-IIR prin metoda invarianței la impuls

Să se proiecteze un filtru FTB discret de tip Chebyshev I, prin metoda invarianței la impuls, având riplul în banda de trecere de maxim $3dB$, atenuarea în banda de oprire de minim $20dB$, $F_s = 2$, iar frecvențele corespunzătoare capetelor benzilor sunt:

$$F_{p1} = 0,3,\ F_{p2} = 0,4,\ F_{s1} = 0,2,\ F_{s2} = 0,5.$$

Se folosește funcția `[numd,dend]=impinvar(num,den,Fs)`, care creează un filtru digital având coeficienții numărătorului și numitorului returnați în vectorii *numd*, respectiv *dend*. Răspunsul la impuls al filtrului numeric se obține prin eșantionarea filtrului analogic, ai cărui coeficienți sunt conținuți în vectorii *num*, respectiv *den*, cu frecvența de eșantionare *Fs* (*Hz*). Dacă nu se precizează *Fs* atunci aceasta se ia implicit $1Hz$. Această funcție se aplică numai pentru filtre descrise prin funcția de transfer. Metoda invarianței răspunsului la impuls nu este adecvată pentru FTS și FOB.

Secvența MATLAB:

```
Rp=3;Rs=20;
Wp=2*pi*[0.3 0.4];
Ws=2*pi*[0.2 0.5];
% calculul ordinului si a frecventei de taiere
% a filtrului analogic
[n,Wn]=cheb1ord(Wp,Ws,Rp,Rs,'s');
% proiectarea filtrului analogic prototip
[z,p,k]=cheb1ap(n,Rp); %vezi help cheb1ap
% determinare coeficienti functie de transfer
[num,den]=zp2tf(z,p,k); %vezi help zp2tf
Wo=sqrt(Wn(1)*Wn(2));
Bw=Wn(2)-Wn(1);
```

```
% translatare de frecventa (vezi HELP lp2bp)
[numt,dent]=lp2bp(num,den,Wo,Bw);
w=linspace(0,1,500)*2*pi;
H=freqs(numt,dent,w);
plot(w/2/pi,20*log10(abs(H)), 'r');
ylabel('Amplitudine (dB)');
hold on;

% proiectare FTB discret prin metoda
% invariantei la impuls Fs=2
[bz,az]=impinvar(numt,dent,2);
[Hd,Wd]=freqz(bz,az);
plot(Wd/(pi), 20*log10(abs(Hd)), 'b'); grid;
xlabel('frecventa normata la frecventa Nyquist');
```

6.4.3 Implementarea FN-IIR prin transformarea biliniară

Să se proiecteze un filtru FTB discret de tip Chebyshev I, prin metoda transformării biliniare, având riplul în banda de trecere de maxim 3*dB*, atenuarea în banda de oprire de minim 20*dB*, iar frecvențele discrete corespunzătoare capetelor benzilor sunt:

$$F_{p1} = 0,2,\ F_{p2} = 0,25,\ F_{s1} = 0,15,\ F_{s2} = 0,3$$

Frecvența de eșantionare este $F_S = 2$.

Se folosește funcția `[numd,dend]=bilinear(num,den,Fs)`, unde:

- *num* și *den* sunt vectori linie ce conțin coeficienții funcției de transfer ai filtrului analogic;
- *numd* și *dend* sunt coeficienții funcției de transfer ai filtrului digital.

Secvența MATLAB:

```
wp=2*pi*[0.1 0.15];
ws=2*pi*[0.05 0.2];
Rp=3;Rs=20;Fs=2;
%specificarea parametrilor filtrului analogic
Wp=wp*Fs;
Ws=ws*Fs;
%calculul ordinului si a frecventei de
%taiere a filtrului analogic
[n,Wn]=cheb1ord(Wp,Ws,Rp,Rs,'s');
%proiectarea filtrului analogic prototip
[z,p,k]=cheb1ap(n,Rp);
[num,den]=zp2tf(z,p,k); %vezi help zp2tf
%translarea de frecventa
```

```
Wo=sqrt(Wn(1)*Wn(2));
Bw=Wn(2)-Wn(1);
[numt,dent]=lp2bp(num,den,Wo,Bw);
%raspunsul in frecventa al filtrului analogic
w=linspace(0,1,500)*2*pi;
h=freqs(numt,dent,w);
%proiectarea filtrului digital Fs=2
[bz,az]=bilinear(numt,dent,Fs);
%raspunsul in frecventa al filtrului
[hd,wd]=freqz(bz,az);
%reprezentarea grafica
figure(1);
plot(wd/pi,20*log10(abs(hd)),'r',w/2/pi,20*log10(abs(h)
),'b'); grid;
xlabel('F (Hz) pt analogic \omega/\pi pt. discret');
ylabel('Amplitudine (db)');
legend('filtru discret','filtru analogic');
```

După cum era de așteptat, frecvența benzii de trecere și a benzii de oprire ale filtrului discret nu coincid cu cele specificate. Acest lucru se datorează neliniarității transformării biliniare.

6.4.4 Implementarea FN-IIR prin optimizarea în planul *z*

Proiectarea unui FN-IIR care aproximează în sensul erorilor pătratice minime o caracteristică de frecvență folosește funcția:

```
[b,a]=yulewalk(n,f,m)
```

unde:

m - un vector cu valorile amplitudinilor dorite;
f – un vector cu valorile frecvențelor specificate.

Exemplu: Proiectați un FN de gradul $n=8$ de tip trece jos, a cărui caracteristică de frecvență este specificată prin:

```
f=[0 0.6 0.6 1]; m=[1 1 0 0 ];
[b,a]=yulewalk(8,f,m);[h,w]=freqz(b,a);
plot(f,m,'r',w/pi,abs(h),'b'); grid;
legend('caracteristica dorita', 'caracteristica
filtrului proiectat');
xlabel('Frecventa normata la frecventa Nyquist');
ylabel('Castig');
```

6.5 Implementarea unui filtru IIR pe dsPIC33FJ256GP506

Relația în timp discret care caracterizează structura unui filtru FIR este:

$$y[n]=\sum_{i=0}^{M} b_i x[n-i]-\sum_{j=0}^{N} a_j y[n-j]$$

unde:

$y[n]$ – este semnalul de la ieşirea filtrului;

$x[n]$ – este semnalul de la intrarea filtrului;

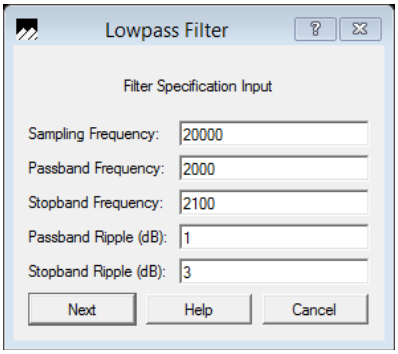
b_i – sunt coeficienții numărătorului funcției de transfer;

a_j – este coeficienții numitorului funcției de transfer.

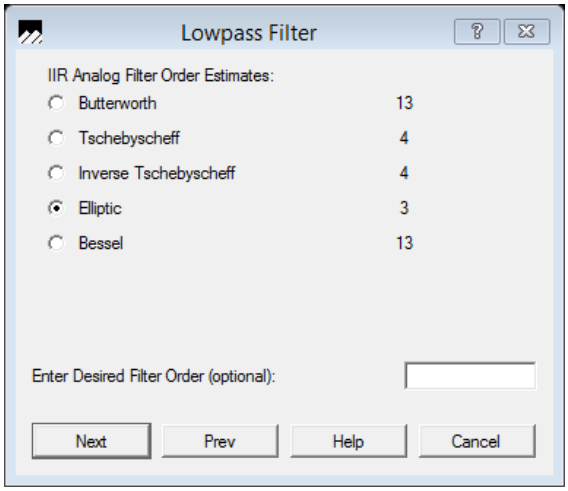
Așa cum se poate observa, eșantionul semnalului de la ieșire la momentul de timp curent se calculează prin combinația liniară a eșantioanelor semnalului de la intrare. Deci, la fiecare moment de timp, trebuie calculată o sumă de produse, adică trebuie executate *M+N+1* înmulțiri și *M+N-1* adunări și o scădere.

Se va implementa un filtru trece jos IIR în formă transpusă, având frecvența de tăiere la 1*kHz*. Se va proiecta filtrul în dsPIC FD Lite.

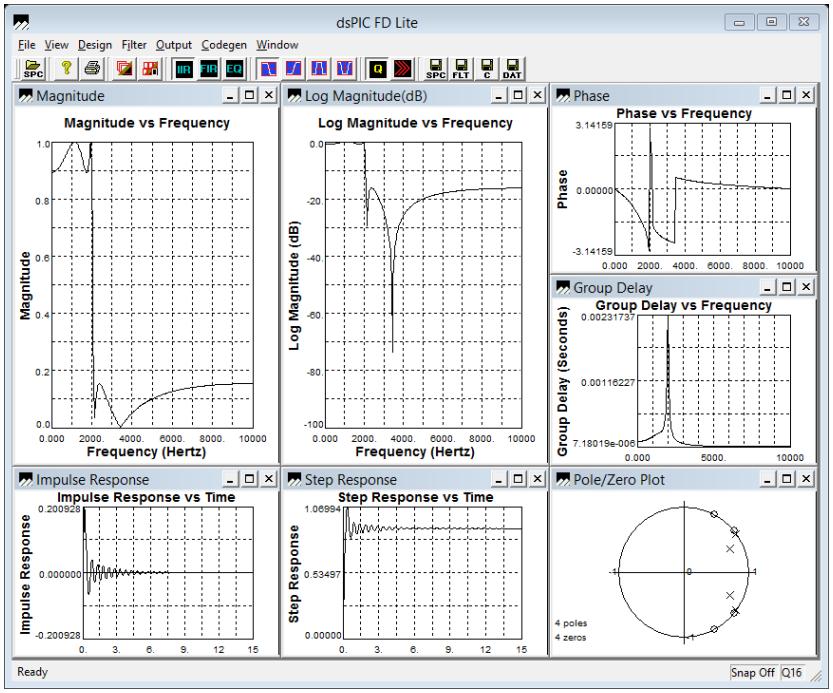
În continuare este prezentată procedura de proiectare a unui filtru folosind programul dsPIC FD Lite. După deschidere, se urmează secvența: *Design* → *IIR Design* → *Lowpass* și se introduc datele, ca în figura de mai jos:



Se alege tipul filtrului și/sau se poate specifica ordinul său. Am ales un filtru eliptic, de ordin 4, aplicația fiind limitată la proiectarea filtrelor IIR de ordin maxim 4.



Următoarea fereastră prezintă grafic răspunsul filtrului și variația altor parametri în funcție de frecvență:



Pentru generarea fișierului cu coeficienți se selectează: *Codegen → Microchip → dsPIC30 → Program Space →* se debifează *C Header File and Sample Calling Sequence (.h)* și *C Wrapper*. Se va salva fișierul cu denumirea *lowpass_psv*. Dacă coeficienții ar fi fost stocați în RAM, se alegea denumirea *lowpass*. În mod asemănător, se procedează și pentru alte tipuri de filtre, cu mențiunea că se folosesc denumirile *highpass, passband,* respectiv *stopband*.

Proiectul va conține fișierele antet *dsp.h* și *dspcommon.inc*, și fișierul sursă *iirtrans.s* și cel furnizat de programul dsPIC FD Lite. Fișierul sursă se găsește în *C:\Microchip Starter Kits\dsPIC Starter Kit - 1\MPLAB C30\src\dsp\asm*.

Rutinele scrise în limbaj de asamblare operează cu o structură numită *IIRTransposedStruct*, descrisă în *dsp.h*. Aceasta conține următoarele variabile:

- *numSectionsLess1* – numărul de secțiuni de ordinul doi cascadate minus una;
- *coeffsBase* – pointer către vectorul de coeficienți;
- *coeffsPage* - variabilă care indică locația în care sunt stocați coeficienții filtrului, în memoria de date *X* (caz în care, se introduce constanta COEFS_IN_DATA, definită în *dsp.h*) sau în memoria de program (în acest caz se introduce numărul paginii la care se află aceștia);
- *delayBase1* – pointer către buffer-ul de întârziere 1;
- *delayBase2* – pointer către buffer-ul de întârziere 2;
- *finalShift* – numărul de biți cu care să se deplaseze biții secvenței de ieșire (implicit este 0; o valoare pozitivă înseamnă deplasare la stânga, iar o valoare negativă înseamnă deplasare la dreapta; această opțiune se folosește pentru scalarea rezultatului, pentru aducerea câștigului filtrului la 1).

În procesul de filtrare trebuie să se țină cont de eșantioanele filtrate și cele care urmează să fie filtrate. Acest lucru se realizează folosind o secvență adițională, numită *delay* (întârziere). Înainte de filtrare, secvența respectivă conține eșantioanele unei secvențe filtrate anterior. După operația de filtrare, secvența conține ultimele eșantioane filtrate. Pentru operare corectă, trebuie ca această secvență să fie inițializată cu 0. Acest lucru se realizează folosind funcția *IIRTransposedInit*, care acceptă ca argument pointer-ul către structura IIR definită cu funcția *IIRTransposedStruct*.

Funcția care realizează efectiv filtrarea este *IIRTransposed*, și are următoarele argumente:

- *numSamps* – numărul de eșantioane de intrare (care trebuie filtrate);
- *dstSamps* – pointer către vectorul de ieșire, ce va conține eșantioanele semnalului rezultat în urma operației de filtrare;
- *srcSamps* – pointer către vectorul de intrare, ce conține eșantioanele semnalului supus filtrării;
- *filter* – pointer către structura IIR definită cu funcția *IIRTransposedStruct*.

Pentru această aplicație se va filtra un semnal dreptunghiular, de frecvență 1*kHz*, eșantionat cu frecvența de 20*kHz*, generat folosind programul dsPICworks. Pentru a nu exista probleme la compilare, se adaugă în arborele de căutare locația *D:\Projects\h*, iar în arborele de căutare pentru fișiere scrise în limbaj de asamblare (*Assembler Include Search Path*), locația *D:\Projects\src*.

Programul principal:

```
#include "..\h\p33FJ256GP506.h"
#include "..\h\dsp.h"

_FGS(GWRP_OFF & GCP_OFF); //Fara protectie la scriere si fara
protectia codului
_FOSCSEL(FNOSC_FRC); //Se foloseste oscilatorul RC rapid
(Fast RC)
_FOSC(FCKSM_CSECMD & OSCIOFNC_ON & POSCMD_NONE); //Se
activeaza comutarea ceasurilor, OSC2 ca pin digital, se
dezactiveaza oscilatorul primar
_FWDT(FWDTEN_OFF); //Se opreste cronometrul watchdog

#define BLOCK_LENGTH 256 //Marimea secventei care va fi
filtrata
#define LOWPASSFILTER
//#define HIGHPASSFILTER
//#define BANDPASSFILTER
//#define BANDSTOPFILTER
#define FILTERCOEFFS_IN_PROGMEM //Se comenteaza aceasta linie
daca coeficientii filtrului sunt stocati in memoria RAM

//Secventa de intrare - semnal dreptunghiular, de frecventa
1kHz, esantionat cu frecventa de 20kHz
fractional square1k[256] =
{0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
```

```
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x8001,0x8001,
0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001,
0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,0x7FFF,
0x7FFF,0x7FFF,0x8001,0x8001,0x8001,0x8001,0x8001,0x8001};

//Secventa de iesire (secventa de intrare filtrata)
fractional FilterOut[BLOCK_LENGTH];

#ifdef LOWPASSFILTER
        #ifdef FILTERCOEFFS_IN_PROGMEM
                extern IIRTransposedStruct lowpass_psvFilter;
//Structura de filtru trece jos, cu coeficientii salvati in
memoria de program
        #else
                extern IIRTransposedStruct lowpass_Filter;
//Structura de filtru trece jos, cu coeficientii salvati in
memoria RAM
        #endif
#endif
#ifdef HIGHPASSFILTER
        #ifdef FILTERCOEFFS_IN_PROGMEM
                extern IIRTransposedStruct
highpass_psvFilter; //Structura de filtru trece sus, cu
coeficientii salvati in memoria de program
        #else
                extern IIRTransposedStruct highpass_Filter;
//Structura de filtru trece sus, cu coeficientii salvati in
memoria RAM
        #endif
#endif
#ifdef BANDPASSFILTER
        #ifdef FILTERCOEFFS_IN_PROGMEM
                extern IIRTransposedStruct
bandpass_psvFilter; //Structura de filtru trece banda, cu
coeficientii salvati in memoria de program
```

```
        #else
                extern IIRTransposedStruct bandpass_Filter;
//Structura de filtru trece banda, cu coeficientii salvati in
memoria RAM
        #endif
#endif
#ifdef BANDSTOPFILTER
        #ifdef FILTERCOEFFS_IN_PROGMEM
                extern IIRTransposedStruct
bandstop_psvFilter; //Structura de filtru opreste banda, cu
coeficientii salvati in memoria de program
        #else
                extern IIRTransposedStruct bandstop_Filter;
//Structura de filtru opreste banda, cu coeficientii salvati
in memoria RAM
        #endif
#endif

int main(void) {

               //Configurarea oscilatorului astfel incat
dispozitivul sa opereze la 40MHz
               //Fosc= Fin*M/(N1*N2), Fcy=Fosc/2
               //Fosc= 7.37M*40/(2*2)=80Mhz pentru tact
intrare de 7.37M
               PLLFBD=41;                      //M=39
               CLKDIVbits.PLLPOST=0;           //N1=2
               CLKDIVbits.PLLPRE=0;            //N2=2
               OSCTUN=0;

               __builtin_write_OSCCONH(0x01);
        //Initializare oscilator la FRC (Fast RC) cu PLL
               __builtin_write_OSCCONL(0x01);
               while (OSCCONbits.COSC != 0b01);
        //Astepta realizarea comutarii
               while(!OSCCONbits.LOCK);
         //Realizarea filtrarii
        #ifdef LOWPASSFILTER
                #ifdef FILTERCOEFFS_IN_PROGMEM
                //Initializarea secventei delay

IIRTransposedInit(&lowpass_psvFilter);
                //Filtrarea secventei de intrare

       IIRTransposed(BLOCK_LENGTH,&FilterOut[0],&square1k[0]
,&lowpass_psvFilter);
                #else
                //Initializarea secventei delay
```

```
                        IIRTransposedInit(&lowpass_Filter);
                //Filtrarea secventei de intrare
IIRTransposed(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&lowpas
sFilter);
                #endif
               #endif
               #ifdef HIGHPASSFILTER
                #ifdef FILTERCOEFFS_IN_PROGMEM
                //Initializarea secventei delay

IIRTransposedInit(&highpass_psvFilter);
                //Filtrarea secventei de intrare
IIRTransposed(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&highpa
ss_psvFilter);
                #else
                //Initializarea secventei delay
                        IIRTransposedInit(&highpass_Filter);
                //Filtrarea secventei de intrare

IIRTransposed(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&highpa
ssFilter);
                #endif
               #endif
        #ifdef BANDPASSFILTER
                #ifdef FILTERCOEFFS_IN_PROGMEM
                //Initializarea secventei delay
                      IIRTransposedInit(&bandpass_psvFilter);
                //Filtrarea secventei de intrare

IIRTransposed(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&bandpa
ss_psvFilter);
                #else
                //Initializarea secventei delay

IIRTransposedFilterInit(&bandpass_Filter);
//Filtrarea secventei de intrare

IIRTransposed(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&bandpa
ssFilter);
                #endif
               #endif
               #ifdef BANDSTOPFILTER
                #ifdef FILTERCOEFFS_IN_PROGMEM
                //Initializarea secventei delay

IIRTransposedInit(&bandstop_psvFilter);
                //Filtrarea secventei de intrare
```

```
IIRTransposed(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&bandst
op_psvFilter);
                #else
                //Initializarea secventei delay
IIRTransposedFilterInit(&bandstop_Filter);
                //Filtrarea secventei de intrare

IIRTransposed(BLOCK_LENGTH,&FilterOut[0],&square1k[0],&bandst
opFilter);
                #endif
        #endif

                //Bucla de program
        while (1);}
```

Se vor vizualiza secvențele de intrare și de ieșire.

Programul principal este conceput utilizând directive de preprocesare, astfel încât să permită folosirea tuturor tipurilor de filtre. De exemplu, dacă se dorește utilizarea unui filtru trece bandă, atunci se urmează pașii:

- se proiectează filtrul în dsPIC FD Lite și se generează fișierul de coeficienți în forma necesară funcțiilor din MPLAB;
- se adaugă fișierul respectiv proiectului;
- se comentează liniile *#define LOWPASSFILTER*, *#define HIGHPASSFILTER* și *#define BANDSTOPFILTER*;
- se scoate de sub comentariu linia *#define BANDPASSFILTER*;
- dacă se dorește salvarea coeficienților în memoria RAM, atunci se comentează și linia *#define FILTERCOEFFS_IN_PROGMEM*.

6.6 Desfășurarea lucrării

6. Se vor rula în MATLAB toate aplicațiile descrise în breviarul teoretic și se va urmări execuția lor.

7. Proiectaţi un FN-FIR de tip trece bandă cu aproximare Butterworth care să satisfacă următoarele specificaţii:

- $R_p = 3dB$ pentru $1000 < f < 1300Hz$;
- $A_s = 30dB$ pentru $f < 800Hz$ şi $f > 1500Hz$;
- frecvenţa de eşantionare este $f_s = 8000Hz$.

Alegeţi cel mai mic grad care satisface aceste condiţii.

Să se repete proiectarea aceluiaşi filtru pentru aproximare Chebyshev I, II şi eliptic (MATLAB).

8. Să se proiecteze un filtru FTB discret de tip Butterworth prin metoda invarianţei la impuls, având riplul în banda de trecere de maxim 1*dB*, atenuarea în banda de oprire de minim 40*dB*, F_s=2, iar frecvenţele corespunzătoare capetelor benzilor sunt:

$$F_{p1} = 0,2,\ F_{p2} = 0,35,\ F_{s1} = 0,1,\ F_{s2} = 0,4$$

9. Să se proiecteze un filtru FTB discret de tip Butterworth prin metoda transformării biliniare, având riplul în banda de trecere de maxim 2*dB*, atenuarea în banda de oprire de minim 30*dB*, frecvenţa de eşantionare este Fs=2, iar frecvenţele discrete corespunzătoare capetelor benzilor sunt:

$$F_{p1} = 0,2,\ F_{p2} = 0,35,\ F_{s1} = 0,15,\ F_{s2} = 0,4$$

10. Se va implementa pe dsPIC33FJ256GP506 un filtru notch ce trebuie să anuleze frecvenţa de 1000*Hz*, factorul de calitate Q = 10 şi frecvenţa de eşantionare 8000*kHz*.

Bibliografie

[1] A. Şerbănescu, G. Şerban, G. Iana, O. Teofil, Procesarea digitală a semnalelor – Aplicații și implementări hardware în structuri reconfigurabile și cu procesoare digitale de semnale, Editura Universității din Pitești, 2009

[2] ZACIU, R. , Prelucrarea digitală a semnalelor, Ed. Albastră, Cluj-Napoca, 2003

Kester, W., Mixed-signaland DSP design techniques, Elsevier Science, 2003

[3] MITRA, S. K., Digital Signal Processing, A Computer based Approach, McGRAW-HILL, N.Y., 2001

[4] MATEESCU, A . ş.a., ”Semnale şi sisteme” , Ed.TEORA, Buc., 2001

[5] TĂRNICERIU, D., Bazele prelucrării numerice ale semnalelor Ed. Vasiliana, Iaşi, 2001

[6] QUINQUIS, A., ŞERBĂNESCU A, RĂDOI EMANUEL, Le traitement du signal sous MATLAB, Pratique et Applicatons, Ed. HERMES Science Pub. , Paris, 2000

[7] MATEESCU, AD. ş.a., Prelucrarea Numerică a Semnalelor, Ed.Tehnică, Bucureşti, 1997

[8] LIM, J.S., OPPENHEIM, A.V., Advanced Topics in Signal Processing, Prentice Hall, Englewood Clifs, N.J.,1988

[9] CIOCHINĂ, S., Prelucrarea Numerică a Semnalelor, Partea I, Univ. Politehnica Bucureşti, 1995

[10] GRIGORAŞ,V. ,Prelucrarea Numerică a Semnalelor, TĂRNICERIU, D., Partea I, II, Ed.Gh.Asachi, Iaşi, 1995

[11] NAFORNIȚĂ, I., Semnale,Circuite şi Sisteme, Partea I, CÂMPEANU, A. şi Univ."Politehnica",Timişoara, 1995

[12] LĂZĂRESCU, V., Prelucrarea Digitală a Semnalelor, Ed.Amco Press, Bucureşti, 1994.

[13] Microchip, dsPIC33FJXXXGPX06A/X08A/X10A foaie de catalog, http://ww1.microchip.com/downloads/en/DeviceDoc/70593d.pdf

[14] Microchip, 16-bit MCU and DSC Programmer’s Reference Manual, High-Performance Microcontrollers (MCU) and Digital Signal Controllers (DSC), http://ww1.microchip.com/downloads/en/DeviceDoc/70157F.pdf

[15] Microchip, MPLAB Starter Kit for dsPIC® Digital Signal Controllers User’s Guide, http://ww1.microchip.com/downloads/en/DeviceDoc/51700B.pdf